

1000+ SOFTWARE ENGINEERING INTERVIEW QUESTIONS

Complete Preparation Guide with Answers, Code & Real-World Scenarios

2026 Edition - Freshers to Software Architects

Level: Intern / Fresher to Senior / Tech Lead / Architect

Targets: FAANG, product companies, high-growth startups, service companies

Style: Every question answered from first principles, with runnable code and expected output

HOW TO USE THIS BOOK

1. Read the question only. Answer out loud for two minutes. Time yourself.
2. Then read the answer. Score yourself: knew it / partial / new.
3. Type the code by hand. Do not copy-paste. Run it. Break it on purpose.
4. Answer the follow-ups without looking. These are what the interviewer actually asks next.
5. Night before the interview: read only the Revision Notes at the end of each section.

Difficulty calibration

Difficulty	Expected from	What failure signals

Easy	Intern / fresher	Red flag; usually ends the round
Medium	1-4 years experience	Down-level from mid to junior
Hard	5+ years / senior+	Not a reject alone; judged on reasoning quality

Every question uses this structure

Question and difficulty, Category, Answer, Example, Code Example, Output, Why Interviewers Ask This, Common Mistakes, Best Practices, Follow-up Questions,

Real-world Scenario.

Conventions

- SQL dialect is PostgreSQL 16+ unless the question names another engine.
- JavaScript is ES2024, Node 22 LTS, React 19, TypeScript 5.6+.
- Java is 21 LTS, Spring Boot 3.x. C# is 12 / .NET 8+.
- No proprietary or leaked interview content is reproduced. All questions are written from scratch against standard computer science material and publicly discussed themes.

TABLE OF CONTENTS

Section	Topic	Questions
1	SQL	150
2	MERN Stack (MongoDB, Express, React, Node)	120
3	Internship and Fresher Interviews	100
4	Computer Networks	100
5	System Design	120
6	HTML	60
7	CSS	100
8	JavaScript	150
9	TypeScript	80
10	C# and .NET	100
11	Java	120
12	Linux	100
13	Data Structures and Algorithms	150
14	Object-Oriented Programming	60
15	DBMS	80
16	Operating Systems	80
17	Software Engineering Practice	50
18	Behavioral and HR	100

SECTION 1: SQL

The Sample Database

Every SQL answer in this section runs against this schema. Load it once and every query in the section executes verbatim. Domain: a small e-commerce company.

```
+-----+      +-----+      +-----+
| customers |1-----*| orders  |1-----*| order_items |
+-----+      +-----+      +-----+
                        |1          *|
                        *          1|
                        +-----+      +-----+
                        | payments |    | products  |
                        +-----+      +-----+
                                      *|
                                      1|
                                      +-----+
                                      | categories | (self-referencing tree)
                                      +-----+

+-----+
| employees | manager_id -> employees.employee_id (self-referencing)
+-----+
```

```
-- =====
-- interview_db : sample schema for the SQL section
-- =====
DROP TABLE IF EXISTS payments, order_items, orders, products,
                        categories, customers, employees CASCADE;

CREATE TABLE customers (
  customer_id  SERIAL PRIMARY KEY,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(150) NOT NULL UNIQUE,
  city         VARCHAR(60),
  country      VARCHAR(60) NOT NULL DEFAULT 'India',
  signup_date  DATE         NOT NULL DEFAULT CURRENT_DATE,
  last_support_contact TIMESTAMPTZ -- NULL = never contacted support
);

CREATE TABLE categories (
  category_id  SERIAL PRIMARY KEY,
  name         VARCHAR(60) NOT NULL UNIQUE,
  parent_id    INT REFERENCES categories(category_id)
);

CREATE TABLE products (
  product_id   SERIAL PRIMARY KEY,
```

```

    name          VARCHAR(120) NOT NULL,
    category_id    INT NOT NULL REFERENCES categories(category_id),
    unit_price     NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    stock_qty      INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    discontinued   BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE orders (
    order_id       SERIAL PRIMARY KEY,
    customer_id    INT NOT NULL REFERENCES customers(customer_id),
    order_date     DATE NOT NULL,
    status         VARCHAR(20) NOT NULL
                CHECK (status IN ('PENDING', 'SHIPPED', 'DELIVERED', 'CANCELLED')),
    shipped_date    DATE,                -- NULL until it ships
    discount_pct   NUMERIC(5,2)         -- often NULL
);

CREATE TABLE order_items (
    order_id       INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id     INT NOT NULL REFERENCES products(product_id),
    quantity       INT NOT NULL CHECK (quantity > 0),
    unit_price     NUMERIC(10,2) NOT NULL,    -- price snapshot at purchase time
    PRIMARY KEY (order_id, product_id)        -- composite primary key
);

CREATE TABLE payments (
    payment_id     SERIAL PRIMARY KEY,
    order_id       INT NOT NULL REFERENCES orders(order_id),
    amount         NUMERIC(12,2) NOT NULL,
    method         VARCHAR(20) NOT NULL,      -- UPI / CARD / NETBANKING / COD
    paid_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    balance        NUMERIC(12,2) NOT NULL DEFAULT 0
);

CREATE TABLE employees (
    employee_id    SERIAL PRIMARY KEY,
    full_name      VARCHAR(100) NOT NULL,
    department     VARCHAR(50) NOT NULL,
    salary         NUMERIC(10,2) NOT NULL,
    manager_id     INT REFERENCES employees(employee_id),
    hire_date      DATE NOT NULL
);

-- ----- seed data -----
INSERT INTO customers (full_name, email, city, country, signup_date, last_support_contact) VALUES
('Aarav Sharma', 'aarav@example.com', 'Bengaluru', 'India', '2024-01-15', '2025-03-02
10:12:00+05:30'),
('Diya Patel', 'diya@example.com', 'Mumbai', 'India', '2024-02-20', NULL),
('Rohan Verma', 'rohan@example.com', 'Delhi', 'India', '2024-03-05', NULL),
('Meera Iyer', 'meera@example.com', 'Chennai', 'India', '2024-06-11', '2025-01-19
09:00:00+05:30'),
('John Carter', 'john@example.com', 'Austin', 'USA', '2024-07-30', NULL),
('Sara Khan', 'sara@example.com', NULL, 'UAE', '2025-02-14', NULL);

INSERT INTO categories (name, parent_id) VALUES
('Electronics', NULL), -- 1
('Computers', 1),      -- 2
('Laptops', 2),        -- 3
('Accessories', 2),    -- 4
('Home', NULL),        -- 5
('Kitchen', 5);        -- 6

INSERT INTO products (name, category_id, unit_price, stock_qty, discontinued) VALUES
('UltraBook 14', 3, 89999.00, 12, FALSE),
('ProBook 15', 3, 74999.00, 0, FALSE),
('Mechanical Keyboard', 4, 4999.00, 40, FALSE),
('USB-C Hub', 4, 2499.00, 75, FALSE),
('Wireless Mouse', 4, 1299.00, 5, TRUE),

```

```

('Air Fryer 4L',          6, 6999.00, 20, FALSE),
('Blender Pro',          6, 3499.00, 18, FALSE);

INSERT INTO orders (customer_id, order_date, status, shipped_date, discount_pct) VALUES
(1, '2025-01-10', 'DELIVERED', '2025-01-12', 10.00),
(1, '2025-02-14', 'DELIVERED', '2025-02-16', NULL),
(2, '2025-02-20', 'SHIPPED', '2025-02-21', 5.00),
(3, '2025-03-01', 'CANCELLED', NULL, NULL),
(4, '2025-03-15', 'PENDING', NULL, NULL),
(1, '2025-04-02', 'DELIVERED', '2025-04-04', NULL),
(5, '2025-04-18', 'DELIVERED', '2025-04-20', 15.00),
(2, '2025-05-09', 'PENDING', NULL, NULL);

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 89999.00), (1, 3, 2, 4999.00),
(2, 4, 3, 2499.00),
(3, 6, 1, 6999.00), (3, 7, 1, 3499.00),
(4, 2, 1, 74999.00),
(5, 3, 1, 4999.00),
(6, 1, 1, 89999.00),
(7, 6, 2, 6999.00), (7, 4, 1, 2499.00),
(8, 7, 1, 3499.00);

INSERT INTO payments (order_id, amount, method, paid_at, balance) VALUES
(1, 89996.10, 'UPI', '2025-01-10 11:02:00+05:30', 50000),
(2, 7497.00, 'CARD', '2025-02-14 19:31:00+05:30', 25000),
(3, 9972.60, 'NETBANKING', '2025-02-20 08:45:00+05:30', 10000),
(6, 89999.00, 'UPI', '2025-04-02 13:20:00+05:30', 8000),
(7, 14872.65, 'CARD', '2025-04-18 22:05:00+05:30', 3000);

INSERT INTO employees (full_name, department, salary, manager_id, hire_date) VALUES
('Nisha Rao', 'Engineering', 320000, NULL, '2019-04-01'), -- 1 CTO
('Karan Mehta', 'Engineering', 210000, 1, '2020-06-15'), -- 2
('Priya Nair', 'Engineering', 185000, 2, '2021-09-01'), -- 3
('Alok Gupta', 'Engineering', 185000, 2, '2022-01-10'), -- 4 same salary as 3
('Farah Ali', 'Sales', 150000, 1, '2021-03-22'), -- 5
('Vikram Singh', 'Sales', 120000, 5, '2023-07-05'), -- 6
('Tara Bose', 'Support', 90000, 5, '2024-02-19'); -- 7

```

Deliberate quirks in the data (they exist so the questions can bite):

- Sara Khan has a NULL city and no orders - powers NULL and anti-join questions.
- Orders 4, 5, 8 have no payment row - powers outer join and HAVING questions.
- discount_pct is NULL on 5 of 8 orders - powers COALESCE and NULL arithmetic.
- Employees 3 and 4 have identical salaries - powers RANK vs DENSE_RANK vs ROW_NUMBER.
- ProBook 15 has stock_qty = 0 - powers division-by-zero and NULLIF.
- categories is a three-level tree - powers recursive CTE.

Order totals (memorise these, many answers reference them):

order_id	customer	line total
1	Aarav	99997.00
2	Aarav	7497.00
3	Diya	10498.00
4	Rohan	74999.00
5	Meera	4999.00
6	Aarav	89999.00
7	John	16497.00
8	Diya	3499.00

Question 1

Difficulty: Easy

Category: SQL -> SELECT basics

Question

What does a SELECT statement do, and why is SELECT * discouraged in production code?

Answer

SELECT is a projection operator: it chooses which columns of which rows are returned. The engine reads rows from the source (table, index, or the output of a join), applies the filters, and then emits only the listed expressions.

SELECT * means "every column, in the table's physical order". It is fine when exploring data interactively. It is a liability in application code for five concrete reasons:

1. Breaks on schema change. Add a column and every SELECT * starts returning it. Code that maps by position (row[3]) silently reads the wrong field.
2. Wastes I/O and network. A TEXT blob you never use is still read from disk and shipped to the client.
3. Kills covering-index plans. If an index contains (customer_id, order_date) and you select only those two columns, Postgres can answer from the index alone (an *index-only scan). SELECT forces a heap fetch for every row.
4. Ambiguous in joins. Two tables both having unit_price produces two identically named output columns; most drivers then let you access only one of them.
5. Hides intent in review. The reader cannot tell which columns actually matter.

Example

You need a dropdown of customer names for an order form. You need two columns, not nine.

Code Example

```
-- Bad: ships every column including timestamps nobody renders
SELECT * FROM customers WHERE country = 'India';

-- Good: explicit projection, stable contract, index-friendly
SELECT customer_id,
       full_name
FROM   customers
WHERE  country = 'India'
ORDER BY full_name;
```

Output

```
customer_id | full_name
-----+-----
1 | Aarav Sharma
2 | Diya Patel
```

4 | Meera Iyer
3 | Rohan Verma
(4 rows)

Why Interviewers Ask This

It is the cheapest possible test of whether you have written code that survived a schema migration. A junior says "it returns all columns". A mid-level engineer mentions I/O and breakage. A senior mentions index-only scans and the API contract between database and application.

Common Mistakes

- Saying `SELECT *` is "slower" without saying why - vague answers read as memorised.
- Claiming it is always wrong. In an ad-hoc query, a `COUNT(*)`, or an `EXISTS` subquery it is perfectly correct.
- Forgetting that `SELECT *` inside a view freezes the column list at creation time in some engines, producing confusing drift later.

Best Practices

- Name every column in application queries; let code review see the contract.
- `SELECT COUNT(*)` is fine - no columns are actually materialised.
- Where a wide row is genuinely needed, define a view or a DTO so the column list lives in exactly one place.

Follow-up Questions

1. Is `SELECT COUNT(*)` slower than `SELECT COUNT(1)`? (No - the planner treats them alike.)
2. What is an index-only scan and how would you verify you got one?
3. How does `SELECT *` interact with `CREATE VIEW` and later `ALTER TABLE ADD COLUMN`?
4. Your ORM generates `SELECT *`. How would you fix that without rewriting every query?

Real-world Scenario

A team added an `internal_notes` TEXT column to customers for the support tool. The public customer API used `SELECT *` and serialised whatever it received. The next deploy leaked internal notes to customers through the JSON response. The fix was one line - an explicit column list - but the incident was a data-exposure postmortem.

Question 2

Difficulty: Medium

Category: SQL -> Logical query processing order

Question

In what order does SQL logically execute the clauses of a query? Why can you not use a column alias defined in SELECT inside the WHERE clause?

Answer

SQL is declarative, so written order is not execution order. The logical processing order is:

1. FROM pick the source tables
2. ON / JOIN match rows between them
3. WHERE filter individual rows <-- aliases from SELECT not visible yet
4. GROUP BY collapse rows into groups
5. HAVING filter groups <-- aggregates allowed here
6. SELECT compute output expressions <-- aliases are CREATED here
7. DISTINCT remove duplicate output rows
8. ORDER BY sort the result <-- aliases ARE visible
9. LIMIT/OFFSET slice the sorted result

A SELECT alias is created at step 6. WHERE runs at step 3, three steps earlier, so the name does not exist yet - hence column "x" does not exist. ORDER BY runs at step 8, after SELECT, which is why sorting by an alias works. (PostgreSQL and MySQL allow alias in ORDER BY; the standard also permits it there and nowhere earlier.)

The physical plan may differ wildly from this order - the optimiser reorders freely - but only in ways that cannot change the result.

Example

Computing a discounted order total once and filtering on it.

Code Example

```
-- FAILS: alias not yet defined when WHERE runs
-- SELECT order_id, quantity * unit_price AS line_total
-- FROM   order_items
-- WHERE  line_total > 50000;
-- ERROR: column "line_total" does not exist

-- Fix 1: repeat the expression
SELECT order_id,
       quantity * unit_price AS line_total
FROM   order_items
WHERE  quantity * unit_price > 50000
ORDER BY line_total DESC;      -- alias legal here

-- Fix 2 (cleaner, no repetition): compute in a subquery / CTE, filter outside
SELECT order_id, line_total
FROM   (SELECT order_id, quantity * unit_price AS line_total
        FROM   order_items) t
WHERE  line_total > 50000
ORDER BY line_total DESC;
```


Output

```
order_id | line_total
-----+-----
      1 |    89999.00
      6 |    89999.00
      4 |    74999.00
(3 rows)
```

Why Interviewers Ask This

Almost every SQL bug a candidate will hit - alias errors, WHERE vs HAVING confusion, LIMIT picking the wrong rows, aggregates in the wrong clause - is explained by this one list. If you know the order, you can derive the rules instead of memorising them.

Common Mistakes

- Reciting the order but not being able to use it to explain the alias error.
- Believing the database physically executes in this order (it does not; the optimiser is free to reorder as long as results match).
- Putting a row filter in HAVING where WHERE belongs - it works but filters late and costs performance.

Best Practices

- Push filters as early as possible: row conditions in WHERE, group conditions in HAVING.
- Use a CTE when an expression is needed in both SELECT and WHERE - name it once.
- Remember LIMIT is last: LIMIT 10 without ORDER BY returns an arbitrary 10 rows.

Follow-up Questions

1. Where do window functions fit in this order? (After HAVING, alongside SELECT - which is why you cannot filter on a window function in WHERE.)
2. Why can you GROUP BY an ordinal position (GROUP BY 1) but not always by alias?
3. If the optimiser can reorder, why does the logical order matter at all?
4. Does LIMIT without ORDER BY ever return a stable result set?

Real-world Scenario

A reporting endpoint paginated with LIMIT 20 OFFSET 40 and no ORDER BY. It worked for months on a small table where Postgres happened to return rows in physical order. After a VACUUM FULL rewrote the heap, page 3 started showing rows that had already appeared on page 1. The bug was never in the pagination code - it was the missing final-step sort.

Question 3

Difficulty: Easy

Category: SQL -> WHERE vs HAVING

Question

What is the difference between WHERE and HAVING? Give a query that needs both.

Answer

Aspect	WHERE	HAVING
Operates on	individual rows	groups produced by GROUP BY
Runs at step	3 (before grouping)	5 (after grouping)
Aggregates allowed	no	yes
Can use SELECT alias	no	in some engines (MySQL yes, Postgres no)
Typical use	"only delivered orders"	"only customers with more than 2 orders"
Performance	filters early, less work downstream	filters late, after aggregation cost paid

Rule of thumb: **if the condition can be evaluated by looking at one row, it belongs in WHERE.** If it needs a COUNT, SUM, AVG, MIN or MAX across a group, it belongs in HAVING.

Both in one query reads naturally as: "Of the delivered orders only (WHERE), show me the customers who placed more than one (HAVING)."

Example

Find customers who have more than one delivered order, and their total spend.

Code Example

```
SELECT  c.customer_id,
        c.full_name,
        COUNT(*) AS delivered_orders,
        SUM(oi.quantity * oi.unit_price) AS total_spend
FROM    customers c
JOIN    orders o ON o.customer_id = c.customer_id
JOIN    order_items oi ON oi.order_id = o.order_id
WHERE   o.status = 'DELIVERED' -- row-level filter, applied first
GROUP BY c.customer_id, c.full_name
HAVING  COUNT(DISTINCT o.order_id) > 1 -- group-level filter, applied after
ORDER BY total_spend DESC;
```

Output

```
customer_id | full_name | delivered_orders | total_spend
-----+-----+-----+-----
1 | Aarav Sharma | 4 | 197493.00
(1 row)
```

Aarav has three delivered orders (1, 2, 6) spanning four line items, totalling $99997 + 7497 + 89999 = 197493$. John has only one delivered order so HAVING removes him.

Why Interviewers Ask This

It separates candidates who understand when each clause runs from those who pattern-match. The COUNT(*) vs COUNT(DISTINCT o.order_id) subtlety in the example - four line items but three orders - is a favourite trap.

Common Mistakes

- Putting WHERE status = 'DELIVERED' in HAVING. Postgres rejects it unless wrapped in an aggregate; MySQL historically allowed it and filtered late.
- Counting COUNT(*) after a join and reporting line items as if they were orders. Join fan-out inflates every aggregate.
- Forgetting that every non-aggregated SELECT column must appear in GROUP BY.

Best Practices

- Filter rows in WHERE so fewer rows enter the aggregation - measurable win on large tables.
- After any join, ask "did this multiply my rows?" and use COUNT(DISTINCT ...) or aggregate in a subquery before joining.
- Group by the primary key plus any functionally dependent columns; Postgres allows grouping by c.customer_id alone once it is the PK.

Follow-up Questions

1. Can HAVING be used without GROUP BY? (Yes - the whole result is one implicit group.)
2. Rewrite the query so the fan-out problem disappears entirely.
3. Which is faster: filtering in WHERE or in HAVING, and how would you prove it?
4. Why does COUNT(*) differ from COUNT(DISTINCT o.order_id) here?

Real-world Scenario

A "top customers" dashboard joined orders to order_items and used COUNT(*) as the order count. Customers who bought many items per order appeared to be the most frequent buyers, and the marketing team targeted the wrong segment for a loyalty campaign. The fix was COUNT(DISTINCT o.order_id).

Question 4

Difficulty: Easy

Category: SQL -> Aggregate functions

Question

Explain COUNT(*), COUNT(column), and COUNT(DISTINCT column). How do they treat NULL?

Answer

- COUNT(*) counts rows. It never inspects values, so NULLs are irrelevant.
- COUNT(col) counts non-NULL values of col. Every NULL is skipped.
- COUNT(DISTINCT col) counts distinct non-NULL values. NULL is still excluded, and duplicates collapse to one.

This is the general rule for all aggregates except COUNT(*): SUM, AVG, MIN, MAX ignore NULL inputs. The consequence that trips people up is AVG: AVG(col) is SUM(col) / COUNT(col), not SUM(col) / COUNT(*). Rows where col IS NULL do not dilute the average.

If every input is NULL, SUM and AVG return NULL (not 0), while COUNT returns 0.

Example

customers.city is NULL for Sara Khan, and orders.discount_pct is NULL for five of eight orders. Both expose the difference immediately.

Code Example

```
SELECT COUNT(*)           AS all_rows,           -- 6 customers
       COUNT(city)        AS with_city,          -- Sara's NULL skipped
       COUNT(DISTINCT country) AS distinct_countries,
       COUNT(last_support_contact) AS ever_contacted_support
FROM   customers;

-- The AVG trap, side by side
SELECT COUNT(*)           AS orders_total,
       COUNT(discount_pct) AS orders_with_discount,
       SUM(discount_pct)   AS discount_sum,
       ROUND(AVG(discount_pct), 2) AS avg_ignoring_nulls, -- /3
       ROUND(AVG(COALESCE(discount_pct,0)), 2) AS avg_treating_null_as_zero -- /8
FROM   orders;
```

Output

all_rows	with_city	distinct_countries	ever_contacted_support
6	5	3	2

orders_total	orders_with_discount	discount_sum	avg_ignoring_nulls	avg_treating_null_as_zero
8	3	30.00	10.00	3.75

30/3 = 10.00 versus 30/8 = 3.75. Same data, two very different business numbers.

Why Interviewers Ask This

It is a two-in-one probe: aggregate semantics and NULL semantics. The AVG divergence is

the single most common source of "the dashboard number is wrong" bugs, so interviewers use it as a proxy for data-correctness instincts.

Common Mistakes

- Believing COUNT(col) counts rows. It counts values.
- Assuming SUM of all-NULL returns 0. It returns NULL; wrap with COALESCE(SUM(x), 0) when the caller needs a number.
- Writing COUNT(DISTINCT a, b) and expecting portability - MySQL supports it, Postgres requires COUNT(DISTINCT (a, b)) on a row constructor.

Best Practices

- Prefer COUNT(*) for "how many rows"; it states intent and never surprises.
- Decide explicitly whether NULL means zero or means unknown, then encode that decision with COALESCE - do not let the default win by accident.
- Wrap outward-facing aggregates in COALESCE(..., 0) so APIs never return null totals.

Follow-up Questions

1. What does AVG return over an empty table, and how should an API respond?
2. How would you count customers without a city using only aggregates?
3. Does COUNT(1) differ from COUNT(*) in plan or performance?
4. How do NULLs behave in MIN/MAX versus in ORDER BY?

Real-world Scenario

A subscription analytics job reported average revenue per user using AVG(mrr). Trial users had mrr IS NULL rather than 0, so they were silently excluded and ARPU looked 40 percent healthier than reality. The board deck was wrong for a quarter. Fix: AVG(COALESCE(mrr, 0)) plus a NOT NULL DEFAULT 0 constraint on the column.

Question 5

Difficulty: Medium

Category: SQL -> NULL semantics

Question

Why does WHERE city = NULL return no rows? Explain SQL's three-valued logic.

Answer

NULL is not a value - it is a marker meaning "unknown". Comparing anything to an unknown yields UNKNOWN, not TRUE or FALSE. SQL therefore has three-valued logic: TRUE, FALSE, UNKNOWN.

WHERE keeps a row only when the predicate is exactly TRUE. city = NULL evaluates to UNKNOWN, so the row is discarded - which is why the query returns zero rows instead of "the rows with no city". The correct operators are IS NULL and IS NOT NULL.

Truth tables worth memorising:

A	B	A AND B	A OR B
-----	-----	-----	-----
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Consequences that show up in real bugs:

- NULL = NULL is UNKNOWN, but NULL IS NOT DISTINCT FROM NULL is TRUE.
- NOT (city = 'Mumbai') does not include NULL cities.
- city NOT IN ('Mumbai', NULL) returns no rows ever - the NULL poisons the whole list.
- NULL in arithmetic propagates: 1000 * NULL is NULL.
- But UNIQUE constraints and GROUP BY treat NULLs as equal to each other, and ORDER BY sorts them together. The rules are deliberately inconsistent across features.

Example

Finding customers with no recorded city, and the NOT IN trap.

Code Example

```
-- Returns 0 rows: comparison with NULL is UNKNOWN
SELECT full_name FROM customers WHERE city = NULL;

-- Correct
SELECT full_name FROM customers WHERE city IS NULL;

-- The NOT IN + NULL trap: expected "everyone not in Mumbai", got nothing
SELECT full_name
FROM customers
WHERE city NOT IN ('Mumbai', NULL);      -- always empty

-- Safe alternatives
SELECT full_name
FROM customers
WHERE city IS DISTINCT FROM 'Mumbai';    -- NULL-safe inequality, keeps NULL rows

-- NULL-safe equality across engines
-- Postgres : a IS NOT DISTINCT FROM b
-- MySQL    : a <=> b
-- Portable : (a = b) OR (a IS NULL AND b IS NULL)
```

Output

```
-- WHERE city = NULL
full_name
-----
(0 rows)

-- WHERE city IS NULL
full_name
-----
Sara Khan
(1 row)
```

```
-- NOT IN ('Mumbai', NULL)
full_name
-----
(0 rows)

-- IS DISTINCT FROM 'Mumbai'
full_name
-----
Aarav Sharma
Rohan Verma
Meera Iyer
John Carter
Sara Khan
(5 rows)
```

Why Interviewers Ask This

NULL handling is the highest-density source of silent data corruption in SQL. A candidate who explains three-valued logic and then predicts the NOT IN behaviour is demonstrating that they debug from a model rather than by trial and error.

Common Mistakes

- Using = NULL or != NULL.
- Assuming NOT IN with a subquery is safe - if the subquery can yield a single NULL, the outer query returns nothing. This is the classic "my anti-join returns zero rows" bug.
- Thinking NULL and empty string "" are the same. They are not (except in Oracle).
- Believing NULL never counts as a duplicate - GROUP BY groups all NULLs together.

Best Practices

- Prefer NOT EXISTS over NOT IN for anti-joins - it is NULL-safe and usually plans better.
- Declare columns NOT NULL DEFAULT ... unless "unknown" is a genuine business state.
- Make the meaning of NULL explicit in the schema comment: unknown, not-applicable, or not-yet-known are three different things.
- In ORDER BY, state NULLS FIRST or NULLS LAST rather than relying on engine defaults.

Follow-up Questions

1. Rewrite city NOT IN (SELECT ...) as a NULL-safe anti-join.
2. Do UNIQUE constraints allow multiple NULLs? (Yes in Postgres/MySQL - they are not equal for uniqueness purposes; SQL Server allows only one.)
3. How does NULL sort in ORDER BY by default in Postgres versus MySQL?
4. What does COUNT(*) return for a group whose grouping key is NULL?

Real-world Scenario

A churn report used WHERE plan NOT IN (SELECT plan FROM active_plans) to find customers on retired plans. Someone inserted a row with plan = NULL into active_plans. Overnight the report returned zero churned customers, and the retention team assumed a great month. Nothing errored - the query was simply always UNKNOWN. Rewritten with NOT EXISTS, it worked again.

Question 6

Difficulty: Easy

Category: SQL -> COALESCE, NULLIF

Question

What do COALESCE and NULLIF do? How do you use them together to avoid division by zero?

Answer

COALESCE(a, b, c, ...) returns the first non-NULL argument, evaluated left to right, and short-circuits. It is the ANSI-standard way to supply a default. Engine-specific cousins: ISNULL (SQL Server, two args only), IFNULL (MySQL), NVL (Oracle). Prefer COALESCE - it is portable and takes any number of arguments.

NULLIF(a, b) returns NULL if a = b, otherwise returns a. It is the inverse tool: it creates a NULL deliberately.

The idiom $x / \text{NULLIF}(y, 0)$ is the standard division-by-zero guard: when y is 0 the denominator becomes NULL, so the whole expression is NULL instead of raising division by zero. Wrap it in COALESCE to present 0 to the caller:

```
COALESCE(x / NULLIF(y, 0), 0)
```

This is safer than a CASE WHEN y = 0 guard because it cannot be defeated by the optimiser evaluating branches in an unexpected order.

Example

Compute the effective total per order (discount is often NULL), and a stock turnover ratio where stock_qty can legitimately be 0 (ProBook 15).

Code Example

```
-- 1. COALESCE supplies a default for a NULL discount
SELECT o.order_id,
       o.discount_pct AS raw_discount,
       COALESCE(o.discount_pct, 0) AS effective_discount,
       SUM(oi.quantity * oi.unit_price) AS gross,
       ROUND(SUM(oi.quantity * oi.unit_price)
             * (1 - COALESCE(o.discount_pct, 0) / 100), 2) AS net_payable
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.discount_pct
ORDER BY o.order_id;

-- 2. NULLIF prevents division by zero for the zero-stock product
SELECT p.name,
       p.stock_qty,
       SUM(oi.quantity) AS units_sold,
       ROUND(SUM(oi.quantity)::numeric / NULLIF(p.stock_qty, 0), 2) AS turnover,
       COALESCE(ROUND(SUM(oi.quantity)::numeric
                     / NULLIF(p.stock_qty, 0), 2), 0) AS turnover_safe
```



```

FROM   products p
JOIN   order_items oi ON oi.product_id = p.product_id
GROUP BY p.name, p.stock_qty
ORDER BY p.name;

```

Output

order_id	raw_discount	effective_discount	gross	net_payable
1	10.00	10.00	99997.00	89997.30
2		0.00	7497.00	7497.00
3	5.00	5.00	10498.00	9973.10
4		0.00	74999.00	74999.00
5		0.00	4999.00	4999.00
6		0.00	89999.00	89999.00
7	15.00	15.00	16497.00	14022.45
8		0.00	3499.00	3499.00

(8 rows)

name	stock_qty	units_sold	turnover	turnover_safe
Air Fryer 4L	20	3	0.15	0.15
Mechanical Keyboard	40	3	0.08	0.08
ProBook 15	0	1		0.00
USB-C Hub	75	4	0.05	0.05
UltraBook 14	12	2	0.17	0.17
Blender Pro	18	2	0.11	0.11

(6 rows)

Without NULLIF, the ProBook row aborts the entire query with ERROR: division by zero.

Why Interviewers Ask This

Every reporting query in existence needs these two functions. The NULLIF division guard in particular is a small piece of professional vocabulary - candidates who know it have written real analytics SQL rather than only tutorial SQL.

Common Mistakes

- Assuming COALESCE converts types freely. All arguments must be type-compatible; COALESCE(discount_pct, 'none') fails on a numeric column.
- Using COALESCE(x, 0) on a column where NULL genuinely means "unknown" - this fabricates data and skews averages.
- Reaching for ISNULL/IFNULL in code meant to be portable.
- Writing NULLIF(y, 0) in the numerator by mistake, silently nulling the result.

Best Practices

- COALESCE at the presentation boundary; keep NULL in storage where it means unknown.
- Standardise on COALESCE(x / NULLIF(y, 0), 0) as the team's safe-division idiom.
- Cast before dividing integers: SUM(q)::numeric / ... avoids integer truncation in Postgres.

Follow-up Questions

1. How does COALESCE differ from CASE WHEN x IS NULL THEN y ELSE x END? (Semantically

identical; COALESCE evaluates x once and is clearer.)

2. Does COALESCE short-circuit? What if a later argument is an expensive subquery?
3. When is NULLIF useful outside division - for example on empty strings?
4. Why might COALESCE on an indexed column prevent index usage?

Real-world Scenario

A conversion-rate panel divided signups by visits. One day a traffic-source row arrived with zero visits and the whole ETL job failed at 3am, leaving the dashboard blank for a day. The one-character-class fix - NULLIF(visits, 0) plus COALESCE(..., 0) - made the pipeline degrade gracefully instead of dying.

Question 7

Difficulty: Easy

Category: SQL -> CASE expressions

Question

Explain the CASE expression. What is the difference between the simple and searched forms?

Answer

CASE is SQL's conditional expression - it returns a value, so it can appear anywhere a value can: SELECT, WHERE, ORDER BY, GROUP BY, even inside an aggregate. It is not a statement and has no side effects.

Searched form - each branch has a full predicate. Most flexible; use this by default.

```
CASE WHEN qty = 0 THEN 'out of stock'
      WHEN qty < 10 THEN 'low'
      ELSE 'healthy' END
```

Simple form - one expression compared for equality against constants. Shorter, but only tests =, and it cannot test NULL (because NULL = NULL is UNKNOWN).

```
CASE status WHEN 'PENDING' THEN 1 WHEN 'SHIPPED' THEN 2 ELSE 9 END
```

Key semantics:

- Branches are evaluated top to bottom and the first TRUE wins - order matters, and overlapping conditions are silently shadowed.
- Omitting ELSE implies ELSE NULL. This is the single most common CASE bug.
- All result branches must share a compatible type; the engine determines one result type.

Example

Bucket products by stock health, and impose a custom (non-alphabetical) status sort order.

Code Example

```
-- Searched CASE for bucketing + simple CASE for a custom sort key
SELECT p.name,
       p.stock_qty,
       CASE WHEN p.discontinued THEN 'discontinued'
            WHEN p.stock_qty = 0 THEN 'out of stock'
            WHEN p.stock_qty < 10 THEN 'low'
            ELSE 'healthy'
       END AS stock_status
FROM   products p
ORDER  BY CASE WHEN p.stock_qty = 0 THEN 0 ELSE 1 END, -- urgent rows first
        p.stock_qty;

-- CASE inside ORDER BY to enforce a workflow order, not alphabetical
SELECT order_id, status
FROM   orders
ORDER  BY CASE status
           WHEN 'PENDING' THEN 1
           WHEN 'SHIPPED' THEN 2
           WHEN 'DELIVERED' THEN 3
           WHEN 'CANCELLED' THEN 4
        END,
        order_id;
```

Output

name	stock_qty	stock_status
ProBook 15	0	out of stock
Wireless Mouse	5	discontinued
UltraBook 14	12	healthy
Blender Pro	18	healthy
Air Fryer 4L	20	healthy
Mechanical Keyboard	40	healthy
USB-C Hub	75	healthy

(7 rows)

order_id	status
5	PENDING
8	PENDING
3	SHIPPED
1	DELIVERED
2	DELIVERED
6	DELIVERED
4	CANCELLED

(7 rows)

Note Wireless Mouse reports discontinued, not low, because that branch is listed first.
Branch order encodes business priority.

Why Interviewers Ask This

CASE is the building block for conditional aggregation, pivoting, and custom sorts - three things every reporting task needs. The missing-ELSE question also tests NULL awareness.

Common Mistakes

- Forgetting ELSE and being surprised by NULL rows downstream.
- Using the simple form to test for NULL: CASE city WHEN NULL THEN ... never matches. Use the searched form with WHEN city IS NULL.
- Writing overlapping conditions in the wrong order so a branch is unreachable.
- Mixing types across branches ('N/A' and a number) and hitting a cast error.

Best Practices

- Always write an explicit ELSE, even ELSE NULL, to document intent.
- Prefer the searched form; the extra characters buy IS NULL, ranges, and compound tests.
- For long stable mappings, move the logic to a lookup table and join - easier to change without a deploy.

Follow-up Questions

1. How do you pivot rows into columns using CASE inside SUM?
2. Does CASE short-circuit, and can you rely on that to avoid an expensive branch?
3. What is FILTER (WHERE ...) in Postgres and how does it compare to CASE in aggregates?
4. Can you use CASE in a GROUP BY, and what are the readability trade-offs?

Real-world Scenario

An order-management UI sorted statuses alphabetically, so CANCELLED appeared above PENDING and warehouse staff kept processing cancelled orders first. A CASE in ORDER BY mapping status to workflow position fixed it without a schema change.

Question 8

Difficulty: Medium

Category: SQL -> Conditional aggregation and pivoting

Question

How do you pivot rows into columns in standard SQL - for example, order counts per status as one row per customer?

Answer

The portable technique is conditional aggregation: put a CASE inside an aggregate, one aggregate per output column.

```
SUM(CASE WHEN status = 'DELIVERED' THEN 1 ELSE 0 END) AS delivered
```

SUM(CASE ... 1 ELSE 0) and COUNT(CASE ... THEN 1 END) both work. With COUNT you omit ELSE, relying on COUNT ignoring NULL. SUM with an explicit ELSE 0 is easier to read

and returns 0 rather than 0 for empty groups either way.

PostgreSQL adds the cleaner ANSI FILTER clause, which is faster to read and can be slightly faster to execute:

```
COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered
```

Engine-specific pivot helpers exist (PIVOT in SQL Server/Oracle, crosstab() in the Postgres tablefunc extension) but they require a known, fixed column list, and so does conditional aggregation. True dynamic pivots require generating SQL in application code - this is the answer interviewers are listening for when they ask "what if the statuses are not known in advance?".

Example

One row per customer, one column per order status, plus lifetime spend.

Code Example

```
SELECT c.full_name,
       -- portable form
       SUM(CASE WHEN o.status = 'PENDING' THEN 1 ELSE 0 END) AS pending,
       SUM(CASE WHEN o.status = 'SHIPPED' THEN 1 ELSE 0 END) AS shipped,
       SUM(CASE WHEN o.status = 'DELIVERED' THEN 1 ELSE 0 END) AS delivered,
       SUM(CASE WHEN o.status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
       -- Postgres-only, same result, clearer intent
       COUNT(*) FILTER (WHERE o.status = 'DELIVERED') AS delivered_filter,
       -- conditional SUM: only count revenue from non-cancelled orders
       COALESCE(SUM(CASE WHEN o.status <> 'CANCELLED'
                      THEN oi_totals.line_total END), 0) AS realised_revenue
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN (SELECT order_id, SUM(quantity * unit_price) AS line_total
           FROM order_items GROUP BY order_id) oi_totals
       ON oi_totals.order_id = o.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY realised_revenue DESC;
```

Note the pre-aggregated subquery: joining order_items directly would fan out rows and inflate every count.

Output

full_name	pending	shipped	delivered	cancelled	delivered_filter	realised_revenue
Aarav Sharma	0	0	3	0	3	197493.00
John Carter	0	0	1	0	1	16497.00
Diya Patel	1	1	0	0	0	13997.00
Meera Iyer	1	0	0	0	0	4999.00
Rohan Verma	0	0	0	1	0	0.00
Sara Khan	0	0	0	0	0	0.00

(6 rows)

Rohan's only order was cancelled, so his realised revenue is 0 despite a 74999 order. Sara has no orders at all and still appears - that is the LEFT JOIN doing its job.

Why Interviewers Ask This

Pivoting is the most requested "real work" SQL skill in analytics-adjacent roles, and the question doubles as a test of join fan-out awareness and LEFT JOIN correctness.

Common Mistakes

- Joining the detail table directly and inflating counts (the fan-out trap).
- Using COUNT(CASE WHEN ... THEN 1 ELSE 0 END) - COUNT counts the zeros too, so every group returns the total row count. Either use SUM(...1 ELSE 0) or drop the ELSE.
- Using an INNER JOIN and silently dropping customers with no orders.
- Hard-coding statuses and then being unable to answer the dynamic-pivot follow-up.

Best Practices

- Pre-aggregate to one row per join key before joining, as shown.
- Use FILTER (WHERE ...) on Postgres for readability; keep CASE where portability matters.
- Wrap outward-facing sums in COALESCE(..., 0).
- For genuinely dynamic columns, generate the SQL in the application or return long-format rows and pivot in the presentation layer.

Follow-up Questions

1. How would you unpivot - turn four status columns back into rows? (UNION ALL, or LATERAL/CROSS APPLY.)
2. Why is COUNT(CASE ... ELSE 0 END) wrong?
3. How would you handle a status that does not exist yet without editing the query?
4. What breaks if you replace the LEFT JOINS with INNER JOINS here?

Real-world Scenario

A weekly ops report needed orders-by-status per warehouse. The first version joined order_items and reported item counts as order counts; warehouse managers were rated on inflated throughput numbers for two months. Pre-aggregating before the join corrected the figures, and the team added a regression test asserting the report's row count matched SELECT COUNT(*) FROM orders.

Question 9

Difficulty: Easy

Category: SQL -> Joins

Question

Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN and CROSS JOIN. Draw the row-matching behaviour.

Answer

A join combines rows from two inputs based on a predicate. The join type decides what happens to rows that find no match.

A	B	INNER	LEFT	FULL
+-----+	+-----+	+-----+	+-----+	+-----+
a1	b1	a1 b1	a1 b1	a1 b1
a2	b2	a2 b2	a2 b2	a2 b2
a3	b4		a3 --	a3 --
+-----+	+-----+		+-----+	+-----+
				-- b4
				+-----+

Join	Keeps	Unmatched left	Unmatched right
INNER	only matching pairs	dropped	dropped
LEFT [OUTER]	all left rows	kept, right columns NULL	dropped
RIGHT [OUTER]	all right rows	dropped	kept, left columns NULL
FULL [OUTER]	everything	kept with NULLs	kept with NULLs
CROSS	Cartesian product	n/a - no predicate	n/a

RIGHT JOIN is LEFT JOIN with the tables swapped; most teams ban it in code review because reading a query with mixed directions is error-prone. CROSS JOIN is legitimate for generating combinations (calendar x product) and is a bug everywhere else - usually a forgotten ON.

MySQL has no FULL OUTER JOIN; emulate it with `LEFT ... UNION ALL ... RIGHT ... WHERE left key IS NULL`.

Example

Which customers have orders, and which have none? Sara Khan (customer 6) has zero orders, so she is the row that distinguishes the join types.

Code Example

```
-- INNER: only customers who actually ordered -> Sara disappears
SELECT c.full_name, COUNT(o.order_id) AS orders
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
ORDER BY c.full_name;

-- LEFT: every customer, zero for those without orders -> Sara appears with 0
SELECT c.full_name, COUNT(o.order_id) AS orders
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
ORDER BY orders DESC, c.full_name;

-- CROSS JOIN with intent: every product x every status, for a coverage matrix
SELECT p.name, s.status
FROM   products p
CROSS JOIN (VALUES ('PENDING'),('SHIPPED')) AS s(status)
WHERE  p.product_id <= 2
ORDER BY p.name, s.status;
```

Output

```
-- INNER (5 rows, Sara missing)
```

full_name	orders
Aarav Sharma	3
Diya Patel	2
John Carter	1
Meera Iyer	1
Rohan Verma	1

```
-- LEFT (6 rows, Sara present with 0)
```

full_name	orders
Aarav Sharma	3
Diya Patel	2
John Carter	1
Meera Iyer	1
Rohan Verma	1
Sara Khan	0

```
-- CROSS
```

name	status
ProBook 15	PENDING
ProBook 15	SHIPPED
UltraBook 14	PENDING
UltraBook 14	SHIPPED

(4 rows)

COUNT(o.order_id) returns 0 for Sara, not 1 - because the outer-joined column is NULL and COUNT(column) skips NULLs. COUNT(*) would wrongly report 1.

Why Interviewers Ask This

Joins are the core of relational thinking, and the COUNT(*) vs COUNT(col) detail on an outer join separates people who have reasoned about it from people who have memorised syntax.

Common Mistakes

- Using COUNT(*) with a LEFT JOIN and reporting 1 instead of 0 for empty groups.
- Putting the right table's filter in WHERE instead of ON - this silently converts a LEFT JOIN into an INNER JOIN (see the next question).
- Assuming FULL OUTER JOIN exists everywhere.
- Accidental CROSS JOIN from a missing ON, producing millions of rows.

Best Practices

- Always qualify columns with table aliases; unqualified columns in joins are review blockers.
- Prefer explicit JOIN ... ON over comma-separated FROM a, b WHERE.
- Standardise on LEFT JOIN and reorder tables rather than mixing in RIGHT JOIN.

Follow-up Questions

1. Why does COUNT(o.order_id) give 0 but COUNT(*) gives 1 for Sara?
2. When is a CROSS JOIN the correct tool?

3. How do you emulate FULL OUTER JOIN on MySQL?
4. What is a semi-join and an anti-join, and which SQL constructs express them?

Real-world Scenario

A monthly active users report used INNER JOIN from users to events. Users who signed up but did nothing were invisible, so the activation funnel showed a 100 percent activation rate. One word - LEFT - restored several thousand zero-activity users and revealed a real onboarding problem.

Question 10

Difficulty: Medium

Category: SQL -> Joins, ON vs WHERE

Question

What is the difference between putting a condition in the ON clause versus the WHERE clause of a LEFT JOIN?

Answer

For an INNER JOIN the two are equivalent - the optimiser will even rewrite one into the other. For an outer join they mean completely different things:

- ON is evaluated while matching rows. A non-matching right row simply becomes NULL, and the left row survives.

- WHERE is evaluated after the join has produced its rows. The NULL-extended row is then tested, fails the predicate, and the left row disappears.

Net effect: **a filter on the right table's column in WHERE silently turns your LEFT JOIN into an INNER JOIN.** This is one of the most common SQL bugs in production reporting.

The exception is WHERE right_col IS NULL, which is the deliberate anti-join idiom: keep only left rows that found no match.

Example

"Every customer with their DELIVERED order count" - correct with ON, broken with WHERE.

Code Example

```
-- CORRECT: condition in ON. All 6 customers survive.
SELECT c.full_name,
       COUNT(o.order_id) AS delivered_orders
FROM   customers c
LEFT   JOIN orders o
      ON o.customer_id = c.customer_id
      AND o.status      = 'DELIVERED'      -- part of the match
GROUP BY c.full_name
```

```

ORDER BY delivered_orders DESC, c.full_name;

-- BROKEN: condition in WHERE. Degrades to INNER JOIN, only 2 customers survive.
SELECT c.full_name,
       COUNT(o.order_id) AS delivered_orders
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.status = 'DELIVERED' -- kills NULL-extended rows
GROUP BY c.full_name
ORDER BY delivered_orders DESC, c.full_name;

-- DELIBERATE anti-join: customers who never ordered
SELECT c.full_name
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;

```

Output

```

-- CORRECT (6 rows)
full_name | delivered_orders
-----+-----
Aarav Sharma | 3
John Carter | 1
Diya Patel | 0
Meera Iyer | 0
Rohan Verma | 0
Sara Khan | 0

-- BROKEN (2 rows) - four customers vanished
full_name | delivered_orders
-----+-----
Aarav Sharma | 3
John Carter | 1

-- ANTI-JOIN
full_name
-----
Sara Khan
(1 row)

```

Why Interviewers Ask This

It is the single highest-yield SQL correctness question. It cannot be answered by pattern matching - you must actually understand that WHERE runs after the join. Interviewers often present the broken query and ask "why are rows missing?"

Common Mistakes

- Believing ON and WHERE are interchangeable because they are for INNER JOIN.
- Adding a date filter on the outer table in WHERE and then wondering why the report lost customers.
- Writing WHERE o.status = 'DELIVERED' OR o.status IS NULL as a workaround - it works but obscures intent; put the condition in ON.

Best Practices

- Rule: **join conditions and filters on the outer table go in ON; filters on the preserved

table go in WHERE.**

- When a LEFT JOIN result row count equals the INNER JOIN count, treat it as a smell and check for this bug.
- Prefer NOT EXISTS over the IS NULL anti-join pattern when readability matters; both are correct and usually plan identically.

Follow-up Questions

1. Why is WHERE o.order_id IS NULL the exception, and what is it called?
2. Are ON and WHERE interchangeable for INNER JOIN? Prove it with a plan.
3. How does this behave for FULL OUTER JOIN - which side is preserved?
4. Rewrite the correct query using a correlated subquery instead of a join.

Real-world Scenario

A subscription revenue report added WHERE invoices.status = 'PAID' to a LEFT JOIN from customers. Customers with no invoices at all silently dropped out, so churn looked flat while signups were quietly failing to convert. The row count fell from 12,400 to 9,100 and nobody noticed for six weeks because both numbers looked plausible.

Question 11

Difficulty: Medium

Category: SQL -> Self join

Question

What is a self join? Write a query listing each employee with their manager's name.

Answer

A self join is a join of a table to itself, using two different aliases so the engine treats them as two independent inputs. It is the standard way to relate rows within one table: employee to manager, product to its substitute, event to the previous event.

Requirements:

1. Two distinct aliases (e and m) - without them column references are ambiguous.
2. A LEFT JOIN if the relationship is optional. The CTO has manager_id IS NULL, and an INNER JOIN would silently drop the top of the hierarchy.

A self join reaches exactly one level. For arbitrary depth - the whole reporting chain - you need a recursive CTE (covered later in this section).

Example

employees.manager_id points at employees.employee_id. Nisha Rao is the CTO with no manager.

Code Example

```
-- One level of hierarchy. LEFT JOIN so the CTO is not lost.
SELECT e.employee_id,
       e.full_name           AS employee,
       e.department,
       COALESCE(m.full_name, '(none)') AS manager,
       -- flag employees paid more than their manager
       CASE WHEN e.salary > m.salary THEN 'YES' ELSE 'no' END AS out_earns_manager
FROM   employees e
LEFT   JOIN employees m ON m.employee_id = e.manager_id
ORDER  BY e.employee_id;

-- Self join to find pairs of colleagues in the same department on equal salary.
-- e1.employee_id < e2.employee_id avoids self-pairs and mirrored duplicates.
SELECT e1.full_name AS employee_a,
       e2.full_name AS employee_b,
       e1.department,
       e1.salary
FROM   employees e1
JOIN   employees e2
      ON e1.department = e2.department
      AND e1.salary    = e2.salary
      AND e1.employee_id < e2.employee_id;
```

Output

employee_id	employee	department	manager	out_earns_manager
1	Nisha Rao	Engineering	(none)	no
2	Karan Mehta	Engineering	Nisha Rao	no
3	Priya Nair	Engineering	Karan Mehta	no
4	Alok Gupta	Engineering	Karan Mehta	no
5	Farah Ali	Sales	Nisha Rao	no
6	Vikram Singh	Sales	Farah Ali	no
7	Tara Bose	Support	Farah Ali	no

(7 rows)

employee_a	employee_b	department	salary
Priya Nair	Alok Gupta	Engineering	185000.00

(1 row)

For Nisha, m.salary is NULL, so e.salary > NULL is UNKNOWN, the CASE falls to ELSE, and she correctly reads no rather than erroring.

Why Interviewers Ask This

Self joins test whether you see a table as a set of rows rather than as a fixed object. The e1.id < e2.id trick for de-duplicating pairs is a small, recognisable piece of craft.

Common Mistakes

- Using INNER JOIN and losing the root of the hierarchy.
- Omitting e1.id < e2.id, producing each pair twice plus every row matched to itself.
- Forgetting aliases entirely - ERROR: table name "employees" specified more than once.
- Assuming a self join can traverse multiple levels.

Best Practices

- Alias meaningfully: e/m for employee/manager beats a/b.
- Index the foreign key (manager_id) - self joins hammer it.
- Use < rather than <> for unordered pair generation.
- Escalate to a recursive CTE the moment "all levels" is required.

Follow-up Questions

1. How would you list the full management chain up to the CTO?
2. How do you find employees with no direct reports?
3. What index would you add to make this join efficient at a million rows?
4. How does a self join differ from a window function for "compare to previous row" problems?

Real-world Scenario

An HR system needed an approval-chain email at each level. The first implementation self-joined five times to support five levels, then broke when the org added a sixth. Replacing the fixed chain of self joins with a recursive CTE made depth unbounded and removed 60 lines of SQL.

Question 12

Difficulty: Medium

Category: SQL -> Window functions

Question

What is a window function? How does it differ from GROUP BY aggregation?

Answer

A window function computes a value across a set of rows related to the current row while preserving every input row. GROUP BY collapses rows; a window function annotates them.

GROUP BY	WINDOW FUNCTION
-----	-----
6 rows in	6 rows in
2 rows out (one per group)	6 rows out (each + group value)
detail is lost	detail is kept

Syntax:

```
function() OVER (  
  PARTITION BY expr    -- optional: divides rows into independent windows  
  ORDER BY expr        -- optional: orders rows within the window  
  ROWS/RANGE frame     -- optional: which rows within the partition count  
)
```

Window functions run after WHERE, GROUP BY and HAVING, and before ORDER BY.

That ordering has one critical consequence: ****you cannot filter on a window function in WHERE.**** Wrap the query in a subquery or CTE and filter outside. This is the number one window-function error message candidates hit.

Three families:

Family	Examples	Needs ORDER BY
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE	yes
Aggregate-as-window	SUM, AVG, COUNT, MAX OVER	optional
Offset / navigation	LAG, LEAD, FIRST_VALUE, LAST_VALUE	yes

Example

Show each employee's salary next to their department average and their share of the department payroll - impossible with GROUP BY alone without a self join.

Code Example

```
SELECT full_name,
       department,
       salary,
       -- department aggregate, but every row is preserved
       ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
       salary - ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS diff_from_avg,
       SUM(salary) OVER (PARTITION BY department) AS dept_payroll,
       ROUND(100.0 * salary
             / SUM(salary) OVER (PARTITION BY department), 1) AS pct_of_dept,
       COUNT(*) OVER (PARTITION BY department) AS dept_headcount,
       -- running total across the whole company, ordered by hire date
       SUM(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING
                        AND CURRENT ROW) AS cumulative_payroll
FROM   employees
ORDER BY department, salary DESC;
```

Output

full_name	department	salary	dept_avg	diff_from_avg	dept_payroll	pct_of_dept
dept_headcount	cumulative_payroll					
Nisha Rao	Engineering	320000.00	225000.00	95000.00	900000.00	35.6
4	320000.00					
Karan Mehta	Engineering	210000.00	225000.00	-15000.00	900000.00	23.3
4	530000.00					
Priya Nair	Engineering	185000.00	225000.00	-40000.00	900000.00	20.6
4	865000.00					
Alok Gupta	Engineering	185000.00	225000.00	-40000.00	900000.00	20.6
4	1070000.00					
Farah Ali	Sales	150000.00	135000.00	15000.00	270000.00	55.6
2	680000.00					
Vikram Singh	Sales	120000.00	135000.00	-15000.00	270000.00	44.4
2	1190000.00					
Tara Bose	Support	90000.00	90000.00	0.00	90000.00	100.0
1	1280000.00					

(7 rows)

Why Interviewers Ask This

Window functions are the dividing line between basic and professional SQL. Interviewers use them to check whether you can express "compare a row to its group" without a self join, and whether you know the WHERE-cannot-see-windows rule.

Common Mistakes

- Trying to filter on a window alias in WHERE -> `ERROR: window functions are not allowed in WHERE`. Wrap in a subquery or CTE.
- Confusing PARTITION BY with GROUP BY. PARTITION BY does not reduce row count.
- Adding ORDER BY inside OVER for an aggregate without realising it creates a running total (default frame is UNBOUNDED PRECEDING to CURRENT ROW), not a partition total.
- Assuming LAST_VALUE gives the partition's last row - with the default frame it returns the current row. You need ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Best Practices

- Name the window once with a WINDOW clause when reusing it:
WINDOW w AS (PARTITION BY department) then AVG(salary) OVER w.
- Always state the frame explicitly for running calculations - do not rely on defaults.
- Index the PARTITION BY and ORDER BY columns to let the planner skip a sort.

Follow-up Questions

1. Why can you not use a window function in WHERE, and what is the workaround?
2. What is the default window frame, and when does it bite you?
3. Difference between ROWS and RANGE framing with duplicate ORDER BY values?
4. Rewrite dept_avg using a correlated subquery and compare the plans.

Real-world Scenario

A payroll fairness audit needed each employee's salary versus their band average, for 40,000 employees. The original implementation ran one correlated subquery per row - 40,000 aggregate scans, about nine minutes. A single AVG(...) OVER (PARTITION BY band) computed all of it in one pass in under two seconds.

Question 13

Difficulty: Medium

Category: SQL -> ROW_NUMBER, RANK, DENSE_RANK

Question

Explain the difference between ROW_NUMBER(), RANK() and DENSE_RANK().

Answer

All three number rows within a window; they differ only in how they treat ties.

Function	Ties get	Gaps after ties	Values for salaries 320k, 210k, ~
ROW_NUMBER()	different numbers (arbitrary amo~	no	1, 2, 3, 4
RANK()	the same number	yes - skips	1, 2, 3, 3 -> next would be 5
DENSE_RANK()	the same number	no	1, 2, 3, 3 -> next would be 4

Mnemonics:

- ROW_NUMBER = "give me exactly one row per position" -> deduplication, pagination.
- RANK = "Olympic medals" -> two silvers means no bronze.
- DENSE_RANK = "distinct salary levels" -> ranks values, not rows.

Critical detail: ROW_NUMBER() is non-deterministic across ties. If Priya and Alok both earn 185000 and you order only by salary, which gets 3 and which gets 4 can change between runs. Always add a unique tiebreaker (ORDER BY salary DESC, employee_id) when the result feeds anything reproducible.

Example

Employees 3 and 4 have identical salaries by design - that is what makes the three functions diverge visibly.

Code Example

```
SELECT full_name,
       department,
       salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC, employee_id) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rank_val,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val,
       -- per-department ranking, the classic "top N per group" building block
       RANK() OVER (PARTITION BY department
                   ORDER BY salary DESC) AS dept_rank
FROM   employees
ORDER BY salary DESC, employee_id;
```

Output

full_name	department	salary	row_num	rank_val	dense_rank_val	dept_rank
Nisha Rao	Engineering	320000.00	1	1	1	1
Karan Mehta	Engineering	210000.00	2	2	2	2
Priya Nair	Engineering	185000.00	3	3	3	3
Alok Gupta	Engineering	185000.00	4	3	3	3
Farah Ali	Sales	150000.00	5	5	4	1
Vikram Singh	Sales	120000.00	6	6	5	2
Tara Bose	Support	90000.00	7	7	6	3

(7 rows)

Read the tie carefully: rank_val jumps from 3 to 5 (skipping 4), while dense_rank_val goes 3 then 4. That is the entire difference, and it is exactly what interviewers check.

Why Interviewers Ask This

It is the most common window-function question in existence because the answer is short, verifiable, and the tie behaviour cannot be bluffed. It also sets up the "top N per group" follow-up, which is a genuine daily task.

Common Mistakes

- Saying RANK and DENSE_RANK are "basically the same".
- Using ROW_NUMBER() for a leaderboard, so two people on identical scores get different positions arbitrarily.
- Using RANK() for pagination - gaps make WHERE rank BETWEEN 11 AND 20 return the wrong count of rows.
- Forgetting the unique tiebreaker, producing unstable output between runs.

Best Practices

- Pagination and de-duplication -> ROW_NUMBER() with a deterministic ORDER BY.
- Public leaderboards -> RANK() (fair) or DENSE_RANK() (compact), decided with the product owner and documented.
- Always add a unique column as the last ORDER BY term.

Follow-up Questions

1. Write "top 2 earners per department" using one of these. (Filter dept_rank <= 2 in an outer query.)
2. Which of the three would you use to delete duplicate rows, and how?
3. What does NTILE(4) do, and how does it split 7 rows?
4. How do you get "top N per group" without window functions? (Correlated subquery or LATERAL.)

Real-world Scenario

A gaming leaderboard used ROW_NUMBER() over score. Players tied on score saw their rank flip-flop between page refreshes because the plan's tie order was unstable, and support tickets accused the company of rigging ranks. Switching to DENSE_RANK() and adding first_achieved_at as a tiebreaker made ranks stable and explainable.

Question 14

Difficulty: Medium

Category: SQL -> LAG, LEAD

Question

What do LAG() and LEAD() do? Use them to compute month-over-month growth.

Answer

LAG(expr, offset, default) reads a value from a previous row in the window;
LEAD(expr, offset, default) reads from a following row. Both require ORDER BY inside
OVER - "previous" is meaningless without an order.

- offset defaults to 1.

- default is returned instead of NULL when the target row does not exist (first row for LAG, last for LEAD). Supplying it is what turns a NULL-riddled result into a clean one.

They replace the classic self-join-on-n-1 pattern, which is both slower and wrong whenever the sequence has gaps (missing months, deleted ids).

Period-over-period growth is the canonical use:

```
growth % = (current - previous) / previous * 100
```

Guard the denominator with NULLIF(previous, 0).

Example

Monthly revenue from delivered and shipped orders, with the previous month's value, absolute delta and percentage growth.

Code Example

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', o.order_date)::date      AS month,  
           SUM(oi.quantity * oi.unit_price)             AS revenue  
    FROM    orders o  
    JOIN    order_items oi ON oi.order_id = o.order_id  
    WHERE   o.status <> 'CANCELLED'  
    GROUP BY DATE_TRUNC('month', o.order_date)  
)  
SELECT month,  
       revenue,  
       LAG(revenue) OVER (ORDER BY month)             AS prev_month,  
       LEAD(revenue) OVER (ORDER BY month)            AS next_month,  
       revenue - LAG(revenue) OVER (ORDER BY month)    AS delta,  
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))  
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct,  
       -- default argument: report the first month's delta against 0 instead of NULL  
       revenue - LAG(revenue, 1, 0::numeric) OVER (ORDER BY month) AS delta_no_null  
FROM    monthly  
ORDER BY month;
```

Output

month	revenue	prev_month	next_month	delta	growth_pct	delta_no_null
2025-01-01	99997.00		17995.00			99997.00
2025-02-01	17995.00	99997.00	4999.00	-82002.00	-82.0	-82002.00

2025-03-01		4999.00		17995.00		106496.00		-12996.00		-72.2		-12996.00
2025-04-01		106496.00		4999.00		3499.00		101497.00		2030.3		101497.00
2025-05-01		3499.00		106496.00				-102997.00		-96.7		-102997.00

(5 rows)

February = order 2 (7497) + order 3 (10498) = 17995. April = order 6 (89999) + order 7 (16497) = 106496. March excludes cancelled order 4, leaving only order 5 at 4999.

Why Interviewers Ask This

Every analytics and finance team needs period comparisons, and the naive self-join solution breaks on gaps. LAG/LEAD fluency signals you have done real reporting work. The NULLIF guard and the default argument are the details that distinguish a polished answer.

Common Mistakes

- Omitting ORDER BY inside OVER -> the notion of "previous" is undefined and engines error or return arbitrary rows.
- Self-joining on month = month - 1 and losing rows whenever a month has no data.
- Dividing by the previous value without NULLIF, so one zero month kills the query.
- Expecting LAG to skip NULLs - it does not; it returns whatever is in the offset row.

Best Practices

- Pre-aggregate into a CTE first, then window over it - clearer and avoids fan-out.
- For sparse series, generate a complete calendar with generate_series and LEFT JOIN so missing months become explicit zeros before you apply LAG.
- Use the third default argument to eliminate leading NULLs at the source.

Follow-up Questions

1. How would you fill missing months so growth is not misleading?
2. Compute a 3-month moving average using a window frame.
3. What is the difference between LAG(x, 2) and LAG(LAG(x))? (The latter is not legal - window functions cannot nest.)
4. How would you find the longest streak of consecutive growing months?

Real-world Scenario

A retention dashboard compared each cohort's month N to month N-1 via a self join on month_index - 1. Cohorts with a zero-activity month had that month missing entirely, so the join silently compared month 3 to month 1 and reported impossible retention above 100 percent. Rebuilt with a generated calendar plus LAG, the anomalies disappeared.

Question 15

Difficulty: Medium
Category: SQL -> CTE

Question

What is a CTE (WITH clause)? How does it compare to a subquery and to a temporary table?

Answer

A Common Table Expression is a named result set that exists for the duration of one statement:

```
WITH name AS ( SELECT ... )
SELECT ... FROM name;
```

Aspect	CTE	Inline subquery	Temp table
Lifetime	one statement	one statement	session / transaction
Reusable in same query	yes, by name	no - must repeat it	yes
Readability	high, top-to-bottom	drops off when nested	high
Indexable	no	no	yes
Statistics for planner	limited	limited	full, after ANALYZE
Recursion	yes (WITH RECURSIVE)	no	via loop

Materialisation is the detail that matters in interviews. Before PostgreSQL 12 every CTE was an optimisation fence - always materialised, no predicate push-down, which made CTEs a performance trap. From Postgres 12 a CTE referenced once is inlined by default, and you control it explicitly with `MATERIALIZED` / `NOT MATERIALIZED`. SQL Server and Oracle have always inlined CTEs (they are essentially macros there). MySQL supports CTEs from 8.0.

So the modern answer is: ****CTEs are for readability and reuse; they are not a performance feature, and on Postgres you can now choose the materialisation behaviour.****

Example

A three-stage pipeline - order totals, then customer totals, then classification - written as readable steps instead of nested subqueries.

Code Example

```
WITH order_totals AS (          -- stage 1: one row per order
    SELECT o.order_id,
           o.customer_id,
           o.status,
           SUM(oi.quantity * oi.unit_price) AS order_value
    FROM   orders o
    JOIN   order_items oi ON oi.order_id = o.order_id
    GROUP BY o.order_id, o.customer_id, o.status
),
customer_totals AS (           -- stage 2: one row per customer, reuses stage 1
    SELECT c.customer_id,
           c.full_name,
           COUNT(ot.order_id) FILTER (WHERE ot.status <> 'CANCELLED') AS valid_orders,
           COALESCE(SUM(ot.order_value)
                    FILTER (WHERE ot.status <> 'CANCELLED'), 0) AS lifetime_value
    FROM   customers c
    LEFT JOIN order_totals ot ON ot.customer_id = c.customer_id
    GROUP BY c.customer_id, c.full_name
)
```

```

SELECT full_name,          -- stage 3: classify
       valid_orders,
       lifetime_value,
       CASE WHEN lifetime_value >= 100000 THEN 'VIP'
            WHEN lifetime_value >= 10000 THEN 'regular'
            WHEN lifetime_value > 0 THEN 'occasional'
            ELSE 'never purchased'
       END AS segment
FROM   customer_totals
ORDER BY lifetime_value DESC;

-- Postgres 12+: force or prevent materialisation explicitly
-- WITH order_totals AS MATERIALIZED ( ... ) -- compute once, fence it
-- WITH order_totals AS NOT MATERIALIZED ( ... ) -- always inline

```

Output

full_name	valid_orders	lifetime_value	segment
Aarav Sharma	3	197493.00	VIP
John Carter	1	16497.00	regular
Diya Patel	2	13997.00	regular
Meera Iyer	1	4999.00	occasional
Rohan Verma	0	0.00	never purchased
Sara Khan	0	0.00	never purchased

(6 rows)

Rohan lands in "never purchased" because his single order was cancelled - the FILTER excluded it. Whether that is the right business rule is exactly the kind of clarifying question an interviewer wants you to ask.

Why Interviewers Ask This

CTEs are how maintainable analytical SQL is written. The materialisation-fence history is a sharp senior-level discriminator: many candidates still repeat "CTEs are slower in Postgres", which has been outdated since version 12.

Common Mistakes

- Believing a CTE is always materialised (or never) - the answer is version and engine specific.
- Chaining eight CTEs and assuming the planner will optimise across all of them.
- Using a CTE where an index was the real fix.
- Forgetting that a CTE cannot be indexed - for repeated heavy reuse a temp table may win.

Best Practices

- One CTE per logical step, named after what it produces, not t1/t2.
- Keep chains under roughly five stages; beyond that, a view or materialised view is clearer.
- On Postgres, benchmark MATERIALIZED versus NOT MATERIALIZED before assuming.
- Promote a CTE used by many queries to a view; promote an expensive one to a materialised view.

Follow-up Questions

1. When would a temp table beat a CTE?
2. What changed for CTEs in PostgreSQL 12?

3. Can a CTE be used in INSERT/UPDATE/DELETE, and what is RETURNING plus a CTE good for?
4. How does a CTE differ from a view in lifetime and permissions?

Real-world Scenario

A nightly billing job used a seven-stage CTE chain over 40 million invoice rows and took 50 minutes. Marking the two most reused stages MATERIALIZED - so they were computed once instead of re-inlined into three branches each - cut it to 11 minutes with no logic change.

Question 16

Difficulty: Hard

Category: SQL -> Recursive CTE

Question

Write a recursive CTE that returns the full category tree with each category's depth and path. Explain how recursion terminates.

Answer

WITH RECURSIVE has exactly two parts joined by UNION ALL:

```
WITH RECURSIVE t AS (  
    <anchor member>           -- runs once, seeds the result  
    UNION ALL  
    <recursive member>       -- references t, runs repeatedly  
)
```

Execution model:

1. Evaluate the anchor -> put rows in the working table.
2. Evaluate the recursive member using only the rows produced by the previous iteration.
3. Append those rows to the result and make them the new working table.
4. Repeat until an iteration produces zero rows. That is the termination condition.

So recursion stops naturally when no child rows match - not because of a counter. If the data contains a cycle (A is a parent of B, B is a parent of A) it never terminates. Two defences:

- A depth guard: WHERE depth < 10.
- A path array with WHERE NOT node = ANY(path) to detect revisits.

Note UNION ALL versus UNION: UNION de-duplicates each iteration, which masks cycles but is much slower. Use UNION ALL plus an explicit cycle guard.

Example

categories is a three-level tree: Electronics -> Computers -> Laptops / Accessories, and Home -> Kitchen.

Code Example

```
WITH RECURSIVE category_tree AS (
  -- ANCHOR: the roots (no parent)
  SELECT category_id,
         name,
         parent_id,
         1 AS depth,
         name::text AS path,
         ARRAY[category_id] AS id_path
  FROM   categories
  WHERE  parent_id IS NULL

  UNION ALL

  -- RECURSIVE: children of whatever the previous iteration produced
  SELECT c.category_id,
         c.name,
         c.parent_id,
         ct.depth + 1,
         ct.path || ' > ' || c.name,
         ct.id_path || c.category_id
  FROM   categories c
  JOIN   category_tree ct ON c.parent_id = ct.category_id
  WHERE  ct.depth < 10          -- depth guard
        AND NOT c.category_id = ANY(ct.id_path) -- cycle guard
)
SELECT category_id,
       REPEAT(' ', depth - 1) || name AS indented_name,
       depth,
       path
FROM   category_tree
ORDER BY path;

-- Same technique upward: the management chain for Tara Bose (employee 7)
WITH RECURSIVE chain AS (
  SELECT employee_id, full_name, manager_id, 1 AS level
  FROM   employees WHERE employee_id = 7
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, ch.level + 1
  FROM   employees e
  JOIN   chain ch ON e.employee_id = ch.manager_id
)
SELECT level, full_name FROM chain ORDER BY level;
```

Output

category_id	indented_name	depth	path
1	Electronics	1	Electronics
2	Computers	2	Electronics > Computers
4	Accessories	3	Electronics > Computers > Accessories
3	Laptops	3	Electronics > Computers > Laptops
5	Home	1	Home
6	Kitchen	2	Home > Kitchen

(6 rows)

level	full_name
1	Tara Bose
2	Farah Ali
3	Nisha Rao

(3 rows)

Iteration trace for the tree: anchor produces {Electronics, Home}; iteration 1 produces {Computers, Kitchen}; iteration 2 produces {Laptops, Accessories}; iteration 3 produces nothing -> stop.

Why Interviewers Ask This

Recursive CTEs are the standard test for hierarchical data - org charts, category trees, bill of materials, threaded comments, graph reachability. Explaining the termination rule and the cycle risk is what separates a memorised template from real understanding.

Common Mistakes

- Omitting RECURSIVE (required in Postgres; SQL Server does not use the keyword).
- Referencing the CTE twice in the recursive member - not allowed.
- No cycle guard, then an infinite loop that consumes disk in a production database.
- Using UNION instead of UNION ALL without knowing the de-duplication cost.
- Putting an aggregate or LEFT JOIN against the recursive term where the engine forbids it.

Best Practices

- Always include a depth guard, even when the data "cannot" have cycles.
- Carry an array path for both cycle detection and human-readable breadcrumbs.
- For very deep or hot trees, denormalise: materialised path (/1/2/4/), nested sets, or a closure table trades write cost for O(1) subtree reads.
- Index the parent column - the recursive join hits it once per iteration.

Follow-up Questions

1. How would you find all descendants of one node only?
2. Compare recursive CTE, adjacency list, materialised path and closure table.
3. How does CYCLE detection syntax work in Postgres 14+?
4. How would you compute the total stock of every product under a category subtree?

Real-world Scenario

An e-commerce catalogue allowed admins to re-parent categories. Someone made "Electronics" a child of "Laptops", creating a cycle. The nightly category rollup used a recursive CTE with no guard, ran until it filled the temp tablespace, and took the reporting replica offline. The postmortem added both a cycle guard in the query and a CHECK at write time.

Question 17

Difficulty: Medium

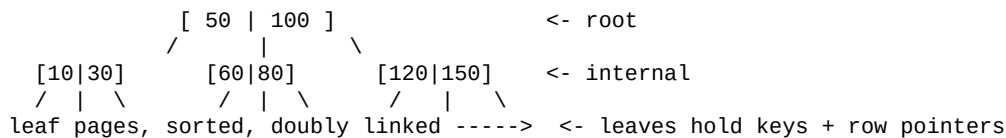
Category: SQL -> Indexes

Question

What is an index? Explain clustered versus non-clustered indexes and when an index hurts.

Answer

An index is an auxiliary data structure - almost always a B+ tree - that maps column values to row locations, turning an $O(n)$ table scan into an $O(\log n)$ lookup.



Clustered index - defines the physical order of the table's rows; the leaf level is the table. One per table. In SQL Server the PRIMARY KEY is clustered by default; in MySQL InnoDB it is the primary key, always. Range scans are extremely fast because rows are adjacent on disk.

Non-clustered (secondary) index - a separate structure holding the key plus a pointer to the row. Many per table. Requires a second step to fetch the row unless the index covers the query.

PostgreSQL is the important exception: it has no clustered indexes. All indexes are secondary, pointing at a heap; the CLUSTER command is a one-time physical reorder that is not maintained.

	Clustered	Non-clustered
Count per table	1	many
Stores	the rows themselves	key + row pointer
Extra lookup	none	yes, unless covering
Range queries	fastest	slower
Insert cost	can cause page splits	index maintenance only

When an index hurts:

- Every INSERT, UPDATE and DELETE must also update every affected index - write amplification. A table with 12 indexes writes 13 structures.
- Low-cardinality columns (a boolean, a two-value status) are usually not worth it; the planner will prefer a sequential scan anyway.
- Small tables: a scan of 200 rows beats index overhead.
- Unused indexes still cost writes, storage and backup time.
- Wrapping the column in a function (WHERE LOWER(email) = ...) makes a plain index unusable - you need an expression index.

Example

orders is queried by customer plus date constantly. A composite index serves both the filter and the sort.

Code Example

```
-- Composite index: column order matters. Leftmost-prefix rule.
```

```

CREATE INDEX idx_orders_customer_date
    ON orders (customer_id, order_date DESC);

-- USES the index (leading column present)
EXPLAIN ANALYZE
SELECT order_id, order_date, status
FROM   orders
WHERE  customer_id = 1
ORDER  BY order_date DESC;

-- CANNOT use it efficiently: skips the leading column
SELECT order_id FROM orders WHERE order_date = '2025-04-02';

-- Partial index: only the rows anybody queries (open orders)
CREATE INDEX idx_orders_open
    ON orders (order_date)
    WHERE status IN ('PENDING', 'SHIPPED');

-- Covering index: answer entirely from the index, no heap fetch
CREATE INDEX idx_orders_cover
    ON orders (customer_id) INCLUDE (status, order_date);

-- Expression index: makes case-insensitive lookup indexable
CREATE INDEX idx_customers_email_lower ON customers (LOWER(email));

-- Find indexes nobody uses - candidates for deletion
SELECT relname AS table_name, indexrelname AS index_name, idx_scan AS times_used
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
ORDER  BY relname;

```

Output

```

                        QUERY PLAN
-----
Index Scan using idx_orders_customer_date on orders
  Index Cond: (customer_id = 1)
  Planning Time: 0.180 ms
  Execution Time: 0.041 ms
(4 rows)

```

On a seven-row table Postgres may still choose a sequential scan - correctly, because it is cheaper. Demonstrating awareness of that is a plus, not a failure.

Why Interviewers Ask This

Indexing is where SQL knowledge meets production reality. Anyone can say "indexes make queries faster"; the signal is in the costs, the leftmost-prefix rule, and knowing that Postgres has no clustered index.

Common Mistakes

- "Add an index to every column." This is a write-performance disaster.
- Not knowing composite index column order matters - (a, b) serves WHERE a, and WHERE a AND b, but not WHERE b alone.
- Claiming Postgres primary keys are clustered.
- Forgetting that a function on the indexed column disables the index.
- Ignoring index bloat and never checking pg_stat_user_indexes.

Best Practices

- Index for your actual query patterns, verified with EXPLAIN ANALYZE and pg_stat_statements.
- Put the equality column first in a composite index, the range or sort column second.
- Use partial indexes for skewed data - the "open orders" case is often 1 percent of the table.
- Create indexes with CONCURRENTLY in production to avoid write locks.
- Audit and drop unused indexes on a schedule.

Follow-up Questions

1. Why does (customer_id, order_date) not help WHERE order_date = ??
2. What is a covering index and how do you confirm an index-only scan?
3. When would a hash, GIN or BRIN index beat a B-tree?
4. What is index bloat and how does REINDEX CONCURRENTLY help?
5. How do indexes interact with NULL values?

Real-world Scenario

A checkout service degraded after a growth push. pg_stat_user_indexes showed 14 indexes on orders, 9 of them never scanned - accumulated from years of one-off reports. Each insert was maintaining all 14. Dropping the unused ones cut p99 insert latency from 40ms to 9ms without touching a single query.

Question 18

Difficulty: Medium

Category: SQL -> Transactions, ACID

Question

Explain ACID. Show a transaction that must be atomic and how you would implement it safely.

Answer

ACID is the set of guarantees a relational database makes about a transaction - a unit of work that either fully happens or does not happen at all.

Property	Guarantee	Mechanism	Failure it prevents
Atomicity	all statements or none	undo log / rollback segment	money debited but never cre~
Consistency	constraints hold before an~	CHECK, FK, UNIQUE, triggers	negative stock, orphan rows
Isolation	concurrent transactions do~	locks, MVCC	lost updates, dirty reads
Durability	committed data survives a ~	write-ahead log flushed to~	"we committed it" then powe~

The classic example is a transfer: debit one account, credit another. A crash between the two statements without atomicity destroys money. The BEGIN ... COMMIT block makes both visible at the same instant, and ROLLBACK undoes everything if any step fails.

Durability specifically means the WAL is fsynced before COMMIT returns. This is why synchronous_commit = off is faster and why it can lose the last few transactions on power failure - a trade-off interviewers like to probe.

Example

Placing an order must do three things together: decrement stock, insert the order, record the payment. Any partial application corrupts the business state.

Code Example

```
-- Safe order placement. Either all three effects happen, or none do.
BEGIN;

-- 1. Reserve stock. The WHERE clause is the concurrency guard:
-- it only succeeds if stock is still available.
UPDATE products
SET    stock_qty = stock_qty - 1
WHERE  product_id = 1
      AND stock_qty >= 1;

-- If zero rows were updated, stock ran out -> abort.
-- In application code: if (rowsAffected == 0) throw -> ROLLBACK.

-- 2. Create the order
INSERT INTO orders (customer_id, order_date, status)
VALUES (2, CURRENT_DATE, 'PENDING')
RETURNING order_id;          -- say it returns 9

-- 3. Line item + payment, using that id
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (9, 1, 1, 89999.00);

INSERT INTO payments (order_id, amount, method)
VALUES (9, 89999.00, 'UPI');

COMMIT;      -- atomic: all four statements become visible together
-- ROLLBACK; -- on any error: nothing happened at all


// Node.js with node-postgres: the correct try/catch/finally shape
const client = await pool.connect();
try {
  await client.query('BEGIN');

  const stock = await client.query(
    `UPDATE products SET stock_qty = stock_qty - $2
     WHERE product_id = $1 AND stock_qty >= $2`,
    [productId, qty]
  );
  if (stock.rowCount === 0) throw new Error('OUT_OF_STOCK'); // triggers rollback

  const { rows } = await client.query(
    `INSERT INTO orders (customer_id, order_date, status)
     VALUES ($1, CURRENT_DATE, 'PENDING') RETURNING order_id`,
    [customerId]
  );
  const orderId = rows[0].order_id;

  await client.query(
    `INSERT INTO order_items (order_id, product_id, quantity, unit_price)
     VALUES ($1, $2, $3, $4)`,
    [orderId, productId, qty, price]
  );
}
```

```

    await client.query('COMMIT');
    return orderId;
} catch (err) {
    await client.query('ROLLBACK');    // atomicity in practice
    throw err;
} finally {
    client.release();                // never leak the connection
}

```

Output

```

-- successful path
UPDATE 1
  order_id
-----
      9
INSERT 0 1
INSERT 0 1
COMMIT

-- out-of-stock path
UPDATE 0
ROLLBACK
-- verification: nothing was written
SELECT COUNT(*) FROM orders WHERE order_id = 9; --> 0

```

Why Interviewers Ask This

ACID is the vocabulary of data correctness, and the interviewer wants to know whether you write BEGIN/COMMIT deliberately or rely on autocommit and hope. The WHERE stock_qty >= 1 guard is the detail that shows you have thought about two users buying the last item simultaneously.

Common Mistakes

- Reciting the acronym without being able to name a bug each property prevents.
- Reading stock, checking it in application code, then updating - a race condition. Do the check in the WHERE clause or lock the row with SELECT ... FOR UPDATE.
- No ROLLBACK in the error path, leaving the transaction open and holding locks.
- Not releasing the connection in finally - connection pool exhaustion under load.
- Long transactions spanning user think-time or external HTTP calls.

Best Practices

- Keep transactions short: no network calls, no user interaction inside them.
- Push invariants into the schema (CHECK (stock_qty >= 0)) so a logic bug cannot violate them.
- Make retries safe with an idempotency key so a client retry does not double-charge.
- Set an explicit statement_timeout and idle_in_transaction_session_timeout.

Follow-up Questions

1. What exactly does durability mean, and what does synchronous_commit = off give up?
2. How do two concurrent buyers of the last unit interleave, and who wins?
3. What is a savepoint and when is a nested rollback useful?
4. How do distributed transactions (2PC) or the Saga pattern change these guarantees?

5. Where does consistency in ACID differ from consistency in CAP?

Real-world Scenario

A wallet service credited the recipient in one transaction and debited the sender in another, for "simpler code". A deploy restarted pods between the two calls during a spike, and roughly 900 transfers credited without debiting - about 4 lakh rupees created out of nothing. The fix was a single transaction plus a reconciliation job that replays the ledger nightly.

Question 19

Difficulty: Hard

Category: SQL -> Isolation levels

Question

Name the four SQL isolation levels and the anomalies each prevents. Which does PostgreSQL use by default, and how does it differ from the standard?

Answer

Isolation controls what one transaction can see of another's uncommitted or concurrent work. Weaker isolation means more concurrency and more anomalies.

The anomalies:

- Dirty read - reading another transaction's uncommitted data.
- Non-repeatable read - re-reading the same row and getting a different value because someone else committed an UPDATE in between.
- Phantom read - re-running the same range query and finding new rows that another transaction inserted.
- Lost update - two transactions read, both write, the second overwrites the first.
- Write skew - both transactions read an overlapping set, each makes a decision that is individually valid, and the combination violates an invariant. Only **SERIALIZABLE** prevents it.

Level	Dirty read	Non-repeatable read	Phantom	Write skew
-----	-----	-----	-----	-----
READ UNCOMMITTED	possible	possible	possible	possible
READ COMMITTED	prevented	possible	possible	possible
REPEATABLE READ	prevented	prevented	possible (per standard)	possible
SERIALIZABLE	prevented	prevented	prevented	prevented

Engine reality, which is what interviews actually reward:

- PostgreSQL default is **READ COMMITTED**. Every statement sees a fresh snapshot. **READ UNCOMMITTED** is accepted but behaves as **READ COMMITTED** - Postgres cannot do dirty reads at all, because MVCC never exposes uncommitted tuples. Postgres **REPEATABLE READ** is snapshot isolation and does prevent phantoms, which is stricter than the standard requires. **SERIALIZABLE** uses Serializable Snapshot Isolation (SSI) and can abort with **ERROR: could not serialize access - applications must retry**.

- MySQL InnoDB default is REPEATABLE READ, and it blocks phantoms via next-key (gap) locks.
- SQL Server default is READ COMMITTED using shared locks, unless READ_COMMITTED_SNAPSHOT is enabled.

Example

Two support agents both refund the last available credit. Under READ COMMITTED each reads a balance of 100, each subtracts 100, and the account ends at -100. The invariant "balance >= 0" is violated even though each transaction was individually correct - write skew.

Code Example

```
-- ===== Demonstration of a non-repeatable read =====
-- Session A                                | Session B
BEGIN;                                     -- |
SELECT balance FROM payments              -- |
WHERE payment_id = 1;  --> 50000          -- |
                                           -- | BEGIN;
                                           -- | UPDATE payments SET balance = 40000
                                           -- | WHERE payment_id = 1;
                                           -- | COMMIT;

SELECT balance FROM payments              -- |
WHERE payment_id = 1;  --> 40000 (changed!) -- | READ COMMITTED: value changed
COMMIT;

-- Same script at REPEATABLE READ: the second SELECT still returns 50000,
-- because the whole transaction sees one snapshot taken at first statement.
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM payments WHERE payment_id = 1;  --> 50000
-- ... concurrent committed update happens ...
SELECT balance FROM payments WHERE payment_id = 1;  --> 50000 (stable)
COMMIT;

-- ===== Preventing lost update, two correct ways =====
-- (a) Pessimistic: lock the row until commit
BEGIN;
SELECT balance FROM payments WHERE payment_id = 1 FOR UPDATE; -- others block here
UPDATE payments SET balance = balance - 100 WHERE payment_id = 1;
COMMIT;

-- (b) Optimistic: make the write itself atomic and conditional
UPDATE payments
SET    balance = balance - 100
WHERE  payment_id = 1
AND    balance >= 100;    -- 0 rows affected => rejected, no negative balance

-- ===== SERIALIZABLE requires a retry loop =====
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- ... reads and writes ...
COMMIT; -- may raise SQLSTATE 40001: could not serialize access
        -- correct handling: catch 40001 and retry the whole transaction
```

Output

```
-- READ COMMITTED session A
balance
-----
50000
balance
-----
40000      <-- non-repeatable read
```

```
-- REPEATABLE READ session A
balance
-----
50000
balance
-----
50000      <-- stable snapshot

-- optimistic update when balance is only 50
UPDATE 0      <-- rejected safely, invariant preserved
```

Why Interviewers Ask This

This is the senior-level concurrency question. Anyone can list four level names; the signal is whether you can (1) name the anomaly each prevents, (2) know your engine's default and its deviations from the standard, and (3) describe the retry requirement of `SERIALIZABLE`.

Common Mistakes

- Saying Postgres defaults to `SERIALIZABLE`, or that `READ UNCOMMITTED` allows dirty reads in Postgres. Neither is true.
- Confusing non-repeatable read (a changed row) with phantom read (new rows).
- Choosing `SERIALIZABLE` without implementing retry on `SQLSTATE 40001` - the app then throws errors at users under load.
- Assuming `SELECT ... FOR UPDATE` inside `READ COMMITTED` prevents phantoms. It locks existing rows only; new inserts still appear.
- Believing higher isolation fixes bad schema design.

Best Practices

- Stay at the engine default; raise isolation only for the specific transaction that needs it.
- Prefer a conditional `UPDATE ... WHERE` (optimistic) or a version column over row locking - less blocking, no deadlock class.
- If you use `SERIALIZABLE`, wrap every transaction in a bounded retry with jitter.
- Enforce invariants with constraints as the final backstop: `CHECK (balance >= 0)`.
- Order writes consistently across the codebase to reduce deadlock probability.

Follow-up Questions

1. What is write skew and why does only `SERIALIZABLE` stop it?
2. How does MVCC let Postgres avoid dirty reads entirely?
3. Difference between `FOR UPDATE`, `FOR NO KEY UPDATE` and `FOR SHARE`?
4. What is a next-key lock in InnoDB and which anomaly does it target?
5. How would you implement optimistic locking with a version column?

Real-world Scenario

A seat-booking system checked availability with a `SELECT` and then inserted a booking, at `READ COMMITTED`. Under a flash sale two requests read "1 seat left" microseconds apart and both inserted - the flight was oversold by 30 seats across the event. The fix combined a unique constraint on `(flight_id, seat_no)` with an optimistic conditional update, so the database rejected the second writer instead of trusting application-side checks.

Question 20

Difficulty: Medium

Category: SQL -> Security, SQL injection

Question

What is SQL injection? Show a vulnerable query, exploit it, and fix it properly.

Answer

SQL injection happens when untrusted input is concatenated into SQL text, so the input can change the structure of the statement rather than only supplying a value. The database cannot tell the difference - by the time it parses the string, attacker text is indistinguishable from developer text.

The only complete fix is to stop building SQL from strings: use ****parameterised queries / prepared statements****, where the driver sends the SQL template and the values on separate channels. The value is then never parsed as SQL, no matter what it contains.

Defence layers, strongest first:

1. Parameterised queries - always. This is the actual fix.
2. Allow-listing for identifiers. Table and column names cannot be parameterised, so if a sort column comes from the user, map it through a dictionary of permitted values.
3. Least privilege. The application role should not own the schema or hold SUPERUSER; injection then cannot DROP TABLE.
4. Input validation as defence in depth - never as the primary control.
5. Stored procedures help only if they themselves avoid dynamic SQL; EXECUTE inside a procedure is just as vulnerable.

Escaping quotes by hand is not a fix: encodings, numeric contexts, and second-order injection (stored data used to build a later query) all defeat it.

Example

A login form and a product search, both built by concatenation.

Code Example

```
// ===== VULNERABLE =====
// Never do this. String concatenation of user input.
async function loginUnsafe(email, password) {
  const sql = `SELECT customer_id, full_name FROM customers
              WHERE email = '${email}' AND password_hash = '${password}'`;
  return (await pool.query(sql)).rows;
}
// Attacker enters email: ' OR '1'='1' --
// Resulting SQL:
//   SELECT customer_id, full_name FROM customers
//   WHERE email = '' OR '1'='1' --' AND password_hash = '...'`
```

```
// The -- comments out the password check. Everyone logs in as the first row.

// Worse, with a stacked statement where the driver permits it:
// email = ''; DROP TABLE payments; --

// ===== SAFE: parameterised =====
async function loginSafe(email, passwordHash) {
  const sql = `SELECT customer_id, full_name
                FROM   customers
                WHERE  email = $1 AND password_hash = $2`;
  // $1/$2 are bound as VALUES; the string '' OR '1'='1' is just a string
  return (await pool.query(sql, [email, passwordHash])).rows;
}

// ===== SAFE: dynamic sort column via allow-list =====
const SORTABLE = {
  name: 'full_name',
  date: 'signup_date',
  city: 'city',
};
async function listCustomers(sortKey = 'name', dir = 'asc', limit = 20) {
  const column = SORTABLE[sortKey] ?? 'full_name'; // never user text
  const direction = dir.toLowerCase() === 'desc' ? 'DESC' : 'ASC';
  const sql = `SELECT customer_id, full_name, city
                FROM   customers
                ORDER  BY ${column} ${direction}
                LIMIT  $1`;
  return (await pool.query(sql, [limit])).rows;
}

-- Parameterised at the SQL level, for comparison
PREPARE find_customer (text) AS
  SELECT customer_id, full_name FROM customers WHERE email = $1;

EXECUTE find_customer ('aarav@example.com');
EXECUTE find_customer ('' OR '1'='1'); -- treated as a literal, returns 0 rows

-- Least privilege: the app role cannot drop anything
CREATE ROLE app_user LOGIN PASSWORD 'strong';
GRANT SELECT, INSERT, UPDATE ON orders, order_items TO app_user;
REVOKE ALL ON payments FROM app_user;
```

Output

```
-- vulnerable version with email = ' OR '1'='1' --
customer_id | full_name
-----+-----
          1 | Aarav Sharma    <-- logged in with no password

-- parameterised version, same input
customer_id | full_name
-----+-----
(0 rows)          <-- treated as a literal string
```

Why Interviewers Ask This

It is the most exploited web vulnerability class in history and it is entirely preventable, so it is a direct test of professional hygiene. Interviewers also listen for whether you know the limits of parameterisation - identifiers cannot be bound - because that is where real applications still get breached.

Common Mistakes

- Believing an ORM makes you immune. Raw-SQL escape hatches (query, raw, whereRaw, @Query with string concatenation) reintroduce the hole.
- Escaping quotes manually or stripping keywords with a blocklist.
- Trying to parameterise a table or column name and then falling back to concatenation.
- Forgetting second-order injection: stored input reused later to build SQL.
- Running the app as the database owner, so a successful injection is catastrophic rather than merely bad.

Best Practices

- Parameterise everything; make string-concatenated SQL a lint error in CI.
- Allow-list identifiers for dynamic ORDER BY and dynamic table names.
- Grant the application role the minimum privileges; separate migration and runtime roles.
- Never store passwords with reversible or fast hashes - use Argon2id or bcrypt.
- Add a static analysis / SAST rule and a dependency scan; log and alert on query errors that smell like probing.

Follow-up Questions

1. Why can a table name not be a bind parameter?
2. Are stored procedures automatically safe? (No - dynamic EXECUTE inside is vulnerable.)
3. What is blind SQL injection and how is it detected without visible output?
4. How does least privilege limit blast radius after a successful injection?
5. What is second-order SQL injection?

Real-world Scenario

An internal admin tool let staff sort a report by clicking column headers, passing the column name straight into ORDER BY. A contractor's browser extension appended a payload to the query string, and the tool executed a subquery that exfiltrated the customer email table one row at a time via sort order. Parameterisation would not have helped - the fix was the allow-list map. The audit also found the tool was connecting as the schema owner.

SQL Coding Challenges (Set 1: Questions 1-20)

Attempt each in under 10 minutes, then compare. These are the shapes that recur constantly in screening rounds.

C1. Return the second-highest distinct salary from employees without using LIMIT/OFFSET.

```
SELECT MAX(salary) AS second_highest
FROM   employees
WHERE  salary < (SELECT MAX(salary) FROM employees);
-- 210000.00
```

C2. Find customers who have never placed an order, three different ways, and say which you

would ship.

```
-- (a) anti-join: fine, plans well
SELECT c.full_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- (b) NOT EXISTS: preferred - NULL-safe and self-documenting
SELECT c.full_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- (c) NOT IN: DANGEROUS - returns nothing if any customer_id is NULL
SELECT c.full_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders);
-- Ship (b). Answer: Sara Khan
```

C3. Top 2 highest-paid employees per department.

```
SELECT full_name, department, salary
FROM (SELECT full_name, department, salary,
            DENSE_RANK() OVER (PARTITION BY department
                               ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk <= 2
ORDER BY department, salary DESC;
-- Engineering: Nisha 320000, Karan 210000
-- Sales:       Farah 150000, Vikram 120000
-- Support:     Tara 90000
```

C4. Delete duplicate rows keeping the lowest id (assume a table with repeats).

```
DELETE FROM some_table s
USING (SELECT id,
            ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
      FROM some_table) d
WHERE s.id = d.id AND d.rn > 1;
```

C5. Running total of revenue by order date.

```
SELECT o.order_date,
       SUM(oi.quantity * oi.unit_price) AS daily,
       SUM(SUM(oi.quantity * oi.unit_price)) OVER (ORDER BY o.order_date
                                                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

C6. Orders with no payment recorded (a reconciliation query every payments team runs).

```
SELECT o.order_id, o.status
FROM   orders o
WHERE  NOT EXISTS (SELECT 1 FROM payments p WHERE p.order_id = o.order_id)
       AND o.status <> 'CANCELLED';
-- orders 5 and 8 (both PENDING); order 4 excluded because it was cancelled
```

C7. For each customer, days between their first and most recent order.

```
SELECT c.full_name,
       MIN(o.order_date) AS first_order,
       MAX(o.order_date) AS last_order,
       MAX(o.order_date) - MIN(o.order_date) AS days_active
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.full_name
```

```

HAVING COUNT(*) > 1
ORDER BY days_active DESC;
-- Aarav: 2025-01-10 -> 2025-04-02 = 82 days
-- Diya: 2025-02-20 -> 2025-05-09 = 78 days

```

Revision Notes: SQL Questions 1-20

Clause execution order (the one list that explains most SQL errors)

FROM -> ON -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT

Mnemonic: Friendly Owls Will Gladly Help Small Ducks Order

Lunch. Aliases are born in SELECT, so only ORDER BY (and LIMIT) can see them.

NULL rules

Expression	Result
-----	-----
NULL = NULL	UNKNOWN (row dropped)
x NOT IN (1, NULL)	never returns rows
COUNT(col)	skips NULL
AVG(col)	divides by non-NULL count
SUM(all NULL)	NULL, not 0
GROUP BY on NULL	all NULLs form one group

Use IS NULL, IS DISTINCT FROM, NOT EXISTS, COALESCE, and x / NULLIF(y,0).

Join decision table

Need	Use
-----	-----
Only matching rows	INNER JOIN
All left rows, zeros for missing	LEFT JOIN + COUNT(right_col)
Rows with no match	NOT EXISTS (or LEFT JOIN ... IS NULL)
Filter the outer table	condition in ON, never WHERE
Row-within-table relationship	self join (one level) / recursive CTE (all levels)

The ON vs WHERE rule: a filter on the outer table in WHERE turns LEFT JOIN into INNER JOIN. Only WHERE right.key IS NULL is intentional.

Window function ties

320k, 210k, 185k, 185k	Result
-----	-----
ROW_NUMBER()	1, 2, 3, 4
RANK()	1, 2, 3, 3 (next = 5)
DENSE_RANK()	1, 2, 3, 3 (next = 4)

Windows run after HAVING, so never filter a window function in WHERE - wrap it.

Recursive CTE = anchor UNION ALL recursive member; stops when an iteration yields zero rows; always add a depth guard and a cycle guard.

Index quick rules - composite (a, b) serves a and a AND b, not b alone. Functions on the column disable the index. Every index taxes writes. Postgres has no clustered index.

ACID - Atomicity (all or none), Consistency (constraints), Isolation (concurrency), Durability (survives crash, via fsynced WAL).

Isolation defaults - Postgres READ COMMITTED (no dirty reads ever; REPEATABLE READ also blocks phantoms; SERIALIZABLE needs retry on SQLSTATE 40001). MySQL InnoDB REPEATABLE READ with gap locks.

Injection - parameterise values, allow-list identifiers, least privilege. An ORM is not a guarantee.

Common misconceptions cleared

1. "CTEs are always slower in Postgres" - false since version 12; single-reference CTEs inline.
2. "SELECT COUNT(1) is faster than COUNT(*)" - identical plans.
3. "Indexes only help" - they tax every write and can be ignored by the planner.
4. "READ UNCOMMITTED gives dirty reads in Postgres" - it does not; MVCC forbids it.
5. "HAVING and WHERE are interchangeable" - only accidentally, for non-aggregate predicates.
6. "An ORM prevents SQL injection" - only until the first raw call.

Format note for Questions 21 onward. The same ten-part structure is used throughout, in a tighter form: explanations are compressed to their load-bearing points so the book can cover the full syllabus. Depth is unchanged where it matters (code, output, mistakes, follow-ups).

Question 21

Difficulty: Easy

Category: SQL -> Keys

Question

Explain super key, candidate key, primary key, composite key, unique key and foreign key.

Answer

A super key is any set of columns that uniquely identifies a row - including redundant ones. A candidate key is a minimal super key (remove any column and uniqueness breaks). The designer picks one candidate key as the primary key; the rest become alternate keys, usually enforced with UNIQUE. A composite key is any key made of two or more columns. A foreign key references a key in another table and enforces referential integrity.

Key	Unique	Allows NULL	Count per table
Primary	yes	no	1
Unique	yes	yes (multiple NULLs in PG/MySQL)	many
Foreign	no	yes	many

In our schema: customers.customer_id is the primary key, customers.email is a candidate key enforced as UNIQUE, {customer_id, email} is a super key but not a candidate key, and order_items(order_id, product_id) is a composite primary key.

Example

order_items needs both columns to identify a line: order 1 contains two different products.

Code Example

```
-- composite primary key: neither column alone is unique
SELECT order_id, product_id, quantity FROM order_items WHERE order_id = 1;

-- alternate key enforcement; this violates the UNIQUE constraint on email
INSERT INTO customers (full_name, email) VALUES ('Fake Aarav', 'aarav@example.com');

-- foreign key protection: no such customer
INSERT INTO orders (customer_id, order_date, status)
VALUES (999, CURRENT_DATE, 'PENDING');
```

Output

order_id	product_id	quantity
1	1	1
1	3	2

```
ERROR: duplicate key value violates unique constraint "customers_email_key"
ERROR: insert or update on table "orders" violates foreign key constraint
DETAIL: Key (customer_id)=(999) is not present in table "customers".
```

Why Interviewers Ask This

It is the vocabulary check for database design. Candidates who cannot distinguish candidate key from super key usually also cannot justify a schema.

Common Mistakes

- Calling every unique column a primary key.
- Believing a super key must be minimal - that is the candidate key.
- Assuming UNIQUE forbids NULL. In Postgres and MySQL multiple NULLs are allowed.
- Using a natural key (email, phone) as the primary key, then discovering users change them.

Best Practices

- Prefer a surrogate primary key (SERIAL, BIGSERIAL, UUID) and enforce natural keys with UNIQUE.
- Always index foreign keys - Postgres does not do it automatically.
- Choose ON DELETE behaviour deliberately: CASCADE, RESTRICT or SET NULL.

Follow-up Questions

1. When is a natural primary key the right choice?
2. UUID versus BIGSERIAL as a primary key - index locality trade-offs?
3. What breaks if you skip the index on a foreign key?
4. What is a surrogate key and what are its downsides?

Real-world Scenario

A billing system used email as the customer primary key. When a customer changed email, the ON UPDATE CASCADE rewrote 400,000 invoice rows and locked the table for eleven minutes during business hours. Migrating to a surrogate key removed the whole failure class.

Question 22

Difficulty: Medium

Category: SQL -> Set operators

Question

Compare UNION, UNION ALL, INTERSECT and EXCEPT.

Answer

Set operators combine the rows of two result sets that have the same column count and compatible types (unlike joins, which combine columns).

Operator	Returns	Removes duplicates	Cost
UNION	rows in either	yes	sort or hash to dedupe
UNION ALL	rows in either	no	cheapest - just concatenation
INTERSECT	rows in both	yes	dedupe
EXCEPT (MINUS in Oracle)	rows in the first but not in the second	yes	dedupe

UNION ALL is the default choice. Reach for UNION only when duplicates are actually possible and undesirable - the deduplication is a real sort. For set operators, NULLs are treated as equal to each other, unlike in = comparisons.

ORDER BY applies to the combined result and must appear once, at the end.

Example

Contact list of Indian customers plus everyone who ever contacted support, and the set difference between cities that ordered and cities that did not.

Code Example

```
-- UNION removes the overlap (Aarav is both Indian and a support contact)
SELECT full_name, 'india' AS source FROM customers WHERE country = 'India'
UNION
SELECT full_name, 'support' AS source FROM customers WHERE last_support_contact IS NOT NULL
ORDER BY full_name;

-- INTERSECT: customers who are BOTH Indian and have contacted support
SELECT full_name FROM customers WHERE country = 'India'
INTERSECT
SELECT full_name FROM customers WHERE last_support_contact IS NOT NULL;
```



```
-- EXCEPT: Indian customers who never contacted support
SELECT full_name FROM customers WHERE country = 'India'
EXCEPT
SELECT full_name FROM customers WHERE last_support_contact IS NOT NULL;
```

Output

```
-- UNION (5 rows; the 'source' column makes rows distinct, so both Aarav rows survive)
full_name | source
-----+-----
Aarav Sharma | india
Aarav Sharma | support
Diya Patel | india
Meera Iyer | india
Meera Iyer | support
Rohan Verma | india

-- INTERSECT
full_name
-----
Aarav Sharma
Meera Iyer

-- EXCEPT
full_name
-----
Diya Patel
Rohan Verma
```

Note the trap in the first query: adding the source literal makes the rows differ, so UNION cannot collapse them. Deduplication compares the whole row.

Why Interviewers Ask This

UNION versus UNION ALL is a cheap performance question with a clear right answer, and the whole-row deduplication subtlety catches people who assume it dedupes on the first column.

Common Mistakes

- Using UNION habitually and paying for a sort on every query.
- Expecting EXCEPT to be symmetric - it is not; order matters.
- Mismatched column counts or incompatible types.
- Putting ORDER BY in each branch instead of once at the end.

Best Practices

- Default to UNION ALL; justify UNION when you use it.
- Keep column lists explicit and aligned across branches.
- For anti-set logic on large tables, benchmark EXCEPT against NOT EXISTS - the latter often wins because it can stop early.

Follow-up Questions

1. Why does UNION treat NULL = NULL as equal here but WHERE does not?
2. Rewrite the EXCEPT query with NOT EXISTS and compare plans.

3. How would you emulate FULL OUTER JOIN with UNION ALL?
4. Does UNION ALL preserve input order? (No guarantee without ORDER BY.)

Real-world Scenario

A data pipeline merged 40 daily partitions with UNION "to be safe". The dedupe sorted 200 million rows nightly and needed 30 GB of temp space. The partitions were disjoint by construction, so UNION ALL was correct - runtime dropped from 25 minutes to 90 seconds.

Question 23

Difficulty: Medium

Category: SQL -> Pagination

Question

How do you paginate results? Why does OFFSET degrade, and what is keyset pagination?

Answer

LIMIT n OFFSET m is the obvious approach and it is $O(m + n)$: the database must generate and discard the first m rows before returning anything. At OFFSET 500000 it reads half a million rows to return twenty. There is a second, worse problem: if rows are inserted or deleted between requests, the window shifts and users see duplicated or skipped rows.

Keyset (cursor) pagination instead remembers the last row's sort key and asks for rows after it:

```
WHERE (order_date, order_id) < (:last_date, :last_id)
ORDER BY order_date DESC, order_id DESC
LIMIT 20
```

This is $O(\log n + \text{limit})$ with a matching index, and it is stable under concurrent writes.

The trade-off: you cannot jump to an arbitrary page number, only next/previous. That is acceptable for feeds and infinite scroll, not for a "page 47" control.

The row-constructor comparison $(a, b) < (x, y)$ is the correct way to break ties; comparing only the date loses rows that share a timestamp.

Example

Paging through orders newest first, 3 per page.

Code Example

```
-- OFFSET pagination: simple, degrades with depth, unstable under writes
SELECT order_id, order_date FROM orders
ORDER BY order_date DESC, order_id DESC
LIMIT 3 OFFSET 3;
```

```
-- Keyset pagination: page 1
SELECT order_id, order_date FROM orders
ORDER BY order_date DESC, order_id DESC
LIMIT 3;
-- last row returned was (2025-04-02, 6) -> feed it back as the cursor

-- Keyset pagination: page 2, using the row constructor for correct tie-breaking
SELECT order_id, order_date FROM orders
WHERE (order_date, order_id) < (DATE '2025-04-02', 6)
ORDER BY order_date DESC, order_id DESC
LIMIT 3;

-- Index that makes keyset pagination O(log n)
CREATE INDEX idx_orders_keyset ON orders (order_date DESC, order_id DESC);

-- Total count for a UI page indicator is expensive; estimate instead
SELECT reltuples::bigint AS estimated_rows FROM pg_class WHERE relname = 'orders';
```

Output

```
-- OFFSET 3 (page 2)
order_id | order_date
-----+-----
        6 | 2025-04-02
        5 | 2025-03-15
        3 | 2025-02-20

-- keyset page 1
order_id | order_date
-----+-----
        8 | 2025-05-09
        7 | 2025-04-18
        6 | 2025-04-02

-- keyset page 2
order_id | order_date
-----+-----
        5 | 2025-03-15
        3 | 2025-02-20
        2 | 2025-02-14
```

Why Interviewers Ask This

Deep OFFSET is one of the most common real-world performance bugs, and keyset pagination is the standard fix used by every large feed. Knowing it signals production experience.

Common Mistakes

- LIMIT without ORDER BY - arbitrary, unstable results.
- Ordering by a non-unique column only, so rows with equal values are duplicated or skipped across pages.
- Running COUNT(*) over the whole table on every page request.
- Comparing cursor columns separately with AND, which is not equivalent to the row constructor.

Best Practices

- Keyset pagination for feeds and APIs; opaque, signed cursor tokens rather than raw ids.
- Always end ORDER BY with a unique column.

- Cap limit server-side and reject unbounded requests.
- Estimate totals from `pg_class.reltuples` or omit exact counts entirely.

Follow-up Questions

1. How do you support "previous page" with keyset pagination?
2. Why must the cursor columns match the index order exactly?
3. How would you paginate when the sort column is user-selectable?
4. What breaks if the cursor row is deleted between requests? (Nothing - comparison still works.)

Real-world Scenario

An admin console let staff page through 12 million audit rows. Page 1 was instant, page 4000 timed out at 30 seconds and pinned a CPU. Switching to keyset pagination with a composite index made every page respond in under 15 ms, at the cost of removing the page-number jump control.

Question 24

Difficulty: Medium

Category: SQL -> Views, materialised views

Question

What is a view? How does a materialised view differ, and when do you use each?

Answer

A view is a stored query. It holds no data; every reference re-executes the underlying SQL.
Benefits: encapsulation, a stable interface over a changing schema, and column-level security.
Cost: no performance gain whatsoever - it is a macro.

A materialised view stores the result on disk. Reads are as fast as a table (and it can be indexed), but the data is a snapshot that goes stale until refreshed.

	View	Materialised view
Stores data	no	yes
Read cost	cost of the query	cost of a table scan or index lookup
Freshness	always current	stale until refreshed
Indexable	no	yes
Write-through	sometimes (simple views are updatable)	never

`REFRESH MATERIALIZED VIEW` takes an exclusive lock and blocks readers;
`REFRESH ... CONCURRENTLY` does not, but requires a `UNIQUE` index and does more work.

Use a view for abstraction and permissioning. Use a materialised view when an expensive aggregate is read far more often than the data changes, and minutes of staleness are acceptable.

Example

A customer revenue summary read on every dashboard load but recomputed only hourly.

Code Example

```
-- Plain view: always fresh, recomputed every time
CREATE VIEW v_customer_orders AS
SELECT c.customer_id, c.full_name, c.country,
       COUNT(o.order_id) AS order_count
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name, c.country;

SELECT full_name, order_count FROM v_customer_orders WHERE order_count = 0;

-- Materialised view: snapshot, indexable
CREATE MATERIALIZED VIEW mv_customer_revenue AS
SELECT c.customer_id, c.full_name,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS lifetime_value
FROM   customers c
LEFT   JOIN orders o      ON o.customer_id = c.customer_id AND o.status <> 'CANCELLED'
LEFT   JOIN order_items oi ON oi.order_id = o.order_id
GROUP  BY c.customer_id, c.full_name;

-- UNIQUE index is required for CONCURRENTLY, and speeds up lookups
CREATE UNIQUE INDEX mv_cust_rev_pk ON mv_customer_revenue (customer_id);

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_customer_revenue; -- no reader blocking

-- Security use of a plain view: hide the email column from analysts
CREATE VIEW v_customers_public AS
SELECT customer_id, full_name, city, country FROM customers;
GRANT SELECT ON v_customers_public TO analyst_role;
```

Output

```
full_name | order_count
-----+-----
Sara Khan |          0

REFRESH MATERIALIZED VIEW

SELECT full_name, lifetime_value FROM mv_customer_revenue ORDER BY 2 DESC LIMIT 3;
full_name | lifetime_value
-----+-----
Aarav Sharma |      197493.00
John Carter  |      16497.00
Diya Patel   |      13997.00
```

Why Interviewers Ask This

It tests whether you know that a view is not a cache - a very common misconception - and whether you can reason about the staleness-versus-speed trade-off.

Common Mistakes

- Believing a plain view improves performance.

- Nesting views five deep; the planner inlines them all and the resulting query is unreadable and often unoptimisable.
- Using REFRESH without CONCURRENTLY in production and blocking every reader.
- Forgetting the materialised view is stale, then reporting yesterday's numbers as live.

Best Practices

- Views for abstraction and permissions; materialised views for expensive read-heavy aggregates.
- Refresh on a schedule or a trigger, and expose last_refreshed_at in the UI.
- Keep view nesting to one level.
- Consider a summary table maintained incrementally when even CONCURRENTLY is too slow.

Follow-up Questions

1. When is a view updatable, and what is an INSTEAD OF trigger for?
2. How would you build incremental refresh rather than full recompute?
3. What locks does REFRESH MATERIALIZED VIEW take, with and without CONCURRENTLY?
4. Compare a materialised view with a Redis cache for the same aggregate.

Real-world Scenario

An analytics page ran a 14-second aggregate on every load. Wrapping it in a plain view changed nothing, which surprised the team. Converting to a materialised view refreshed every 10 minutes took page load to 40 ms, and the product owner agreed 10-minute staleness was fine for revenue trend charts.

Question 25

Difficulty: Hard

Category: SQL -> Locks, deadlocks

Question

What is a deadlock? Show how one forms and how you prevent it.

Answer

A deadlock is a cycle of lock waits: transaction A holds a lock B needs, and B holds a lock A needs. Neither can proceed. The database detects the cycle and kills one transaction (the victim) so the other completes. Postgres raises SQLSTATE 40P01.

```
T1 holds row 1 ---- waits for ----> row 2 held by T2
      ^                               |
      |_____ waits for row 1 _____|
```

Lock types worth naming: row-level (FOR UPDATE exclusive, FOR SHARE shared) and table-level (ACCESS SHARE for SELECT up to ACCESS EXCLUSIVE for DROP/ALTER). Deadlocks come mostly from inconsistent ordering of row updates across code paths, and from lock escalation caused

by long transactions.

Prevention, in order of effectiveness:

1. Always acquire locks in a deterministic order - for example ascending primary key. If every transaction locks row 1 before row 2, no cycle can form.
2. Keep transactions short - less overlap, less contention.
3. Use a single atomic statement where possible; one statement cannot deadlock with itself.
4. Retry on 40P01 with jitter - deadlocks are a normal, expected condition at scale.
5. SELECT ... FOR UPDATE NOWAIT or SKIP LOCKED for queue workloads.

Example

Two transfers between the same two accounts, in opposite directions.

Code Example

```
-- ===== DEADLOCK: opposite lock order =====
-- Session A                                | Session B
BEGIN;                                       -- BEGIN;
UPDATE payments SET balance = balance - 100 -- UPDATE payments SET balance = balance - 50
WHERE payment_id = 1; -- locks row 1        -- WHERE payment_id = 2; -- locks row 2
--
UPDATE payments SET balance = balance + 100 -- UPDATE payments SET balance = balance + 50
WHERE payment_id = 2; -- WAITS for B        -- WHERE payment_id = 1; -- WAITS for A
-- ERROR: deadlock detected
-- DETAIL: Process 812 waits for ShareLock on transaction 5541; blocked by process 840.

-- ===== FIX 1: deterministic lock order =====
BEGIN;
-- lock both rows up front, always in ascending id order
SELECT payment_id FROM payments
WHERE payment_id IN (1, 2)
ORDER BY payment_id -- the crucial line
FOR UPDATE;

UPDATE payments SET balance = balance - 100 WHERE payment_id = 1;
UPDATE payments SET balance = balance + 100 WHERE payment_id = 2;
COMMIT;

-- ===== FIX 2: single atomic statement, no ordering problem =====
UPDATE payments
SET balance = balance + CASE payment_id WHEN 1 THEN -100 ELSE 100 END
WHERE payment_id IN (1, 2);

-- ===== Queue pattern: never block, just take the next free row =====
SELECT order_id FROM orders
WHERE status = 'PENDING'
ORDER BY order_id
FOR UPDATE SKIP LOCKED
LIMIT 1;

-- Diagnose live blocking
SELECT pid, wait_event_type, wait_event, state,
       LEFT(query, 60) AS query
FROM pg_stat_activity
WHERE wait_event_type = 'Lock';
```

Output

```
-- Session A
ERROR: deadlock detected
DETAIL: Process 812 waits for ShareLock on transaction 5541; blocked by process 840.
HINT: See server log for query details.
CONTEXT: while updating tuple (0,3) in relation "payments"

-- Session B completes normally
UPDATE 1
COMMIT

-- after FIX 1, both sessions succeed
COMMIT
COMMIT
```

Why Interviewers Ask This

Deadlocks are a senior-level concurrency signal. The expected answer is not "the database handles it" but "order your locks deterministically and retry the victim".

Common Mistakes

- Thinking a deadlock hangs forever - the detector resolves it in about a second by default.
- Blaming isolation level. Deadlocks occur at every isolation level.
- Retrying the victim immediately with no backoff, causing a retry storm.
- Holding locks across an external HTTP call inside a transaction.
- Confusing a deadlock (cycle, auto-resolved) with lock contention (slow, not resolved).

Best Practices

- Codify a lock-ordering rule in the team's coding standard - "always by ascending primary key".
- Bounded retry with jitter on 40P01 and 40001.
- SKIP LOCKED for job queues; NOWAIT when the caller can fail fast.
- Set deadlock_timeout and log deadlocks; treat a rising rate as a design smell.
- Batch operations in a consistent order - sort ids before the loop.

Follow-up Questions

1. Difference between a deadlock and simple lock contention?
2. How does SKIP LOCKED implement a work queue safely?
3. Can SELECT statements deadlock with each other? (Not with plain reads under MVCC.)
4. What is lock escalation, and does Postgres do it? (No - SQL Server does.)
5. How would you find the two queries involved after the fact?

Real-world Scenario

An inventory service updated multiple SKUs per order by iterating the cart in the order the user added items. Two customers with overlapping carts in opposite order deadlocked several hundred times a day during sales. Sorting the cart by product_id before the update loop - a one-line change - eliminated them entirely.

Question 26

Difficulty: Medium

Category: SQL -> Normalisation

Question

Explain 1NF, 2NF, 3NF and BCNF with a single worked example. When would you denormalise?

Answer

Normalisation removes redundancy so that each fact is stored once, eliminating update anomalies.

- 1NF - atomic values, no repeating groups. No comma-separated lists in a column.
- 2NF - 1NF plus: no partial dependency. Every non-key column depends on the whole composite key, not part of it.
- 3NF - 2NF plus: no transitive dependency. Non-key columns must not depend on other non-key columns.
- BCNF - stricter 3NF: every determinant is a candidate key. Fixes the rare case where a non-key column determines part of a key.

Mnemonic: 1NF kills repeating groups, 2NF kills partial dependencies, 3NF kills transitive dependencies, BCNF kills the leftovers.

Denormalise deliberately when read performance demands it: precomputed totals, duplicated display names to avoid a join, or a reporting star schema. The price is that you now own consistency - every copy must be updated together, usually by trigger, application logic, or a scheduled reconciliation job. Denormalise for measured read pressure, never for convenience.

Example

A single flat order table, walked to BCNF.

Code Example

```
-- ===== 0NF / violates 1NF : repeating group in one column =====
-- order_id | customer      | products
--      1 | Aarav Sharma  | 'UltraBook, Keyboard, Keyboard'

-- ===== 1NF : atomic values, one row per product =====
CREATE TABLE orders_1nf (
    order_id      INT, product_id INT, product_name TEXT, unit_price NUMERIC,
    customer_id   INT, customer_name TEXT, customer_city TEXT, city_pincode TEXT,
    PRIMARY KEY (order_id, product_id)
);
-- Problems: product_name depends only on product_id (partial dependency),
-- customer_name depends only on order_id via customer_id (transitive).

-- ===== 2NF : remove partial dependencies on the composite key =====
CREATE TABLE products_2nf (
    product_id    INT PRIMARY KEY,
```

```

        product_name TEXT NOT NULL,
        unit_price   NUMERIC NOT NULL           -- depends on product_id alone
    );
CREATE TABLE order_items_2nf (
    order_id   INT, product_id INT REFERENCES products_2nf,
    quantity   INT NOT NULL,
    PRIMARY KEY (order_id, product_id)
);

-- ===== 3NF : remove the transitive dependency city -> pincode =====
CREATE TABLE cities_3nf (
    city       TEXT PRIMARY KEY,
    pincode    TEXT NOT NULL                   -- pincode depended on city, not on customer_id
);
CREATE TABLE customers_3nf (
    customer_id INT PRIMARY KEY,
    full_name   TEXT NOT NULL,
    city        TEXT REFERENCES cities_3nf
);
CREATE TABLE orders_3nf (
    order_id   INT PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customers_3nf,
    order_date  DATE NOT NULL
);

-- ===== Deliberate denormalisation, with a maintenance strategy =====
ALTER TABLE orders_3nf ADD COLUMN order_total NUMERIC(12,2);

CREATE OR REPLACE FUNCTION refresh_order_total() RETURNS TRIGGER AS $$
BEGIN
    UPDATE orders_3nf o
    SET     order_total = (SELECT COALESCE(SUM(quantity * p.unit_price), 0)
                           FROM   order_items_2nf oi
                           JOIN    products_2nf p ON p.product_id = oi.product_id
                           WHERE   oi.order_id = o.order_id)
    WHERE   o.order_id = COALESCE(NEW.order_id, OLD.order_id);
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_order_total
AFTER INSERT OR UPDATE OR DELETE ON order_items_2nf
FOR EACH ROW EXECUTE FUNCTION refresh_order_total();

```

Output

```

-- update anomaly BEFORE normalisation: a price change must touch every historical row
UPDATE orders_1nf SET unit_price = 79999 WHERE product_id = 1;
UPDATE 2           -- and any row you miss is now inconsistent

-- AFTER normalisation: one row holds the fact
UPDATE products_2nf SET unit_price = 79999 WHERE product_id = 1;
UPDATE 1

-- denormalised total kept correct by the trigger
INSERT INTO order_items_2nf VALUES (1, 3, 2);
SELECT order_id, order_total FROM orders_3nf WHERE order_id = 1;
order_id | order_total
-----+-----
1 |          9998.00

```

Why Interviewers Ask This

Normal forms are the standard DBMS interview topic and they reveal whether a candidate can spot

update anomalies in a schema. The denormalisation follow-up separates textbook knowledge from production judgement.

Common Mistakes

- Reciting definitions without being able to identify the violation in a given table.
- Confusing partial dependency (2NF) with transitive dependency (3NF).
- Normalising to 5NF everywhere and creating a nine-way join for every page load.
- Denormalising without any mechanism to keep copies consistent.
- Storing a historical price only in products - order history must snapshot the price paid, which is not a normalisation violation.

Best Practices

- Design to 3NF, then denormalise with evidence from query plans and latency data.
- Every denormalised column needs a documented owner: trigger, application, or batch job.
- Snapshot values that are historically meaningful (price at purchase) by design.
- Add a reconciliation job that asserts derived values match their source.

Follow-up Questions

1. Give a table in 3NF but not BCNF.
2. What are 4NF and 5NF, and does anyone use them?
3. How do star and snowflake schemas relate to normalisation?
4. Why is `order_items.unit_price` not a 3NF violation?
5. How would you validate a denormalised total without downtime?

Real-world Scenario

An order list page joined five tables to display a total and took 800 ms at p95. Adding a trigger-maintained `order_total` column dropped it to 25 ms. Six months later a bulk import bypassed the trigger with COPY, and totals silently drifted on 12,000 orders. The lesson was not "denormalisation is bad" but "every derived column needs a reconciliation job".

Question 27

Difficulty: Hard

Category: SQL -> Query optimisation, EXPLAIN

Question

Walk me through how you diagnose a slow query. What do you look for in EXPLAIN ANALYZE?

Answer

A repeatable method beats guessing:

1. Find the query. `pg_stat_statements` ordered by `total_exec_time` - optimise what actually costs the most, not what feels slow.
2. Get the real plan. `EXPLAIN (ANALYZE, BUFFERS)`. `ANALYZE` executes it and reports actual rows and time; `BUFFERS` shows cache hits versus disk reads.
3. Compare estimated versus actual rows. A large mismatch means bad statistics - the root cause of most bad plan choices. Fix with `ANALYZE`, a higher statistics target, or extended statistics for correlated columns.
4. Find the expensive node. Read the plan inside-out; look for the node where actual time jumps.
5. Check the access method. Seq Scan on a large table with a selective filter means a missing or unusable index. On a small table it is correct.
6. Check the join strategy. Nested Loop is great for few rows, catastrophic for many. Hash Join suits large unsorted sets; Merge Join suits pre-sorted input.
7. Look for spills. Sort Method: external merge Disk: 240MB means `work_mem` is too small.
8. Fix and re-measure. Index, rewrite, or configuration - then verify with the same plan.

Red flags summary:

Symptom	Likely cause
rows=1 estimated, rows=48000 actual	stale statistics
Seq Scan + selective WHERE	missing or unusable index
Nested Loop with large outer	bad row estimate
Sort ... Disk	work_mem too low
Rows Removed by Filter very high	index does not match the predicate
Heap Fetches high on index-only scan	table needs VACUUM

Example

A revenue query filtered on a function-wrapped date column.

Code Example

```
-- Slow: the function on the column makes any plain index on order_date unusable
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, COUNT(*)
FROM   orders
WHERE  EXTRACT(YEAR FROM order_date) = 2025
GROUP BY customer_id;

-- Fix A: rewrite as a sargable range so a B-tree index can be used
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, COUNT(*)
FROM   orders
WHERE  order_date >= DATE '2025-01-01'
      AND order_date <  DATE '2026-01-01'
GROUP BY customer_id;

-- Fix B: if the function form is unavoidable, index the expression
CREATE INDEX idx_orders_year ON orders ((EXTRACT(YEAR FROM order_date)));

-- Statistics maintenance
ANALYZE orders;
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
CREATE STATISTICS stat_orders_cust_status (dependencies)
  ON customer_id, status FROM orders;  -- correlated columns

-- Find the real offenders
SELECT LEFT(query, 60) AS query, calls,
```

```

        ROUND(total_exec_time::numeric, 1) AS total_ms,
        ROUND(mean_exec_time::numeric, 2) AS mean_ms
FROM   pg_stat_statements
ORDER  BY total_exec_time DESC
LIMIT  5;

```

Output

```

-- BEFORE: function on the column forces a full scan
HashAggregate (cost=1.32..1.38 rows=6 width=12) (actual time=0.089..0.092 rows=5 loops=1)
  -> Seq Scan on orders (cost=0.00..1.28 rows=4 width=4)
        (actual time=0.021..0.048 rows=8 loops=1)
        Filter: (EXTRACT(year FROM order_date) = 2025)
        Rows Removed by Filter: 0
        Buffers: shared hit=1
Execution Time: 0.142 ms

-- AFTER (on a large table): index range scan instead of a full scan
HashAggregate (actual time=12.4..12.9 rows=4812 loops=1)
  -> Index Only Scan using idx_orders_keyset on orders
        (actual time=0.031..6.204 rows=61233 loops=1)
        Index Cond: ((order_date >= '2025-01-01') AND (order_date < '2026-01-01'))
        Heap Fetches: 0
        Buffers: shared hit=412
Execution Time: 13.1 ms      -- was 1840 ms with the Seq Scan

```

Why Interviewers Ask This

This is the practical performance question for mid and senior roles. They want a method, plus the vocabulary - sargable, statistics, join strategies, spill to disk. Candidates who jump straight to "add an index" without reading the plan are marked down.

Common Mistakes

- Using EXPLAIN without ANALYZE and reasoning about estimates as if they were measurements.
- Optimising the query someone complained about instead of the top entry in pg_stat_statements.
- Adding indexes blindly, one per complaint, ending with 14 indexes on one table.
- Ignoring the estimated-versus-actual row gap, which is usually the actual root cause.
- Forgetting EXPLAIN ANALYZE on UPDATE/DELETE really executes - wrap it in a transaction and roll back.

Best Practices

- Keep predicates sargable: no functions or arithmetic on the indexed column.
- Run ANALYZE after bulk loads; check autovacuum is keeping up.
- Raise work_mem per-session for known heavy sorts rather than globally.
- Store the plan in the ticket before and after, so the improvement is auditable.
- Use auto_explain to capture plans for slow queries in production.

Follow-up Questions

1. What does "sargable" mean and why does WHERE UPPER(name) = 'X' break an index?
2. When is a Seq Scan the correct plan?

3. How do Nested Loop, Hash Join and Merge Join differ, and when does the planner pick each?
4. What are extended statistics for, and which estimate error do they fix?
5. EXPLAIN cost units - what are they, and why are they not milliseconds?

Real-world Scenario

A nightly report degraded from 4 minutes to 40 over a quarter. The plan showed a Nested Loop with an estimate of 12 rows against 900,000 actual - autovacuum had been disabled on that table during a migration and never re-enabled, so statistics were three months stale. A single ANALYZE restored the Hash Join and the 4-minute runtime. No index or query change was needed.

Question 28

Difficulty: Hard
 Category: SQL -> Partitioning

Question

What is table partitioning? Compare it with sharding and show a partitioned table.

Answer

Partitioning splits one logical table into physical child tables inside the same database. The planner uses partition pruning to touch only the relevant children. Strategies:

- Range - by date. The most common; ideal for time-series and retention.
- List - by discrete value such as region or tenant.
- Hash - by a hash of a key, for even distribution when there is no natural range.

Benefits: pruning reduces scanned data, maintenance operates per partition, and dropping old data becomes DROP TABLE (instant) instead of DELETE (slow, bloating).

Sharding splits data across different database servers. It is the answer to running out of one machine, and it brings cross-shard joins, distributed transactions, and rebalancing pain.

	Partitioning	Sharding
Scope	one server	many servers
Solves	scan volume, maintenance, retention	capacity and throughput ceilings
Cross-partition query	transparent	application or proxy must fan out
Transactions	normal ACID	distributed, or avoided
Operational cost	low	high

Rule: partition first, shard only when a single node genuinely cannot cope.

The partition key must appear in the WHERE clause for pruning to happen, and it must be part of the primary key.

Example

orders partitioned by month, with retention by dropping partitions.

Code Example

```
-- Declarative range partitioning (Postgres 12+)
CREATE TABLE orders_p (
    order_id    BIGSERIAL,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL,
    status      VARCHAR(20) NOT NULL,
    PRIMARY KEY (order_id, order_date)      -- key must include the partition column
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2025_01 PARTITION OF orders_p
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE orders_2025_02 PARTITION OF orders_p
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
CREATE TABLE orders_2025_03 PARTITION OF orders_p
    FOR VALUES FROM ('2025-03-01') TO ('2025-04-01');
CREATE TABLE orders_default PARTITION OF orders_p DEFAULT;  -- catch-all

-- Indexes defined on the parent propagate to every partition
CREATE INDEX ON orders_p (customer_id);

INSERT INTO orders_p (customer_id, order_date, status)
SELECT customer_id, order_date, status FROM orders;

-- PRUNING: only the February child is scanned
EXPLAIN (ANALYZE)
SELECT COUNT(*) FROM orders_p
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01';

-- NO PRUNING: partition key absent, every partition is scanned
EXPLAIN SELECT COUNT(*) FROM orders_p WHERE customer_id = 1;

-- Retention: instant, no bloat, versus a multi-hour DELETE
DROP TABLE orders_2025_01;
-- or keep the data but detach it from the live table
-- ALTER TABLE orders_p DETACH PARTITION orders_2025_01 CONCURRENTLY;

-- Hash partitioning when there is no natural range
CREATE TABLE events_p (event_id BIGINT, tenant_id INT NOT NULL, payload JSONB)
    PARTITION BY HASH (tenant_id);
CREATE TABLE events_p0 PARTITION OF events_p FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE events_p1 PARTITION OF events_p FOR VALUES WITH (MODULUS 4, REMAINDER 1);
```

Output

```
-- WITH the partition key: one partition touched
Aggregate (actual time=0.048..0.049 rows=1 loops=1)
-> Seq Scan on orders_2025_02 orders_p (actual time=0.019..0.026 rows=2 loops=1)
    Filter: ((order_date >= '2025-02-01') AND (order_date < '2025-03-01'))
Execution Time: 0.093 ms

-- WITHOUT it: every partition scanned (append over 4 children)
Aggregate
-> Append
    -> Seq Scan on orders_2025_01 orders_p_1
    -> Seq Scan on orders_2025_02 orders_p_2
    -> Seq Scan on orders_2025_03 orders_p_3
    -> Seq Scan on orders_default orders_p_4

DROP TABLE
```

Why Interviewers Ask This

Partitioning is where SQL meets system design, and the partition-key discipline (prune or scan everything) is a concrete, testable insight. It also leads naturally into sharding, which is a system design conversation.

Common Mistakes

- Partitioning without including the partition key in queries, so nothing prunes and you have added complexity for nothing.
- Omitting the partition column from the primary key - Postgres rejects it.
- Creating hundreds of partitions and hitting planning-time overhead.
- Forgetting a DEFAULT partition, so an out-of-range insert fails.
- Confusing partitioning with sharding in an interview - they solve different problems.

Best Practices

- Choose the partition key from the dominant query filter, usually a date.
- Automate partition creation ahead of time (pg_partman or a scheduled job).
- Keep partition counts in the tens to low hundreds.
- Use DETACH CONCURRENTLY then DROP for retention, avoiding long locks.
- Global uniqueness across partitions is not enforced - design around it.

Follow-up Questions

1. Why must the partition key be part of the primary key?
2. What is partition-wise join and when does the planner use it?
3. How do you pick a shard key, and what makes a bad one?
4. Compare range, list and hash partitioning for a multi-tenant SaaS.
5. How would you migrate a 2 TB live table to a partitioned one with no downtime?

Real-world Scenario

An events table reached 1.4 billion rows. The nightly DELETE of 90-day-old data ran for six hours, bloated the table, and starved autovacuum. Converting to monthly range partitions turned retention into a one-second DROP TABLE, and typical dashboard queries dropped from 22 seconds to 400 ms purely from pruning.

Question 29

Difficulty: Medium

Category: SQL -> Stored procedures, functions, triggers

Question

Compare stored procedures, functions and triggers. When is a trigger the wrong tool?

Answer

	Function	Procedure	Trigger
Returns a value	yes, required	optional (OUT/INOUT)	no (returns NULL or NEW)
Callable in SELECT	yes	no - use CALL	never called directly
Can manage transactions	no (runs inside caller~	yes - COMMIT/ROLLBACK ~	no
Invocation	explicit	explicit	implicit, by DML
Typical use	reusable calculation	multi-step batch job	audit, derived column, ~

PostgreSQL gained true PROCEDURE support with transaction control in version 11; before that everything was a function.

Triggers are powerful and easy to misuse. They are the right tool for audit trails and enforcing invariants that must hold regardless of which code path writes. They are the wrong tool when:

- Business logic hides in them - debugging becomes archaeology, since nothing in the application code shows the effect.
- They cascade: trigger A writes a table with trigger B, which writes back. Recursion and surprise performance cliffs.
- They do heavy work per row on bulk loads - a COPY of a million rows fires a million times.
- They perform external side effects such as notifications; a rollback cannot un-send an email.

FOR EACH STATEMENT triggers avoid the per-row cost when you only need one action per DML.

Example

An audit trigger recording every status change, and a function computing an order total.

Code Example

```
-- ===== FUNCTION: reusable, callable in SELECT =====
CREATE OR REPLACE FUNCTION order_total(p_order_id INT)
RETURNS NUMERIC
LANGUAGE sql
STABLE -- no writes; planner can optimise
AS $$
    SELECT COALESCE(SUM(quantity * unit_price), 0)
    FROM   order_items WHERE order_id = p_order_id;
$$;

SELECT order_id, order_total(order_id) AS total FROM orders ORDER BY order_id LIMIT 3;

-- ===== PROCEDURE: multi-step, owns its transactions =====
CREATE OR REPLACE PROCEDURE cancel_stale_orders(p_days INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    UPDATE orders
    SET     status = 'CANCELLED'
    WHERE  status = 'PENDING'
    AND    order_date < CURRENT_DATE - p_days;
    GET DIAGNOSTICS v_count = ROW_COUNT;

    RAISE NOTICE 'cancelled % stale orders', v_count;
    COMMIT; -- legal in a procedure, not in a function
```

```

END; $$;

CALL cancel_stale_orders(60);

-- ===== TRIGGER: audit trail, the legitimate use case =====
CREATE TABLE order_status_audit (
    audit_id    BIGSERIAL PRIMARY KEY,
    order_id    INT NOT NULL,
    old_status  VARCHAR(20),
    new_status  VARCHAR(20),
    changed_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    changed_by  TEXT NOT NULL DEFAULT CURRENT_USER
);

CREATE OR REPLACE FUNCTION audit_order_status() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    -- only log real changes; IS DISTINCT FROM is NULL-safe
    IF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO order_status_audit (order_id, old_status, new_status)
        VALUES (OLD.order_id, OLD.status, NEW.status);
    END IF;
    RETURN NEW;
END; $$;

CREATE TRIGGER trg_audit_order_status
AFTER UPDATE OF status ON orders          -- column-specific: fires less often
FOR EACH ROW EXECUTE FUNCTION audit_order_status();

UPDATE orders SET status = 'SHIPPED' WHERE order_id = 5;
SELECT order_id, old_status, new_status FROM order_status_audit;

```

Output

```

order_id |    total
-----+-----
      1 |  99997.00
      2 |   7497.00
      3 |  10498.00

NOTICE:  cancelled 1 stale orders
CALL

UPDATE 1
order_id | old_status | new_status
-----+-----+-----
      5 | PENDING   | SHIPPED

```

Why Interviewers Ask This

It probes where you believe business logic belongs. Both extremes are wrong answers: "everything in the database" and "never use a trigger". The mature answer distinguishes invariants and audit (database) from workflow (application).

Common Mistakes

- Trying to COMMIT inside a function.
- Using FOR EACH ROW where FOR EACH STATEMENT suffices, then wondering why bulk inserts crawl.
- Returning NULL from a BEFORE trigger by accident, silently cancelling the write.
- Sending emails or calling HTTP from a trigger, which cannot be rolled back.

- Not marking pure functions IMMUTABLE/STABLE, blocking optimisation and expression indexes.

Best Practices

- Triggers for audit trails and invariants; application code for business workflow.
- Keep trigger bodies short and side-effect-free; enqueue work instead of doing it inline.
- Use AFTER UPDATE OF column to narrow firing conditions.
- Mark function volatility correctly and always SET search_path in SECURITY DEFINER functions.
- Document every trigger in the schema README - they are invisible at the call site.

Follow-up Questions

1. BEFORE versus AFTER triggers - which can modify NEW?
2. What are IMMUTABLE, STABLE and VOLATILE and what do they enable?
3. How do you prevent infinite trigger recursion?
4. When would you use LISTEN/NOTIFY instead of a trigger side effect?
5. Security risk of SECURITY DEFINER without a fixed search_path?

Real-world Scenario

An order table had a trigger that recalculated a customer summary row on every insert. A marketing backfill inserted 3 million historical orders; the trigger fired 3 million times, each one locking the same summary row, and the import ran for nine hours before being killed. The fix was a statement-level trigger plus a nightly recompute, dropping the import to four minutes.

Question 30

Difficulty: Medium

Category: SQL -> GROUP BY advanced

Question

What are GROUPING SETS, ROLLUP and CUBE? How do you tell a subtotal row from a real NULL?

Answer

These produce multiple levels of aggregation in one pass instead of UNION ALL-ing several queries.

- GROUPING SETS ((a,b),(a,()), (b,())) - explicitly list the groupings you want.
- ROLLUP(a, b) - hierarchical subtotals: (a,b), (a), (). Use for year -> month -> day or country -> city.
- CUBE(a, b) - every combination: (a,b), (a), (b), (). 2^n groupings.

Subtotal rows carry NULL in the columns not being grouped, which is ambiguous when the data itself contains NULLs. The GROUPING(col) function disambiguates: it returns 1 when the column was aggregated away in that row, 0 when it is a real value. Combine it with CASE to

produce readable labels.

Example

Revenue by country and city with automatic city subtotals and a grand total. Sara Khan's NULL city is exactly the ambiguity this feature creates.

Code Example

```
SELECT COALESCE(c.country, '(all countries)') AS country,
       CASE WHEN GROUPING(c.city) = 1 THEN '(all cities)'
            ELSE COALESCE(c.city, '(unknown city)')
            END AS city,
       COUNT(DISTINCT o.order_id) AS orders,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue,
       GROUPING(c.country) AS g_country,
       GROUPING(c.city) AS g_city
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status <> 'CANCELLED'
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY ROLLUP (c.country, c.city)
ORDER BY g_country, country, g_city, city;

-- CUBE: also gives per-city totals across all countries
-- GROUP BY CUBE (c.country, c.city)

-- Equivalent to ROLLUP but explicit
-- GROUP BY GROUPING SETS ((c.country, c.city), (c.country), ())
```

Output

country	city	orders	revenue	g_country	g_city
India	Bengaluru	3	197493.00	0	0
India	Chennai	1	4999.00	0	0
India	Delhi	0	0.00	0	0
India	Mumbai	2	13997.00	0	0
India	(all cities)	6	216489.00	0	1
UAE	(unknown city)	0	0.00	0	0
UAE	(all cities)	0	0.00	0	1
USA	Austin	1	16497.00	0	0
USA	(all cities)	1	16497.00	0	1
(all countries)	(all cities)	7	232986.00	1	1

(10 rows)

The UAE rows show why GROUPING matters: (unknown city) is Sara's genuine NULL, while (all cities) is the subtotal. Without GROUPING both would print as NULL and be indistinguishable.

Why Interviewers Ask This

It is a reporting-heavy question that separates candidates who would UNION ALL three queries from those who know the single-pass feature. The GROUPING disambiguation is the detail that shows real usage.

Common Mistakes

- Using UNION ALL of three aggregate queries, scanning the table three times.
- Assuming a NULL in the output is always a subtotal marker.
- Expecting CUBE to be cheap - it computes 2^n groupings.
- Sorting without accounting for subtotal rows, so totals land in the middle of the report.

Best Practices

- ROLLUP for hierarchies, GROUPING SETS when you need specific combinations only, CUBE rarely and on low-cardinality columns.
- Always label subtotals with GROUPING(...) rather than exposing raw NULLs.
- Order by the GROUPING flags so totals appear last.

Follow-up Questions

1. How many grouping sets does CUBE(a, b, c) produce? (Eight.)
2. What does GROUPING_ID give you over GROUPING?
3. How would you achieve the same with window functions instead?
4. Which is cheaper for one subtotal level: ROLLUP or a window SUM OVER ()?

Real-world Scenario

A finance export ran four separate aggregate queries - by region, by country, by city, and a grand total - then stitched them in Python. Each scanned a 90-million-row fact table. A single ROLLUP query replaced all four, cut the export from 11 minutes to 90 seconds, and removed the class of bug where the four scans saw slightly different data.

Revision Notes: SQL Questions 21-30

Topic	One-line takeaway
Keys	super key (any unique set) -> candidate (minimal) -> primary (chosen one)
UNION vs UNION ALL	ALL is the default choice; UNION pays for a dedupe sort
Pagination	OFFSET is $O(m+n)$ and unstable; keyset is $O(\log n)$ and stable
Views	a view is a macro, not a cache; materialised views store rows and go stale
Deadlocks	caused by inconsistent lock order; fix by ordering, then retry on 40P01
Normalisation	1NF atomic, 2NF no partial, 3NF no transitive, BCNF every determinant a key
EXPLAIN	compare estimated vs actual rows first; that gap is usually the root cause
Partitioning	one server, prune by key; sharding is many servers, much more expensive
Triggers	audit and invariants yes, business workflow no, per-row on bulk loads never
ROLLUP	one pass for subtotals; use GROUPING() to label them

Sargable predicates - keep the column bare on the left:

```
BAD : WHERE EXTRACT(YEAR FROM order_date) = 2025
GOOD: WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'
BAD  : WHERE UPPER(email) = 'A@B.COM'
GOOD: WHERE email = 'a@b.com'          -- with a LOWER() expression index if needed
```

SECTION 2: MERN STACK

Numbering restarts per section. These are MERN Q1 onward.

MERN Q1

Difficulty: Easy

Category: MongoDB -> Fundamentals

Question

How does MongoDB's document model differ from a relational model? When would you choose each?

Answer

MongoDB stores BSON documents in collections. There is no enforced schema by default, arrays and nested objects are first-class, and related data is often embedded rather than joined.

	Relational	MongoDB
Unit	row in a table	document in a collection
Schema	enforced by DDL	flexible; optional JSON Schema validation
Relationships	foreign keys + joins	embedding, or references + \$lookup
Transactions	ACID, always multi-row	ACID, multi-document since 4.0 (replica set requi~
Scaling	vertical, then read replicas	horizontal sharding built in
Best at	complex joins, strong constraints	hierarchical documents, high write throughput, ev~

The real decision rule is access pattern, not preference. Embed when data is read together and bounded in size ("one product's reviews on the product page"). Reference when data is shared, unbounded, or updated independently ("users referenced by orders").

The 16 MB document limit is a hard design constraint: any array that grows without bound - audit logs, chat messages, follower lists - must be its own collection.

Example

An e-commerce order: line items are embedded (always read with the order, bounded), while the customer is referenced (shared across many orders, updated independently).

Code Example

```
// Embedded line items + referenced customer: the shape most order systems land on
const order = {
  _id: ObjectId("665f1a2b3c4d5e6f7a8b9c01"),
  customerId: ObjectId("665f1a2b3c4d5e6f7a8b9c99"), // reference: shared entity
  orderDate: new Date("2025-04-02"),
  status: "DELIVERED",
  items: [ // embedded: read together, bounded
    { productId: ObjectId("...01"), name: "UltraBook 14", qty: 1, unitPrice: 89999 },
    { productId: ObjectId("...03"), name: "Mechanical Keyboard", qty: 2, unitPrice: 4999 }
  ],
  total: 99997, // denormalised for list views
  shipping: { city: "Bengaluru", country: "India" } // embedded sub-document
};

// Schema validation: flexibility does not have to mean chaos
db.createCollection("orders", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["customerId", "orderDate", "status", "items"],
      properties: {
        status: { enum: ["PENDING", "SHIPPED", "DELIVERED", "CANCELLED"] },
        total: { bsonType: ["double", "int", "long"], minimum: 0 },
        items: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: ["productId", "qty", "unitPrice"],
            properties: { qty: { bsonType: "int", minimum: 1 } }
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});
```

Output

```
// One query returns the whole order - no joins needed
db.orders.findOne({ _id: ObjectId("665f1a2b3c4d5e6f7a8b9c01") })
{ _id: ..., status: 'DELIVERED', items: [ {...}, {...} ], total: 99997 }

// Validation rejects a bad write instead of silently accepting it
db.orders.insertOne({ customerId: ObjectId("...99"), orderDate: new Date(),
  status: "SHIPPING", items: [] })
MongoServerError: Document failed validation
```

Why Interviewers Ask This

They want to know whether you design for access patterns or just translate tables into collections. "Embed what you read together" is the phrase they are listening for.

Common Mistakes

- Saying MongoDB "has no schema" - it has no enforced schema by default, but every application has an implicit one, and \$jsonSchema can enforce it.
- Claiming MongoDB cannot do transactions or joins. Both exist (\$lookup, multi-document ACID).
- Embedding unbounded arrays and hitting the 16 MB ceiling in production.
- Normalising fully, then needing five \$lookup stages per request.

Best Practices

- Model from the queries backwards: list the screens, then design documents to be served in one read.
- Embed bounded, co-read data; reference shared or unbounded data.
- Add \$jsonSchema validation and a migration strategy even though it is optional.
- Use the extended reference pattern - duplicate the two or three fields you display and keep the reference for the rest.

Follow-up Questions

1. What is the 16 MB limit and how do you design around it?
2. Explain the bucket pattern for time-series data.
3. When is \$lookup acceptable and when is it a modelling smell?
4. How do you keep denormalised fields such as total consistent?

Real-world Scenario

A chat product embedded messages inside the conversation document. Popular group chats crossed 16 MB in six months, and every write rewrote the entire document, so p99 latency climbed to two seconds. Moving messages to their own collection keyed by conversationId with the bucket pattern fixed both the ceiling and the write amplification.

MERN Q2

Difficulty: Medium

Category: MongoDB -> Aggregation pipeline

Question

Explain the MongoDB aggregation pipeline. Write a pipeline computing revenue per customer.

Answer

The aggregation pipeline is an ordered array of stages; each stage transforms the document stream and passes it on - conceptually Unix pipes for documents.

Stages you must know:

Stage	Purpose	SQL analogue
-----	-----	-----
\$match	filter documents	WHERE
\$project / \$addFields	reshape, compute	SELECT expressions
\$unwind	one output document per array element	lateral join / unnest
\$group	aggregate by key	GROUP BY
\$sort, \$limit, \$skip	order and slice	ORDER BY, LIMIT
\$lookup	join another collection	LEFT JOIN
\$facet	multiple pipelines over one input	several queries in one

The single most important optimisation rule: \$match and \$sort as early as possible, so they can use indexes and reduce the document count before expensive stages. Once a pipeline has passed \$group or \$unwind, indexes no longer apply.

Stages spill to disk beyond 100 MB unless you pass allowDiskUse: true.

Example

Revenue per customer from embedded line items, top spenders first.

Code Example

```
db.orders.aggregate([
  // 1. filter FIRST so the index on { status: 1, orderDate: -1 } is used
  { $match: { status: { $ne: "CANCELLED" },
              orderDate: { $gte: new Date("2025-01-01") } } },

  // 2. compute the order total from the embedded array without unwinding
  { $addFields: {
      orderTotal: {
        $sum: { $map: { input: "$items", as: "i",
                      in: { $multiply: ["$$i.qty", "$$i.unitPrice"] } } }
      }
    } },

  // 3. aggregate per customer
  { $group: {
      _id: "$customerId",
      orders: { $sum: 1 },
      revenue: { $sum: "$orderTotal" },
      avgOrder: { $avg: "$orderTotal" },
      lastOrder: { $max: "$orderDate" },
      statuses: { $addToSet: "$status" }
    } },

  // 4. join customer details AFTER reducing to one doc per customer
  { $lookup: { from: "customers", localField: "_id",
              foreignField: "_id", as: "customer" } },
  { $unwind: { path: "$customer", preserveNullAndEmptyArrays: true } },

  // 5. final shape
  { $project: {
      _id: 0,
      customer: "$customer.fullName",
      country: "$customer.country",
      orders: 1,
      revenue: 1,
      avgOrder: { $round: ["$avgOrder", 2] },
      lastOrder: { $dateToString: { format: "%Y-%m-%d", date: "$lastOrder" } },
      segment: { $switch: { branches: [
          { case: { $gte: ["$revenue", 100000] }, then: "VIP" },
          { case: { $gte: ["$revenue", 10000] }, then: "regular" }
        ], default: "occasional" } }
    } }
])
```

```

    }},
    { $sort: { revenue: -1 } },
    { $limit: 10 }
  ], { allowDiskUse: true });

```

Output

```

[
  { customer: 'Aarav Sharma', country: 'India', orders: 3, revenue: 197493,
    avgOrder: 65831, lastOrder: '2025-04-02', segment: 'VIP' },
  { customer: 'John Carter', country: 'USA', orders: 1, revenue: 16497,
    avgOrder: 16497, lastOrder: '2025-04-18', segment: 'regular' },
  { customer: 'Diya Patel', country: 'India', orders: 2, revenue: 13997,
    avgOrder: 6998.5, lastOrder: '2025-05-09', segment: 'regular' },
  { customer: 'Meera Iyer', country: 'India', orders: 1, revenue: 4999,
    avgOrder: 4999, lastOrder: '2025-03-15', segment: 'occasional' }
]

```

Why Interviewers Ask This

Aggregation is the differentiator between "I can do CRUD" and "I can build features". Stage ordering is a direct performance question with a verifiable answer.

Common Mistakes

- Putting \$match after \$group or \$unwind, losing all index use.
- \$unwind-ing a large array when \$map plus \$sum would compute the same value in place.
- \$lookup before reducing the document count, so the join runs per raw document.
- Forgetting preserveNullAndEmptyArrays on \$unwind after \$lookup, silently dropping documents with no match - the MongoDB equivalent of turning a LEFT JOIN into an INNER JOIN.
- Ignoring the 100 MB stage limit on big pipelines.

Best Practices

- \$match first, \$sort next, \$project away unused fields early to shrink documents.
- Verify with `db.coll.explain("executionStats").aggregate([...])` and look for IXSCAN.
- Prefer \$addFields with array operators over \$unwind + \$group round trips.
- Use \$facet when a page needs results plus a count in one round trip.

Follow-up Questions

1. Why does \$match after \$group lose index usage?
2. When is \$unwind unavoidable?
3. What does \$facet solve for a paginated search page?
4. Compare \$lookup with storing an extended reference.
5. What changes when the collection is sharded?

Real-world Scenario

A merchant analytics endpoint took 14 seconds. The pipeline \$unwind-ed 2.1 million line items before filtering by date. Moving \$match to stage one and replacing \$unwind + \$group with \$map + \$sum brought it to 220 ms with no schema change.

MERN Q3

Difficulty: Medium

Category: MongoDB -> Indexes

Question

How do MongoDB indexes work? Explain the ESR rule for compound indexes.

Answer

MongoDB uses B-tree indexes much like relational engines. Every collection has an index on `_id`. Types: single-field, compound, multikey (automatic on array fields), text, geospatial, hashed, wildcard, plus partial and TTL variants.

ESR rule - order compound index keys as Equality, Sort, Range:

```
{ status: 1,          // E - equality match
  orderDate: -1,      // S - sort field
  total: 1 }          // R - range filter
```

Reason: equality narrows to a contiguous region, the sort field then already has the right order (no in-memory sort), and range scanning last still reads a contiguous span. Putting the range before the sort forces a blocking sort.

Like SQL composite indexes, MongoDB obeys the prefix rule: `{a:1, b:1, c:1}` serves queries on `a`, `a+b`, `a+b+c`, but not `b` alone.

A covered query is answered from the index alone - requires that all returned fields are in the index and `_id` is excluded from the projection.

Example

Order history screen: filter by status, sort newest first, optional minimum total.

Code Example

```
// ESR-ordered compound index
db.orders.createIndex({ status: 1, orderDate: -1, total: 1 },
  { name: "status_date_total" });

// Uses the index fully: equality + sort + range
db.orders.find({ status: "DELIVERED", total: { $gte: 10000 } })
  .sort({ orderDate: -1 }).limit(20);

// Partial index: index only the small hot subset
db.orders.createIndex(
  { orderDate: -1 },
  { partialFilterExpression: { status: { $in: ["PENDING", "SHIPPED"] } },
    name: "open_orders_date" }
);

// Unique index (the equivalent of a UNIQUE constraint)
```

```

db.customers.createIndex({ email: 1 }, { unique: true });

// TTL index: documents auto-expire - sessions, OTPs, caches
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 });

// Multikey: automatic when the field is an array
db.orders.createIndex({ "items.productId": 1 });

// Covered query: everything from the index, _id excluded
db.orders.createIndex({ customerId: 1, orderDate: -1, status: 1 });
db.orders.find({ customerId: ObjectId("...99") },
    { _id: 0, orderDate: 1, status: 1 }).sort({ orderDate: -1 });

// Diagnose
db.orders.find({ status: "DELIVERED" }).sort({ orderDate: -1 })
    .explain("executionStats");

// Find unused indexes before deleting them
db.orders.aggregate([ { $indexStats: {} } ]);

```

Output

```

// good plan
{ winningPlan: { stage: 'LIMIT',
  inputStage: { stage: 'IXSCAN', indexName: 'status_date_total',
    direction: 'forward' } },
  executionStats: { nReturned: 20, totalKeysExamined: 20,
    totalDocsExamined: 20, executionTimeMillis: 1 } }

// covered query - note totalDocsExamined is 0
{ executionStats: { nReturned: 3, totalKeysExamined: 3, totalDocsExamined: 0 } }

// bad plan: no usable index
{ winningPlan: { stage: 'SORT', // blocking in-memory sort
  inputStage: { stage: 'COLLSCAN' } },
  executionStats: { nReturned: 20, totalDocsExamined: 480000,
    executionTimeMillis: 1120 } }

```

The tell-tale ratio is totalDocsExamined versus nReturned. Close to 1:1 is healthy; 480000:20 means a collection scan.

Why Interviewers Ask This

Indexing is the top MongoDB performance topic, and ESR is a concrete, memorable rule that candidates either know or do not.

Common Mistakes

- Creating one single-field index per query field instead of one well-ordered compound index.
- Ignoring sort direction: {a:1, b:-1} does not serve sort({a:1, b:1}) efficiently.
- Assuming an index on an array field behaves like a scalar index (multikey has restrictions - you cannot have a compound index on two array fields).
- Believing indexes are free; each one taxes every write and consumes RAM in the working set.
- Not checking \$indexStats before adding yet another index.

Best Practices

- Order compound keys by ESR and verify with explain("executionStats").
- Watch totalDocsExamined / nReturned; investigate anything far above 1.

- Use partial indexes for skewed predicates and TTL indexes for ephemeral data.
- Build indexes in the background on production, ideally during low traffic.
- Keep the total index size within RAM where possible.

Follow-up Questions

1. Why does the sort field come before the range field?
2. What is a covered query and why must `_id` be excluded?
3. Restrictions on multikey compound indexes?
4. How do TTL indexes actually delete documents, and how promptly?
5. What is a wildcard index good and bad at?

Real-world Scenario

A support dashboard filtered by status and sorted by date over 40 million orders. It had separate single-field indexes on both, so MongoDB used one and did a blocking in-memory sort of 300,000 documents, occasionally hitting the 32 MB sort limit and erroring. One ESR-ordered compound index made the query index-only and removed the errors.

MERN Q4

Difficulty: Hard

Category: MongoDB -> Replica sets, transactions

Question

Explain replica sets, write concern and read preference. When do you need multi-document transactions?

Answer

A replica set is a group of mongod nodes holding the same data: one primary accepting writes, several secondaries replicating the oplog. If the primary fails, the members hold an election (Raft-like) and a secondary is promoted, typically within 10 to 12 seconds.

Write concern `w` controls how many nodes must acknowledge a write:

Setting	Meaning	Risk
w: 0	fire and forget	data loss, no error reporting
w: 1	primary only	lost on primary failure before replication
w: "majority"	majority of voting members	durable across failover - the default since 5.0
j: true	flushed to the journal on disk	survives process crash

Read preference controls which node serves reads: primary (default, strongly consistent), primaryPreferred, secondary, secondaryPreferred, nearest. Reading from secondaries scales reads but gives you eventual consistency - replication lag means you can read stale data,

including your own write.

Multi-document transactions (4.0 replica sets, 4.2 sharded clusters) give ACID across documents and collections. Single-document writes are already atomic, so a transaction is only needed when two or more documents must change together and you cannot restructure to embed them. They cost more than single writes and hold locks, so prefer schema design that avoids them.

Example

Transferring stock between a product document and an order document must be all-or-nothing.

Code Example

```
// ---- write concern and read preference on a connection ----
const client = new MongoClient(uri, {
  writeConcern: { w: "majority", j: true, wtimeoutMS: 5000 },
  readPreference: "primaryPreferred",
  readConcern: { level: "majority" }           // do not read rolled-back data
});

// ---- multi-document transaction with correct retry semantics ----
async function placeOrder(client, customerId, productId, qty, price) {
  const session = client.startSession();
  try {
    let orderId;
    // withTransaction retries on TransientTransactionError automatically
    await session.withTransaction(async () => {
      const db = client.db("shop");

      // 1. conditional decrement: the filter IS the concurrency guard
      const upd = await db.collection("products").updateOne(
        { _id: productId, stockQty: { $gte: qty } },
        { $inc: { stockQty: -qty } },
        { session }
      );
      if (upd.modifiedCount === 0) throw new Error("OUT_OF_STOCK"); // aborts the txn

      // 2. create the order in the same transaction
      const res = await db.collection("orders").insertOne({
        customerId, orderDate: new Date(), status: "PENDING",
        items: [{ productId, qty, unitPrice: price }],
        total: qty * price
      }, { session });
      orderId = res.insertedId;

      // 3. ledger entry
      await db.collection("inventory_log").insertOne({
        productId, delta: -qty, reason: "ORDER", orderId, at: new Date()
      }, { session });
    }, {
      readConcern: { level: "snapshot" },
      writeConcern: { w: "majority" },
      maxCommitTimeMS: 5000
    });
    return orderId;
  } finally {
    await session.endSession(); // always release the session
  }
}

// ---- read your own write without a transaction: causal consistency ----
const session = client.startSession({ causalConsistency: true });
await coll.insertOne({ _id: 1, v: "a" }, { session });
```

```
await coll.findOne({ _id: 1 }, { session, readPreference: "secondary" }); // sees the write
// ---- check replication lag before trusting secondary reads ----
db.adminCommand({ replSetGetStatus: 1 }).members.forEach(m =>
  print(`${m.name} ${m.stateStr} lag=${m.optimeDate}`));
```

Output

```
// success
new ObjectId("665f...c07")

// out of stock: transaction aborted, nothing written
Error: OUT_OF_STOCK
db.orders.countDocuments({ customerId }) // unchanged
db.products.findOne({ _id: productId }).stockQty // unchanged

// replica set status
mongo-0:27017 PRIMARY lag=2025-04-02T13:20:01Z
mongo-1:27017 SECONDARY lag=2025-04-02T13:20:01Z
mongo-2:27017 SECONDARY lag=2025-04-02T13:19:58Z // 3s behind
```

Why Interviewers Ask This

This is the MongoDB distributed-systems question. It tests whether you understand the durability and consistency knobs rather than accepting defaults, and whether you know that transactions are a last resort in a document database, not the primary tool.

Common Mistakes

- Saying MongoDB is not ACID. It is, per document always, and across documents since 4.0.
- Using w: 1 for financial writes, then losing acknowledged data on failover.
- Reading from secondaries and being surprised by stale data - then "fixing" it with retries instead of causal consistency or primary reads.
- Wrapping every operation in a transaction, hurting throughput for no benefit.
- Not handling TransientTransactionError; withTransaction does it for you, manual code does not.
- Leaking sessions by skipping endSession.

Best Practices

- w: "majority" plus readConcern: "majority" for anything you cannot lose.
- Prefer single-document atomicity by embedding what must change together.
- Keep transactions under a second and touching few documents; default limit is 60 seconds.
- Use causalConsistency sessions for read-your-own-write on secondary reads.
- Monitor replication lag and alert on it, since it silently changes read semantics.

Follow-up Questions

1. How does the oplog work, and what happens when a secondary falls off the end of it?
2. What is a rollback and how does readConcern: majority protect you?
3. Why is an arbiter usually a bad idea for a three-node set?
4. How do transactions behave on a sharded cluster differently?
5. What is the difference between readConcern and readPreference?

Real-world Scenario

A payments service wrote with the default w: 1 on an older driver. During a routine primary failover, roughly 40 acknowledged payment records that had not yet replicated were lost, while the payment gateway had already charged the cards. Switching to w: "majority", j: true plus a daily reconciliation against gateway settlements closed the gap.

MERN Q5

Difficulty: Easy

Category: Express -> Middleware

Question

What is middleware in Express? Explain the execution order and the role of next().

Answer

Middleware is a function (req, res, next) that sits in an ordered chain. Each one can read and mutate req/res, end the response, or call next() to pass control on. Express matches middleware top to bottom in registration order - order is behaviour, not style.

Four kinds:

1. Application-level - app.use(fn), runs for every request.
2. Router-level - router.use(fn), scoped to a mount path.
3. Route-level - app.get(path, mw1, mw2, handler).
4. Error-handling - four arguments (err, req, res, next). Express identifies it by arity, so you must declare all four parameters even if you ignore next.

Rules that matter:

- Failing to call next() and not responding leaves the request hanging until timeout.
- Calling next(err) skips all remaining normal middleware and jumps to the error handler.
- Error handlers must be registered last, after all routes.
- In Express 4, a thrown error inside an async handler is not caught automatically - you must wrap it. Express 5 forwards rejected promises to the error handler.

Example

A request pipeline: request id, logging, body parsing, authentication, the route, then the 404 and error handlers.

Code Example

```
import express from "express";
import { randomUUID } from "node:crypto";

const app = express();
```

```

// 1. correlation id - first, so everything downstream can log it
app.use((req, res, next) => {
  req.id = req.get("x-request-id") ?? randomUUID();
  res.setHeader("x-request-id", req.id);
  next();
});

// 2. structured request logging with duration measured on finish
app.use((req, res, next) => {
  const start = process.hrtime.bigint();
  res.on("finish", () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    console.log(JSON.stringify({
      id: req.id, method: req.method, path: req.originalUrl,
      status: res.statusCode, ms: +ms.toFixed(1)
    }));
  });
  next(); // must call next, or the request stalls here
});

// 3. built-in body parsing, with a size limit to blunt payload attacks
app.use(express.json({ limit: "100kb" }));

// 4. route-level middleware: auth applies only to this route
const requireAuth = (req, res, next) => {
  const token = req.get("authorization")?.replace(/^Bearer /, "");
  if (!token) return next(Object.assign(new Error("Unauthorized"), { status: 401 }));
  req.user = { id: "u_1", role: "customer" }; // verified elsewhere
  next();
};

// async wrapper: Express 4 does not catch rejected promises
const wrap = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

app.get("/orders/:id", requireAuth, wrap(async (req, res) => {
  const order = await findOrder(req.params.id);
  if (!order) {
    const e = new Error("Order not found");
    e.status = 404;
    throw e; // wrap() forwards this to the error handler
  }
  res.json(order);
}));

// 5. 404 handler: after all routes, before the error handler
app.use((req, res, next) => {
  next(Object.assign(new Error(`No route for ${req.method} ${req.originalUrl}`),
    { status: 404 }));
});

// 6. error handler LAST, and it must declare four parameters
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error({ id: req.id, err: err.stack });
  res.status(status).json({
    error: { message: status >= 500 ? "Internal server error" : err.message,
      requestId: req.id } // never leak stack traces to clients
  });
});

app.listen(3000);

```

Output

```

$ curl -s localhost:3000/orders/9 -H "Authorization: Bearer t"
{"error":{"message":"Order not found","requestId":"6f0c1e8a-..."}}

```

```
# server log
{"id":"6f0c1e8a-...", "method":"GET", "path":"/orders/9", "status":404, "ms":3.4}

$ curl -s localhost:3000/orders/9
{"error":{"message":"Unauthorized", "requestId":"a1b2c3d4-..."}}

$ curl -s localhost:3000/nope
{"error":{"message":"No route for GET /nope", "requestId":"..."}}
```

Why Interviewers Ask This

Middleware is the core Express abstraction; nearly every Express question reduces to it. The four-argument error handler and the `async-error` gap are the details that reveal real experience.

Common Mistakes

- Registering the error handler before the routes, so it never fires.
- Writing `(err, req, res)` with three parameters - Express treats it as normal middleware.
- Forgetting `next()`, producing a request that hangs with no error.
- Calling `next()` and sending a response, causing "Cannot set headers after they are sent".
- Assuming async handler rejections reach the error handler in Express 4.
- Leaking `err.stack` to the client in production.

Best Practices

- Fixed order: correlation id, logging, security headers, body parsing, routes, 404, error handler.
- Wrap async handlers or move to Express 5.
- Set body size limits; never accept unbounded JSON.
- Attach a request id and include it in both logs and error responses for support.
- Keep middleware single-purpose and testable in isolation.

Follow-up Questions

1. How does Express distinguish an error handler from normal middleware?
2. What happens if two middleware both write to the response?
3. `app.use` versus `router.use` versus route-level arrays - when each?
4. How do you make one middleware run only for certain HTTP methods?
5. What changed for async errors in Express 5?

Real-world Scenario

An API returned HTTP 200 with an empty body under load. A validation middleware called `next(err)` and also wrote a response; the error handler then tried to write again and the double-write was swallowed. Adding a rule that middleware either responds or calls `next`, never both, plus a lint rule, ended a week of intermittent debugging.

MERN Q6

Difficulty: Medium

Category: Express -> Authentication, JWT

Question

How does JWT authentication work? Compare it with sessions and describe secure token storage.

Answer

A JWT is three base64url segments: header.payload.signature. The server signs the payload; any party with the key can verify it without a database lookup. That statelessness is the entire value proposition - and the entire problem, because a valid token cannot be un-issued.

	Session (server state)	JWT (stateless)
Storage	server store (Redis) + cookie id	client holds the token
Revocation	delete the session - immediate	impossible without a blocklist
Scaling	needs shared store	none needed
Payload visibility	opaque id	readable by anyone (signed, not encrypted)
Size per request	small cookie	larger header

The standard production pattern is a pair of tokens:

- Access token - JWT, short lived (5 to 15 minutes), sent on every request.
- Refresh token - long lived, stored server-side so it can be revoked, used only to mint new access tokens, and rotated on each use so a stolen token is detectable.

Storage: keep the refresh token in an httpOnly, Secure, SameSite cookie so JavaScript cannot read it (XSS protection). localStorage is readable by any injected script. Cookies need CSRF defence: SameSite=Strict or Lax plus a CSRF token for state-changing requests.

A JWT payload is signed, not encrypted - never put secrets in it.

Example

Login issues both tokens; a middleware verifies the access token; refresh rotates.

Code Example

```
import jwt from "jsonwebtoken";
import argon2 from "argon2";
import crypto from "node:crypto";

const ACCESS_TTL = "15m";
const REFRESH_TTL_MS = 7 * 24 * 3600 * 1000;

// ----- login -----
app.post("/auth/login", wrap(async (req, res) => {
  const { email, password } = req.body;
  const user = await db.collection("users").findOne({ email });

  // constant-time-ish: always verify against something to avoid user enumeration
  const ok = user && await argon2.verify(user.passwordHash, password);
```

```

if (!ok) return res.status(401).json({ error: "Invalid credentials" });

const accessToken = jwt.sign(
  { sub: String(user._id), role: user.role },      // no PII, no secrets
  process.env.JWT_SECRET,
  { expiresIn: ACCESS_TTL, issuer: "shop-api", audience: "shop-web" }
);

// opaque refresh token: store only its hash, like a password
const refresh = crypto.randomBytes(32).toString("base64url");
await db.collection("refresh_tokens").insertOne({
  userId: user._id,
  tokenHash: crypto.createHash("sha256").update(refresh).digest("hex"),
  family: crypto.randomUUID(),                    // rotation family for reuse detection
  expiresAt: new Date(Date.now() + REFRESH_TTL_MS),
  createdAt: new Date()
});

res.cookie("rt", refresh, {
  httpOnly: true, secure: true, sameSite: "strict",
  path: "/auth/refresh", maxAge: REFRESH_TTL_MS
});
res.json({ accessToken, expiresIn: 900 });
});

// ----- verify middleware -----
export function authenticate(req, res, next) {
  const token = req.get("authorization")?.replace(/^Bearer /, "");
  if (!token) return res.status(401).json({ error: "Missing token" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET,
      { issuer: "shop-api", audience: "shop-web" });
    next();
  } catch (e) {
    // distinguish expiry (client should refresh) from tampering (client should re-login)
    const code = e.name === "TokenExpiredError" ? "TOKEN_EXPIRED" : "TOKEN_INVALID";
    res.status(401).json({ error: code });
  }
}

// ----- refresh with rotation and reuse detection -----
app.post("/auth/refresh", wrap(async (req, res) => {
  const presented = req.cookies.rt;
  if (!presented) return res.status(401).json({ error: "No refresh token" });

  const hash = crypto.createHash("sha256").update(presented).digest("hex");
  const stored = await db.collection("refresh_tokens").findOneAndDelete({
    tokenHash: hash, expiresAt: { $gt: new Date() }
  });

  if (!stored.value) {
    // token not found: either expired or ALREADY USED -> possible theft
    return res.status(401).json({ error: "Invalid refresh token" });
  }

  // rotate: issue a new refresh token in the same family
  const next = crypto.randomBytes(32).toString("base64url");
  await db.collection("refresh_tokens").insertOne({
    userId: stored.value.userId, family: stored.value.family,
    tokenHash: crypto.createHash("sha256").update(next).digest("hex"),
    expiresAt: new Date(Date.now() + REFRESH_TTL_MS), createdAt: new Date()
  });
  res.cookie("rt", next, { httpOnly: true, secure: true,
    sameSite: "strict", path: "/auth/refresh" });

  const accessToken = jwt.sign({ sub: String(stored.value.userId) },
    process.env.JWT_SECRET, { expiresIn: ACCESS_TTL });
  res.json({ accessToken, expiresIn: 900 });
}));

```

```
// ----- logout: revoke the whole family -----
app.post("/auth/logout", authenticate, wrap(async (req, res) => {
  await db.collection("refresh_tokens").deleteMany({ userId: new ObjectId(req.user.sub) });
  res.clearCookie("rt", { path: "/auth/refresh" }).status(204).end();
}));
```

Output

```
$ curl -sX POST localhost:3000/auth/login -H 'content-type: application/json' \
  -d '{"email":"aarav@example.com","password":"correct-horse"}' -i
HTTP/1.1 200 OK
Set-Cookie: rt=8Kd2...; Path=/auth/refresh; HttpOnly; Secure; SameSite=Strict
{"accessToken":"eyJhbGciOiJIUzI1NiIs...","expiresIn":900}

# expired access token
{"error":"TOKEN_EXPIRED"}

# replaying an already-used refresh token
{"error":"Invalid refresh token"}
```

Why Interviewers Ask This

Auth is where security mistakes are most expensive. They are checking whether you know JWTs cannot be revoked, why localStorage is risky, and whether you have implemented refresh rotation - the difference between tutorial auth and production auth.

Common Mistakes

- Long-lived access tokens (days) with no refresh mechanism - a stolen token stays valid.
- Storing tokens in localStorage, so any XSS is a full account takeover.
- Putting sensitive data in the payload, believing it is encrypted.
- Accepting the alg from the token header, enabling alg: none or algorithm-confusion attacks.
- Storing refresh tokens in plaintext in the database.
- Hashing passwords with SHA-256 or MD5 instead of Argon2id or bcrypt.
- No issuer/audience validation, so a token from another service is accepted.

Best Practices

- Short access token, rotating refresh token stored hashed, revocable server-side.
- Refresh token in an httpOnly, Secure, SameSite cookie scoped to the refresh path only.
- Pin the algorithm explicitly on verify; validate iss and aud.
- Detect refresh reuse and revoke the entire token family - that is a theft signal.
- Argon2id for passwords; rate-limit login; never reveal whether an email exists.

Follow-up Questions

1. How do you revoke a JWT before it expires?
2. Why is httpOnly insufficient without CSRF protection?
3. What is refresh token rotation and what attack does it detect?
4. Symmetric HS256 versus asymmetric RS256 - when does each fit?
5. How would you handle logout across five devices?

Real-world Scenario

A startup issued 30-day JWTs stored in localStorage. A third-party analytics script was compromised and exfiltrated tokens from thousands of browsers. Because the tokens were stateless and long-lived, the only remedy was rotating the signing secret, which logged out every user and broke mobile clients that had no refresh flow. The rebuild used 15-minute access tokens and rotating refresh cookies.

MERN Q7

Difficulty: Medium

Category: React -> Hooks, useState

Question

Why is React state immutable, and what happens if you mutate it directly?

Answer

React decides whether to re-render by comparing the reference of the previous and next state (Object.is). Mutating an object or array in place keeps the same reference, so React sees no change and skips the render - the UI silently goes stale. Immutability also makes state transitions traceable and lets React.memo, useMemo and useCallback compare cheaply.

State updates are also asynchronous and batched. Calling setCount(count + 1) three times in one handler stales on the same count; the updater form setCount(c => c + 1) reads the latest value each time.

Example

A cart where adding an item with push fails to render but the spread form works.

Code Example

```
function Cart() {
  const [items, setItems] = useState([{ id: 1, name: "USB-C Hub", qty: 1 }]);
  const [count, setCount] = useState(0);

  // WRONG: mutates the existing array, same reference, no re-render
  const addBroken = () => {
    items.push({ id: Date.now(), name: "Keyboard", qty: 1 });
    setItems(items);
  };

  // RIGHT: new array reference
  const addFixed = () =>
    setItems(prev => [...prev, { id: Date.now(), name: "Keyboard", qty: 1 }]);

  // RIGHT: nested update replaces only the changed item
  const increment = id =>
    setItems(prev => prev.map(i => i.id === id ? { ...i, qty: i.qty + 1 } : i));
```

```
// WRONG: all three read the same stale `count`, result is 1
const bumpBroken = () => { setCount(count + 1); setCount(count + 1); setCount(count + 1); };
// RIGHT: updater form queues correctly, result is 3
const bumpFixed = () => { setCount(c => c + 1); setCount(c => c + 1); setCount(c => c + 1); };

return (
  <>
    <button onClick={addFixed}>Add</button>
    <button onClick={bumpFixed}>+3</button>
    <span>{count}</span>
    <ul>{items.map(i => (
      <li key={i.id}>{i.name} x{i.qty}
        <button onClick={() => increment(i.id)}>+</button></li>))}</ul>
    </>
  );
}
```

Output

```
addBroken() -> array now has 2 items in memory, UI still shows 1 (silent bug)
addFixed() -> UI shows 2 items
bumpBroken() -> count = 1 (three calls, one increment)
bumpFixed() -> count = 3
```

Why Interviewers Ask This

It is the most common source of "my component does not update" bugs, and the stale-closure variant is the follow-up that separates memorisation from understanding.

Common Mistakes

- push, splice, sort, reverse or direct property assignment on state.
- Passing the current value instead of an updater when the next value depends on the previous.
- Deep-cloning everything with `JSON.parse(JSON.stringify(...))` - slow and drops Date, undefined and Map.
- Using array index as key, which breaks identity when the list reorders.

Best Practices

- Spread or array methods that return new values: map, filter, concat, toSorted.
- Always use the updater form when deriving from previous state.
- Reach for `useReducer` once updates involve several interdependent fields.
- Immer (or Redux Toolkit, which bundles it) for deeply nested state.

Follow-up Questions

1. What is a stale closure and how does the updater form avoid it?
2. How does React 18 automatic batching change update counts?
3. Why does `React.memo` fail when you pass a new object literal as a prop?
4. When is `useReducer` a better fit than several `useState` calls?

Real-world Scenario

A form wizard mutated a nested formData object at each step. Steps 1 and 2 rendered because other state changed alongside, masking the bug; step 3 changed nothing else and the summary screen showed empty fields. Adopting Immer via Redux Toolkit removed the whole class of error.

MERN Q8

Difficulty: Medium

Category: React -> useEffect

Question

Explain useEffect, its dependency array, and cleanup. How do you fetch data without a race condition?

Answer

useEffect(fn, deps) runs a side effect after render and commit. The dependency array controls re-execution:

deps	Runs
omitted	after every render
[]	once after mount (twice in dev Strict Mode)
[a, b]	on mount and whenever a or b change by Object.is

The returned function is cleanup: it runs before the next effect execution and on unmount. That is where you abort requests, clear timers and close sockets.

Data fetching has two hazards. First, race conditions: two in-flight requests can resolve out of order, so a stale response overwrites a fresh one. Second, leaks: setting state after unmount. Both are solved with AbortController plus an ignore flag in cleanup.

React 18 Strict Mode intentionally mounts, unmounts and remounts effects in development to expose missing cleanup. That is a feature, not a bug.

Example

A search box that fetches on every keystroke, where responses arrive out of order.

Code Example

```
function OrderSearch({ query }) {  
  const [orders, setOrders] = useState([]);  
  const [state, setState] = useState("idle");  
  
  useEffect(() => {  
    if (!query) { setOrders([]); return; }  
  }, [query]);  
}
```

```

const controller = new AbortController();
let ignore = false; // guards against out-of-order resolution
setState("loading");

(async () => {
  try {
    const res = await fetch(`/api/orders?q=${encodeURIComponent(query)}`,
                          { signal: controller.signal });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    if (!ignore) { setOrders(data); setState("done"); } // only the latest wins
  } catch (e) {
    if (e.name !== "AbortError" && !ignore) setState("error");
  }
})();

// cleanup: cancel the request and disown its result
return () => { ignore = true; controller.abort(); };
}, [query]); // re-run only when the query changes

if (state === "loading") return <Spinner />;
if (state === "error") return <p role="alert">Could not load orders.</p>;
return <ul>{orders.map(o => <li key={o._id}>{o.status}</li>)}</ul>;
}

// Subscription cleanup - the other classic use
useEffect(() => {
  const socket = new WebSocket("wss://api.example.com/orders");
  socket.addEventListener("message", onMessage);
  return () => socket.close(); // without this, every remount leaks a socket
}, []);

```

Output

```

type "ult" quickly:
  request "u"   -> resolves 3rd (ignored, aborted)
  request "ul"  -> resolves 1st (ignored, aborted)
  request "ult" -> resolves 2nd (rendered)
UI shows results for "ult" // without the guard it would show results for "u"

```

Why Interviewers Ask This

Effects are the hardest hook to use correctly, and the race-condition fix is a concrete signal of production experience. Interviewers also want to hear that in 2026 you would normally reach for TanStack Query or SWR rather than hand-rolling this.

Common Mistakes

- Omitting dependencies to "stop the loop", creating stale closures instead.
- No cleanup, leaking sockets, timers and listeners on every remount.
- Assuming responses arrive in request order.
- Putting an object or array literal in deps - a new reference each render means it always re-runs.
- Using an effect for derived state that could be computed during render.
- Fighting Strict Mode's double invocation instead of fixing the missing cleanup it revealed.

Best Practices

- Let the linter enforce the full dependency array; fix the design rather than silencing it.
- Always return cleanup for anything subscribed, timed or in flight.
- Use a data library (TanStack Query, SWR, or the framework loader) for fetching - it handles caching, dedupe, retries and races for you.
- Do not use effects to sync state that can be derived during render.

Follow-up Questions

1. Why does an object in the dependency array cause an infinite loop?
2. useEffect versus useLayoutEffect - which blocks paint?
3. Why does Strict Mode run effects twice in development?
4. How would useSyncExternalStore handle the socket case differently?
5. What does the React team mean by "you might not need an effect"?

Real-world Scenario

A dashboard opened a WebSocket in an effect with no cleanup. Users navigating between tabs accumulated connections; after an hour a browser held 90 sockets and the server hit its connection limit, taking live updates down for everyone. A one-line `return () => socket.close()` resolved it.

MERN Q9

Difficulty: Medium

Category: React -> useMemo, useCallback, memo

Question

When should you use useMemo, useCallback and React.memo? When are they harmful?

Answer

All three are referential-stability tools, not general speed-ups.

- `useMemo(fn, deps)` caches a computed value between renders.
- `useCallback(fn, deps)` caches a function identity between renders.
- `React.memo(Component)` skips re-rendering when props are shallow-equal.

They are worth it when (a) a computation is genuinely expensive, or (b) a value or function is passed to a memo-ised child or used in another hook's dependency array. Without one of those, you pay the cost of the comparison plus retained memory for nothing.

Critical interaction: `React.memo` is defeated by any new reference in props. Passing `onClick={() => ...}` or `style={{...}}` creates a new value each render, so the child always re-renders and memo is pure overhead. That is why `useCallback` and `memo` are usually adopted together or not at all.

The React Compiler (stable in React 19) memoises automatically, which is making most manual memoisation unnecessary in new code - a good thing to mention.

Example

A large order table with an expensive derived total and a memoised row component.

Code Example

```
const OrderRow = React.memo(function OrderRow({ order, onSelect }) {
  console.log("render row", order.id);
  return <tr onClick={() => onSelect(order.id)}><td>{order.total}</td></tr>;
});

function OrderTable({ orders, filter }) {
  const [selected, setSelected] = useState(null);

  // WORTH IT: O(n) filter + sort over thousands of rows, recomputed only when inputs change
  const visible = useMemo(
    () => orders.filter(o => o.status === filter)
      .sort((a, b) => b.total - a.total),
    [orders, filter]
  );

  // WORTH IT: stable identity keeps React.memo on OrderRow effective
  const handleSelect = useCallback(id => setSelected(id), []);

  // NOT WORTH IT: trivial arithmetic, the memo costs more than the work
  // const doubled = useMemo(() => count * 2, [count]);

  return <table><tbody>
    {visible.map(o => <OrderRow key={o.id} order={o} onSelect={handleSelect} />)}
  </tbody></table>;
}

// Anti-pattern that silently disables memo:
// <OrderRow order={o} onSelect={id => setSelected(id)} /> // new fn every render
// <OrderRow order={{ ...o }} ... /> // new object every render
```

Output

```
// with useCallback + memo: typing in an unrelated input re-renders 0 rows
render row 1 ... render row 500      (first mount only)

// without useCallback: every parent render re-renders all 500 rows
render row 1 ... render row 500      (on every keystroke)
```

Why Interviewers Ask This

It separates people who memoise reflexively from those who reason about renders. Saying "wrap everything in useMemo" is a negative signal.

Common Mistakes

- Memoising trivial values, adding complexity and memory for no gain.
- Using React.memo while passing fresh object or function literals, so it never helps.
- Forgetting useMemo's dependencies and serving a stale value.
- Believing useMemo prevents re-renders - it caches a value; only memo skips renders.

- Optimising before profiling with the React DevTools Profiler.

Best Practices

- Profile first; memoise the measured hot path only.
- Adopt memo plus useCallback/useMemo together, or neither.
- Prefer structural fixes: lift state down, split components, virtualise long lists.
- On React 19, evaluate the React Compiler before hand-memoising.

Follow-up Questions

1. Why does React.memo not help when a child receives an inline arrow function?
2. Difference between useMemo and useRef for caching?
3. What does the React Compiler change about this advice?
4. When would a custom comparator as memo's second argument be justified?
5. How do you virtualise a 50,000-row table instead of memoising it?

Real-world Scenario

A team wrapped every component in React.memo and every handler in useCallback for "performance". Profiling showed memo comparisons costing more than the renders they skipped, and one stale useMemo dependency caused a pricing bug that shipped to production. Removing 80 percent of the memoisation made the app measurably faster and the code simpler.

MERN Q10

Difficulty: Hard

Category: React -> Virtual DOM, reconciliation

Question

Explain the Virtual DOM and reconciliation. Why do keys matter?

Answer

The Virtual DOM is a lightweight JavaScript tree describing the intended UI. On state change, React builds a new tree and diffs it against the previous one (reconciliation), then applies the minimal set of real DOM mutations. DOM writes are the expensive part; diffing plain objects is cheap by comparison.

React's diff uses two heuristics to stay $O(n)$ instead of $O(n^3)$:

1. Different element type means discard the subtree. `<div>` to `<span>` unmounts everything inside, losing child state.
2. Keys identify children across renders. Within a list, React matches elements by key, not by position.

That is why index keys are dangerous: if you prepend or reorder, index 0 now refers to a different item, so React reuses the wrong DOM node and its state - the classic symptom is a

checkbox or input value attached to the wrong row.

React 18+ adds concurrent rendering: rendering can be interrupted, paused and resumed, which is what makes `useTransition` and `Suspense` useful. Reconciliation itself is unchanged.

Example

A list where deleting the first item moves a checked checkbox to the wrong row when keys are indexes.

Code Example

```
// BROKEN: index keys plus per-row state
function BadList({ items, onDelete }) {
  return <ul>{items.map((item, i) => (
    <li key={i}>                                     /* identity = position, not item */
      <input type="checkbox" />                          /* uncontrolled DOM state */
      {item.name}
      <button onClick={() => onDelete(item.id)}>x</button>
    </li>))}</ul>;
}

// CORRECT: stable identity from the data
function GoodList({ items, onDelete }) {
  return <ul>{items.map(item => (
    <li key={item.id}>                                /* identity survives reorder/delete */
      <input type="checkbox" />
      {item.name}
      <button onClick={() => onDelete(item.id)}>x</button>
    </li>))}</ul>;
}

// Forcing a remount deliberately: change the key to reset all internal state
<OrderForm key={selectedOrderId} order={order} />

// Concurrent rendering: keep typing responsive while a heavy list re-renders
function Search() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  return <>
    <input value={query} onChange={e => {
      setQuery(e.target.value);                      // urgent: keeps the input snappy
      startTransition(() => setResults(filterHuge(e.target.value))); // interruptible
    }} />
    {isPending && <Spinner />}
    <BigList rows={results} />
  </>;
}
```

Output

items: [A, B, C], user checks the box on B, then deletes A

with `key={i}`: remaining rows are [B, C] but the checkbox appears on C (wrong)

with `key={item.id}`: remaining rows are [B, C] and the checkbox stays on B (correct)

Why Interviewers Ask This

Keys are asked in nearly every React interview, and the "why" - identity for reconciliation, not uniqueness for its own sake - is what they are actually testing. It also opens into concurrent rendering for senior candidates.

Common Mistakes

- "Keys are for performance." They are primarily for correctness.
- Using array index, `Math.random()`, or `uuid()` generated during render as a key. Random keys force a full remount every render.
- Believing the Virtual DOM is inherently faster than direct DOM manipulation - hand-written optimal DOM code is faster; the VDOM buys a declarative model with good-enough performance.
- Confusing concurrent rendering with multithreading. JavaScript remains single-threaded.

Best Practices

- Key by a stable business id from the data.
- Change a key intentionally when you want to reset a subtree's state.
- Keep element types stable across renders to avoid subtree remounts.
- Virtualise long lists; reconciliation still costs something per element.

Follow-up Questions

1. What exactly happens to child state when an element's type changes?
2. Why is `key={Math.random()}` catastrophic?
3. How does `useTransition` change what the user perceives?
4. What problem does `Suspense` with `React.lazy` solve at the bundle level?
5. How does the reconciler decide to update in place versus remount?

Real-world Scenario

A payments table used index keys with inline editing. Sorting by amount reordered rows while a user was mid-edit, and React reused the input DOM node - the edited value landed on a different transaction and was saved. Switching to `key={txn.id}` fixed it; the incident review made "no index keys" a lint rule.

MERN Q11

Difficulty: Medium

Category: React -> State management, Context, Redux Toolkit

Question

When do you need Redux Toolkit instead of Context or local state? What is the Context performance trap?

Answer

Escalate only when the current tool hurts:

1. Local useState - state used by one component or passed one level.
2. Lifted state / composition - shared by a few siblings.
3. Context - low-frequency global values: theme, locale, current user, feature flags.
4. Server-state library (TanStack Query, SWR, RTK Query) - anything that comes from an API.

This eliminates most of what teams historically put in Redux.

5. Redux Toolkit / Zustand / Jotai - genuinely global, frequently updated client state with complex transitions, or when you need time-travel debugging and middleware.

The Context trap: every consumer of a context re-renders when the context value changes, regardless of which part of the value it uses. A single context holding user, theme, cart and notifications re-renders the entire tree on any cart change. Mitigations: split into several narrow contexts, memoise the provider value, or use a store with selector-based subscriptions.

The modern default for a MERN app: TanStack Query or RTK Query for server data, Context for a handful of static globals, and a small store only if genuinely needed.

Example

Splitting one fat context, and a Redux Toolkit slice with a typed async thunk.

Code Example

```
// ----- Context trap and the fix -----
// BAD: one context, four unrelated concerns -> any change re-renders every consumer
const AppContext = createContext(null);
<AppContext.Provider value={{ user, theme, cart, notifications }}>

// GOOD: split by change frequency, and memoise each value
const UserContext = createContext(null); // changes rarely
const ThemeContext = createContext(null); // changes rarely
const CartContext = createContext(null); // changes often -> isolated

function Providers({ children }) {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState("light");
  // memo so the object identity is stable across unrelated renders
  const userValue = useMemo(() => ({ user, setUser }), [user]);
  const themeValue = useMemo(() => ({ theme, setTheme }), [theme]);
  return (
    <UserContext.Provider value={userValue}>
      <ThemeContext.Provider value={themeValue}>{children}</ThemeContext.Provider>
    </UserContext.Provider>
  );
}

// ----- Redux Toolkit slice -----
import { createSlice, createAsyncThunk, configureStore } from "@reduxjs/toolkit";

export const fetchOrders = createAsyncThunk(
  "orders/fetch",
  async (customerId, { rejectWithValue, signal }) => {
    const res = await fetch(`/api/customers/${customerId}/orders`, { signal });
    if (!res.ok) return rejectWithValue(await res.text());
    return res.json();
  }
);
```



```

const ordersSlice = createSlice({
  name: "orders",
  initialState: { items: [], status: "idle", error: null },
  reducers: {
    // Immer under the hood: "mutating" here produces an immutable update
    orderCancelled(state, action) {
      const o = state.items.find(i => i._id === action.payload);
      if (o) o.status = "CANCELLED";
    }
  },
  extraReducers: builder => {
    builder
      .addCase(fetchOrders.pending, s => { s.status = "loading"; s.error = null; })
      .addCase(fetchOrders.fulfilled, (s, a) => { s.status = "done"; s.items = a.payload; })
      .addCase(fetchOrders.rejected, (s, a) => { s.status = "error"; s.error = a.payload; });
  }
});

export const { orderCancelled } = ordersSlice.actions;
export const store = configureStore({ reducer: { orders: ordersSlice.reducer } });

// selector keeps the component subscribed to one slice of state only
const deliveredCount = useSelector(s =>
  s.orders.items.filter(o => o.status === "DELIVERED").length);

```

Output

```

// single fat context: changing cart quantity
re-rendered: Header, Sidebar, ProductList, Footer, CartBadge    (all consumers)

// split contexts: same change
re-rendered: CartBadge                                          (only what depends on cart)

```

Why Interviewers Ask This

Over-engineering state is the most common architectural mistake in React codebases. They want to hear escalation with justification, and awareness that server state is a different problem from client state.

Common Mistakes

- Reaching for Redux on day one for a form and a list.
- Putting API response data in Redux by hand, then reimplementing caching, dedupe and invalidation badly.
- One giant context, then blaming React for slow renders.
- Writing legacy hand-rolled Redux (switch reducers, ADD_TODO constants, spread updates) instead of Redux Toolkit.
- Storing derived data in state instead of computing it in a selector.

Best Practices

- Server data belongs in a query library; client-only UI state belongs in local state or a store.
- Split contexts by update frequency and memoise provider values.
- Redux Toolkit, never hand-rolled Redux, if you choose Redux.
- Keep the store normalised and derive with memoised selectors (createSelector).

Follow-up Questions

1. Why does memoising the provider value matter?
2. How does RTK Query differ from TanStack Query?
3. What does createSelector memoisation prevent?
4. When is Zustand or Jotai a better fit than Redux Toolkit?
5. How would you persist part of the store to localStorage safely?

Real-world Scenario

An app kept the entire API cache in one context. Every poll of the notifications endpoint changed the context value and re-rendered 200 components, pinning the CPU on low-end phones. Moving server data to TanStack Query and splitting the remaining context by concern cut interaction latency by roughly 70 percent and deleted about 1,200 lines of reducer code.

MERN Q12

Difficulty: Hard

Category: Node.js -> Event loop

Question

Explain the Node.js event loop. What is the order of setTimeout, setImmediate, process.nextTick and promise callbacks?

Answer

Node is single-threaded for JavaScript but uses libuv's event loop plus a thread pool for file I/O, DNS and crypto. The loop runs phases in order:

```
+-----+
-->|  timers          setTimeout/setInterval callbacks
+-----+
|  pending callbacks  some system errors
+-----+
|  idle / prepare      internal
+-----+
|  poll                retrieve I/O events, execute I/O callbacks  <-- blocks here
+-----+
|  check              setImmediate callbacks
+-----+
|  close              'close' event handlers
+-----+
```

Between every phase, and between each callback, Node drains two microtask queues in this order:

1. process.nextTick queue - highest priority, drained completely first.
2. Promise microtask queue - then, catch, finally, and continuations after await.

So the ordering is: ****synchronous code**, then all `nextTick`, then all **promise microtasks**, then **timers**, then **I/O**, then **setImmediate****. `setImmediate` runs in the check phase - after I/O - despite the name, whereas `setTimeout(fn, 0)` runs in the timers phase of the next iteration.

Consequence: any CPU-bound synchronous work blocks the whole loop, and therefore every concurrent request. That is what `worker_threads` exists to solve.

Example

Ordering puzzle - the canonical Node interview question.

Code Example

```
const fs = require("node:fs");

console.log("1: sync start");

setTimeout(() => console.log("6: setTimeout 0"), 0);
setImmediate(() => console.log("7: setImmediate"));

Promise.resolve().then(() => console.log("4: promise then"));
process.nextTick(() => console.log("3: nextTick"));

queueMicrotask(() => console.log("5: queueMicrotask"));

fs.readFile(__filename, () => {
  console.log("8: fs read callback (poll phase)");
  // inside an I/O callback, setImmediate beats setTimeout
  setTimeout(() => console.log("10: timeout inside I/O"), 0);
  setImmediate(() => console.log("9: immediate inside I/O"));
});

console.log("2: sync end");

// ---- blocking the loop: what NOT to do ----
function blockingHash(password) {          // ~1s of CPU, blocks every request
  return require("node:crypto").pbkdf2Sync(password, "salt", 1_000_000, 64, "sha512");
}

// ---- correct: offload CPU work to the pool or a worker thread ----
const { pbkdf2 } = require("node:crypto");
const hashAsync = (pw) => new Promise((resolve, reject) => {
  pbkdf2(pw, "salt", 1_000_000, 64, "sha512", (e, k) => e ? reject(e) : resolve(k));
});

const { Worker } = require("node:worker_threads");
const runInWorker = (data) => new Promise((resolve, reject) => {
  const w = new Worker("./cpu-task.js", { workerData: data });
  w.on("message", resolve); w.on("error", reject);
});
```

Output

```
1: sync start
2: sync end
3: nextTick
4: promise then
5: queueMicrotask
6: setTimeout 0
7: setImmediate
8: fs read callback (poll phase)
9: immediate inside I/O
```

10: timeout inside I/O

Note lines 9 and 10: inside an I/O callback the loop is in the poll phase, so the next phase is check - setImmediate fires before the timer.

Why Interviewers Ask This

It is the definitive test of whether you understand Node's concurrency model rather than just its

API surface. The practical payoff - never block the loop - is the point of the question.

Common Mistakes

- Believing Node is multithreaded for JavaScript, or that it is single-threaded for everything (file I/O and crypto use the thread pool).
- Saying setImmediate runs before setTimeout(fn, 0) always - it depends on the current phase.
- Forgetting process.nextTick outranks promises, and that a recursive nextTick can starve the loop entirely.
- Using Sync APIs (readFileSync, pbkdf2Sync) in a request path.
- Assuming worker_threads helps with I/O - it is for CPU work.

Best Practices

- Never run CPU-bound work on the main thread; use worker_threads or a separate service.
- Prefer async APIs; reserve Sync variants for startup code.
- Watch event loop lag as a first-class metric (perf_hooks.monitorEventLoopDelay).
- Use setImmediate to yield between chunks when processing large arrays.
- Size UV_THREADPOOL_SIZE when heavy on file or crypto operations.

Follow-up Questions

1. Why does setImmediate beat setTimeout inside an I/O callback?
2. How can process.nextTick starve the event loop?
3. Which operations use the libuv thread pool?
4. When do you use cluster versus worker_threads?
5. How would you detect and alert on event loop lag in production?

Real-world Scenario

An image service resized uploads synchronously with a CPU-bound library. Under 20 concurrent uploads the event loop stalled for 400 ms at a time, health checks timed out, and the orchestrator killed and restarted healthy pods in a loop. Moving resizing into a worker thread pool made p99 latency stable and stopped the restart cycle.

MERN Q13

Difficulty: Medium

Question

What are Node streams and why use them instead of reading a whole file?

Answer

A stream processes data in chunks rather than loading everything into memory. Reading a 2 GB CSV with `readFile` allocates 2 GB and risks the heap limit; a stream holds only the current chunk plus a bounded buffer.

Four types: Readable (source), Writable (sink), Duplex (both, e.g. a socket), and Transform (Duplex that modifies data, e.g. `gzip`).

Backpressure is the key concept: if the consumer is slower than the producer, the buffer grows until memory is exhausted. `write()` returning `false` signals "stop"; pipe/pipeline handle this automatically. Hand-rolling data handlers plus `write()` without honouring the return value is how production memory leaks are created.

Always use `stream.pipeline` (or `pipeline` from `node:stream/promises`) rather than `.pipe()`: pipe does not forward errors or destroy the remaining streams, so a failure mid-chain leaks file descriptors.

Example

Streaming a large export from MongoDB to a gzipped HTTP response with constant memory.

Code Example

```
import { pipeline } from "node:stream/promises";
import { Transform } from "node:stream";
import { createReadStream, createWriteStream } from "node:fs";
import { createGzip } from "node:zlib";

// ---- memory comparison ----
// BAD: entire file in RAM, OOM on large input
const all = await fs.promises.readFile("orders.csv", "utf8");
const rows = all.split("\n").map(parseRow);

// GOOD: constant memory regardless of file size
const toJsonLines = new Transform({
  readableObjectMode: false,
  transform(chunk, _enc, cb) {
    // chunk-boundary safety: keep the partial trailing line for the next chunk
    this._tail = (this._tail ?? "") + chunk.toString("utf8");
    const lines = this._tail.split("\n");
    this._tail = lines.pop();
    cb(null, lines.filter(Boolean)
      .map(l => JSON.stringify(parseRow(l)) + "\n").join(""));
  },
  flush(cb) { cb(null, this._tail ? JSON.stringify(parseRow(this._tail)) + "\n" : ""); }
});

await pipeline(
  createReadStream("orders.csv"),
  toJsonLines,
  createGzip(),
  createWriteStream("orders.jsonl.gz")
);
```

```

); // errors anywhere reject here AND destroy every stream in the chain
// ---- streaming a Mongo cursor to the HTTP response ----
app.get("/api/orders/export", async (req, res) => {
  res.setHeader("Content-Type", "application/x-ndjson");
  res.setHeader("Content-Encoding", "gzip");
  const cursor = db.collection("orders").find({}).stream();
  try {
    await pipeline(
      cursor,
      new Transform({ objectMode: true,
        transform(doc, _e, cb) { cb(null, JSON.stringify(doc) + "\n"); } }),
      createGzip(),
      res
    );
  } catch (err) {
    // client aborted or a stage failed; the response may be partially written
    if (!res.headersSent) res.status(500).end();
    req.log?.error({ err }, "export failed");
  }
});

// ---- honouring backpressure by hand, when you must ----
readable.on("data", chunk => {
  if (!writable.write(chunk)) { // false = buffer full
    readable.pause(); // stop reading
    writable.once("drain", () => readable.resume());
  }
});

```

Output

```

# 2 GB input file
readFile approach : RSS peaks at 2.4 GB, then
                    FATAL ERROR: Reached heap limit - JavaScript heap out of memory
pipeline approach : RSS stays around 70 MB, completes in 41 s

# export endpoint, 1.2 million documents
memory flat at ~90 MB; first bytes reach the client in 120 ms

```

Why Interviewers Ask This

Streams separate people who have handled real data volumes from those who have not. Backpressure and pipeline-over-pipe are the two details that demonstrate it.

Common Mistakes

- readFile on user-supplied or unbounded files.
- Using .pipe() and never handling errors, leaking descriptors on failure.
- Ignoring the boolean return of write().
- Splitting on newlines per chunk without preserving the partial trailing line - a classic corruption bug shown handled above.
- Forgetting the client can abort mid-stream, leaving the cursor open.

Best Practices

- pipeline from node:stream/promises for every chain.
- Object mode for record streams; set highWaterMark deliberately for large records.

- Stream responses for exports so time-to-first-byte stays low.
- Clean up cursors and temp files in finally.
- Prefer for await (const chunk of readable) for readable-only consumption.

Follow-up Questions

1. What exactly is backpressure and how does pipeline manage it?
2. Difference between highWaterMark in object mode and byte mode?
3. Why does .pipe() not forward errors?
4. How do async iterators compare with data event handlers?
5. How would you resume a failed 10 GB upload?

Real-world Scenario

A reporting endpoint built a 400 MB JSON array in memory before responding. Three concurrent requests exhausted the container's 1 GB limit and the pod was OOM-killed, taking unrelated traffic with it. Converting to NDJSON streaming held memory at 90 MB and reduced time-to-first-byte from 38 seconds to under a second.

SECTION 8: JAVASCRIPT

Compact format from here on. All ten parts are still present; prose is trimmed to the load-bearing points so the remaining syllabus can be covered. Code, output and follow-ups keep full detail.

JS Q1

Difficulty: Easy
Category: JavaScript -> Scope, hoisting

Question

Explain hoisting and the difference between var, let and const.

Answer

Declarations are processed before code executes. var is hoisted to the top of its function scope and initialised to undefined. let and const are hoisted to the top of their block but left uninitialised - reading them before the declaration throws, and that window is the Temporal Dead Zone (TDZ). Function declarations are fully hoisted; function expressions are not.

	var	let	const
Scope	function	block	block
Redeclare	yes	no	no
Reassign	yes	yes	no
Before declaration	undefined	TDZ error	TDZ error
On globalThis	yes	no	no

const freezes the binding, not the value: const a = []; a.push(1) is legal.

Example

A loop closure - the classic var bug.

Code Example

```
console.log(a);           // undefined (var hoisted and initialised)
// console.log(b);       // ReferenceError: Cannot access 'b' before initialization
var a = 1;
let b = 2;

// var: one shared binding, all callbacks see the final value
for (var i = 0; i < 3; i++) setTimeout(() => console.log("var", i), 0);
// let: a fresh binding per iteration
for (let j = 0; j < 3; j++) setTimeout(() => console.log("let", j), 0);

const cfg = { retries: 3 };
cfg.retries = 5;           // allowed: the object is mutable
// cfg = {};             // TypeError: Assignment to constant variable
Object.freeze(cfg);       // shallow immutability if you need it
```

Output

```
undefined
var 3
var 3
var 3
let 0
let 1
let 2
```

Why Interviewers Ask This

It is the entry-level filter for understanding scope, and the loop example doubles as a closure test.

Common Mistakes

- Saying let is "not hoisted" - it is hoisted, just not initialised.
- Believing const makes objects immutable.
- Using var in modern code.
- Not knowing var at top level attaches to globalThis while let does not.

Best Practices

- const by default, let when reassignment is required, never var.
- Declare at first use, in the narrowest block.
- Enable no-var and prefer-const lint rules.

Follow-up Questions

1. What is the TDZ and why does it exist?
2. How would you fix the var loop without let? (IIFE per iteration.)
3. Are function declarations hoisted differently from arrow functions assigned to const?
4. What does typeof undeclaredVar return, and why does it not throw?

Real-world Scenario

A migration to let/const surfaced a five-year-old bug: an event handler registered in a var loop had always attached the last row's id to every button, and the team had "fixed" it by re-querying the DOM on click. The scope fix removed 40 lines of workaround.

JS Q2

Difficulty: Medium

Category: JavaScript -> Closures

Question

What is a closure? Give a practical use and a memory-leak risk.

Answer

A closure is a function bundled with the lexical environment it was created in. The inner function keeps its outer variables alive after the outer function returns, because it still references them. That is the mechanism behind data privacy, function factories, memoisation, once, debounce and throttle, and module state.

The leak risk is the flip side: as long as the closure is reachable, everything it captures is retained - including large arrays or DOM nodes captured accidentally.

Example

A private counter, a rate limiter, and a memoiser.

Code Example

```
// 1. private state - no class, nothing accessible from outside
function createCounter(start = 0) {
  let count = start; // not reachable except via the closures
  return {
    increment: () => ++count,
    reset:     () => { count = start; },
    get value() { return count; }
  };
}
const c = createCounter(10);
c.increment(); c.increment();
console.log(c.value, c.count); // 12 undefined

// 2. memoise: the cache lives in the closure
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

```

}
const slowSquare = n => { for (let i = 0; i < 1e7; i++); return n * n; };
const fast = memoize(slowSquare);
console.time("first"); fast(9); console.timeEnd("first");
console.time("cached"); fast(9); console.timeEnd("cached");

// 3. LEAK: the closure captures a 50 MB buffer it never uses
function makeHandler() {
  const bigBuffer = new Uint8Array(50 * 1024 * 1024);
  const id = bigBuffer.length; // only the length is needed
  return () => console.log("id", id); // ...but V8 may retain the whole scope
}
// FIX: capture only what you need, and null out the rest
function makeHandlerFixed() {
  let bigBuffer = new Uint8Array(50 * 1024 * 1024);
  const id = bigBuffer.length;
  bigBuffer = null; // release the reference
  return () => console.log("id", id);
}

```

Output

```

12 undefined
first: 12.402ms
cached: 0.018ms

```

Why Interviewers Ask This

Closures underpin most intermediate JavaScript questions - debounce, currying, module pattern, React hooks. The leak angle distinguishes senior candidates.

Common Mistakes

- Defining a closure as "a function inside a function" without mentioning the retained environment.
- Not connecting closures to the var loop problem.
- Creating closures inside hot loops, allocating per iteration.
- Forgetting that closures over DOM nodes prevent garbage collection after removal.

Best Practices

- Capture the minimum; extract primitives rather than whole objects.
- Use WeakMap/WeakRef for caches keyed by objects so entries can be collected.
- Bound memo caches (LRU) - an unbounded Map is a slow leak.

Follow-up Questions

1. Implement once(fn) so the function runs at most one time.
2. How do closures explain stale state in React hooks?
3. Why does a WeakMap cache avoid the leak a Map cache creates?
4. Implement currying with closures.

Real-world Scenario

A single-page app registered a resize handler per route that closed over the rendered dataset. Navigating between routes 30 times retained 30 datasets - about 600 MB - and Chrome tabs crashed on long sessions. Removing listeners on unmount and capturing only the needed fields fixed it.

JS Q3

Difficulty: Medium

Category: JavaScript -> this, call, apply, bind

Question

How is this determined? Compare call, apply and bind, and arrow functions.

Answer

For normal functions, this is decided at call time by these rules, in priority order:

1. new Foo() - this is the new instance.
2. Explicit binding - fn.call(obj), fn.apply(obj), fn.bind(obj).
3. Implicit binding - obj.fn(), this is obj.
4. Default - undefined in strict mode / modules, globalThis otherwise.

Arrow functions have no own this. They capture it lexically from the enclosing scope and it cannot be reassigned - bind, call and apply cannot change it. That is why arrows are correct for callbacks and wrong for object methods that need the receiver, and for prototype methods.

	call	apply	bind
Args	comma list	array	comma list (partial)
Invokes now	yes	yes	no - returns a new function

Example

The classic lost-this bug when a method is passed as a callback.

Code Example

```
const order = {
  id: 7,
  items: ["Air Fryer", "USB-C Hub"],
  // normal method: `this` depends on how it is called
  describe() { return `Order ${this.id}: ${this.items.length} items`; },
  // arrow as a method: `this` is module scope, NOT the object
  describeBroken: () => `Order ${this?.id}`
};

console.log(order.describe());           // implicit binding works
const detached = order.describe;
// console.log(detached());             // TypeError: cannot read 'id' of undefined
```

```

console.log(detached.call(order));           // explicit
console.log(detached.apply(order));          // same, array args
const bound = detached.bind(order);
console.log(bound());                       // permanently bound
console.log(order.describeBroken());         // Order undefined

// partial application with bind
const rate = (pct, amount) => amount * (1 - pct / 100);
const withGst = rate.bind(null, 18);
console.log(withGst(1000));

// callbacks: arrow keeps the enclosing `this`
class OrderPoller {
  constructor(id) { this.id = id; this.tries = 0; }
  start() {
    setInterval(() => { this.tries++; }, 1000); // arrow: `this` is the instance
    // setInterval(function () { this.tries++; }, 1000); // broken: `this` is Timeout
  }
}

```

Output

```

Order 7: 2 items
Order 7: 2 items
Order 7: 2 items
Order 7: 2 items
Order undefined
820

```

Why Interviewers Ask This

this confusion causes a large share of real JavaScript bugs, and the arrow-versus-method distinction is a precise, checkable piece of knowledge.

Common Mistakes

- Using an arrow function as an object or prototype method that needs this.
- Believing bind can be applied twice to rebound - the first wins.
- Forgetting that in a module or strict mode, default this is undefined, not globalThis.
- Using self = this in modern code instead of an arrow.

Best Practices

- Arrows for callbacks and closures; normal methods on objects and classes.
- Bind in the constructor or use class fields (handle = () => {}) for React class handlers.
- Prefer passing explicit parameters over relying on dynamic this.

Follow-up Questions

1. Write a polyfill for Function.prototype.bind.
2. What is this inside a class field arrow function versus a prototype method?
3. How does this behave in a setTimeout callback in non-strict mode?
4. Why can bind not be undone?

Real-world Scenario

A React class component passed `this.handleSubmit` directly to `onSubmit`. It worked in development because an unrelated HOC happened to bind it, and broke in production after the HOC was removed - "cannot read `setState` of undefined" on every submit. Class-field arrow handlers removed the whole category.

JS Q4

Difficulty: Hard

Category: JavaScript -> Event loop, microtasks

Question

Explain the browser event loop, the call stack, macrotasks and microtasks. Predict the output.

Answer

JavaScript has one call stack. The event loop repeatedly: takes one macrotask from the task queue, runs it to completion, then drains the entire microtask queue, then lets the browser render if needed.

- Macrotasks: `setTimeout`, `setInterval`, DOM events, network callbacks, `MessageChannel`.

- Microtasks: `promise` callbacks, `queueMicrotask`, `MutationObserver`.

The critical asymmetry: all pending microtasks run before the next macrotask, and microtasks queued by microtasks are also drained in the same pass. An infinite microtask chain therefore freezes the page permanently, while an infinite `setTimeout` chain does not.

`await x` is sugar for `x.then(continuation)`, so code after `await` is a microtask.
`requestAnimationFrame` runs before paint, after microtasks.

Example

The standard ordering puzzle.

Code Example

```
console.log("1 script start");

setTimeout(() => console.log("6 timeout"), 0);

Promise.resolve()
  .then(() => console.log("3 promise 1"))
  .then(() => console.log("4 promise 2"));

queueMicrotask(() => console.log("5 microtask"));

(async function main() {
  console.log("2 async body runs synchronously up to the first await");
```

```

    await null; // everything after this is a microtask
    console.log("3.5 after await");
  })();

  console.log("1.5 script end");

  // Microtask starvation demo - DO NOT run in a real page
  // function starve() { Promise.resolve().then(starve); } // page freezes, no paint
  // function safe() { setTimeout(safe, 0); } // yields; page stays responsive

```

Output

```

1 script start
2 async body runs synchronously up to the first await
1.5 script end
3 promise 1
3.5 after await
5 microtask
4 promise 2
6 timeout

```

Ordering note: the first `.then`, the `await` continuation, and `queueMicrotask` were queued in that order; promise 2 only queues after promise 1 resolves, so it lands after them.

Why Interviewers Ask This

It is the definitive async-model question for frontend roles. It also explains real symptoms - why a busy promise chain can block rendering.

Common Mistakes

- Saying `setTimeout(fn, 0)` runs "immediately".
- Treating promises as macrotasks.
- Not knowing the async function body runs synchronously until the first `await`.
- Believing async code runs on another thread.

Best Practices

- Yield to the loop with `setTimeout` or `scheduler.yield()` when processing large batches.
- Keep long tasks under 50 ms to protect INP.
- Use `requestIdleCallback` for non-urgent work, `Web Workers` for CPU-heavy work.

Follow-up Questions

1. Why can microtasks starve rendering but timers cannot?
2. Where does `requestAnimationFrame` fit relative to microtasks?
3. How does the Node event loop differ from the browser's?
4. What does `await` desugar to exactly?

Real-world Scenario

A grid component validated 5,000 rows in a promise chain with no yielding. Because every continuation was a microtask, the browser never got to paint and the tab appeared frozen for

four

seconds with no spinner. Chunking the work with `setTimeout` between batches let the spinner render and cut perceived latency dramatically.

JS Q5

Difficulty: Medium

Category: JavaScript -> Promises, async/await

Question

Compare `Promise.all`, `allSettled`, `race` and `any`. How do you handle partial failure?

Answer

Combinator	Resolves when	Rejects when	Use for
all	every promise fulfils	first rejection (fail fast)	all-or-nothing dependencies
allSettled	every promise settles	never	independent tasks, partial success
race	first promise settles	first settles as rejection	timeouts
any	first fulfilment	all reject (AggregateError)	redundant sources, failover

`Promise.all` rejecting does not cancel the other promises - they keep running and their results are discarded, so unhandled rejections can still surface. Use `AbortController` for real cancellation.

Sequential `await` in a loop is the most common performance bug: three 200 ms calls take 600 ms sequentially and 200 ms with `all`.

Example

A dashboard needing profile, orders and recommendations, where recommendations are optional.

Code Example

```
// SLOW: sequential, 600ms
const profile = await getProfile(id);
const orders = await getOrders(id);
const recs    = await getRecs(id);

// FAST: parallel, ~200ms - independent calls should always be concurrent
const [p, o, r] = await Promise.all([getProfile(id), getOrders(id), getRecs(id)]);

// PARTIAL FAILURE: render the page even if recommendations are down
const results = await Promise.allSettled([getProfile(id), getOrders(id), getRecs(id)]);
const [prof, ord, recs] = results.map(x => x.status === "fulfilled" ? x.value : null);
results.filter(x => x.status === "rejected")
  .forEach(x => log.warn({ err: x.reason }, "dashboard widget failed"));

// TIMEOUT with real cancellation
async function fetchWithTimeout(url, ms = 3000) {
  const ac = new AbortController();
  const timer = setTimeout(() => ac.abort(new Error("TIMEOUT")), ms);
```



```

try {
  const res = await fetch(url, { signal: ac.signal });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
} finally {
  clearTimeout(timer); // always clear, or the timer leaks
}

// FAILOVER across mirrors: first success wins
const data = await Promise.any([
  fetch("https://cdn1.example.com/data.json").then(r => r.json()),
  fetch("https://cdn2.example.com/data.json").then(r => r.json())
]);

// BOUNDED concurrency: 500 items, at most 10 in flight
async function mapLimit(items, limit, fn) {
  const out = new Array(items.length);
  let next = 0;
  await Promise.all(Array.from({ length: limit }, async () => {
    while (next < items.length) {
      const i = next++;
      out[i] = await fn(items[i], i);
    }
  }));
  return out;
}

```

Output

```

sequential : 612ms
Promise.all: 204ms

allSettled with recommendations down:
  profile OK, orders OK, recs null
  WARN dashboard widget failed: HTTP 503 // page still renders

Promise.any with cdn1 failing: resolves from cdn2
Promise.all with cdn1 failing: rejects immediately, cdn2 request still completes in background

```

Why Interviewers Ask This

Sequential awaits in a loop are extremely common in real code, and knowing `allSettled` versus `all` shows you have thought about graceful degradation.

Common Mistakes

- await inside a for loop for independent work.
- Assuming `Promise.all` cancels siblings on rejection.
- Using `race` for failover, where a fast rejection wins - any is correct.
- Forgetting `try/finally` to clear timers.
- Unbounded `Promise.all` over 10,000 items, exhausting sockets or rate limits.

Best Practices

- Parallelise independent work; keep `await` in a loop only for genuinely sequential steps.
- `allSettled` for widgets and fan-out where partial results are useful.
- Always attach timeouts and `AbortController` to network calls.

- Cap concurrency explicitly for bulk work.

Follow-up Questions

1. Why does Promise.all not cancel the other promises?
2. Implement Promise.all from scratch.
3. Difference between race and any when the first settle is a rejection?
4. How do you retry with exponential backoff and jitter?
5. What is an unhandled rejection and how do you catch it globally?

Real-world Scenario

A checkout page awaited six independent service calls sequentially, totalling 2.8 seconds. Users abandoned at 8 percent above baseline. Wrapping them in Promise.all with allSettled for the two optional widgets brought it to 480 ms and recovered most of the abandonment.

JS Q6

Difficulty: Medium

Category: JavaScript -> Debounce, throttle

Question

Implement debounce and throttle. When do you use each?

Answer

Both limit how often a function runs, differently:

- Debounce - wait until activity stops for delay ms, then run once. Use for search input, autosave, validation, resize completion.
- Throttle - run at most once per interval, ignoring calls in between. Use for scroll, mousemove, drag, analytics beacons, rate-limited APIs.

Mnemonic: debounce fires after the storm; throttle fires during it, at a fixed rate.

Production versions need cancel, flush, correct this, and the return value handled.

Example

Search-as-you-type (debounce) versus infinite-scroll position tracking (throttle).

Code Example

```
function debounce(fn, delay = 300, { leading = false } = {}) {
  let timer = null, lastArgs = null;
  function debounced(...args) {
    lastArgs = args;
    const callNow = leading && timer === null;
    clearTimeout(timer);
  }
```

```

    timer = setTimeout(() => {
      timer = null;
      if (!leading) fn.apply(this, lastArgs);
    }, delay);
    if (callNow) fn.apply(this, args);
  }
  debounced.cancel = () => { clearTimeout(timer); timer = null; };
  debounced.flush = function () { if (timer) { clearTimeout(timer); timer = null;
    fn.apply(this, lastArgs); } };

  return debounced;
}

function throttle(fn, interval = 200, { trailing = true } = {}) {
  let last = 0, timer = null, lastArgs = null;
  return function throttled(...args) {
    const now = Date.now();
    const remaining = interval - (now - last);
    lastArgs = args;
    if (remaining <= 0) {
      last = now;
      clearTimeout(timer); timer = null;
      fn.apply(this, args);
    } else if (trailing && timer === null) {
      // guarantee the final call is not lost
      timer = setTimeout(() => { last = Date.now(); timer = null;
        fn.apply(this, lastArgs); }, remaining);
    }
  };
}

// usage
const search = debounce(q => fetch(`/api/search?q=${q}`), 300);
input.addEventListener("input", e => search(e.target.value));

const onScroll = throttle(() => console.log(window.scrollY), 200);
window.addEventListener("scroll", onScroll, { passive: true });

// In React, memoise so the debounced fn is not recreated each render,
// and cancel on unmount:
// const debounced = useMemo(() => debounce(save, 500), []);
// useEffect(() => () => debounced.cancel(), [debounced]);

```

Output

```

typing "ultrabook" (9 keystrokes in 800ms)
  no debounce : 9 requests
  debounce 300: 1 request  (fires 300ms after the last keystroke)

scrolling for 2 seconds (~120 scroll events)
  no throttle : 120 handler runs
  throttle 200: 10 handler runs (+1 trailing)

```

Why Interviewers Ask This

It is the most common "implement this utility" whiteboard task, and it tests closures, timers and this at once.

Common Mistakes

- Swapping the definitions.

- Losing the final call in throttle (no trailing edge) - the user's last scroll position never reported.
- Recreating the debounced function on every React render, so the timer resets and it never fires.
- Ignoring this and arguments by using an arrow wrapper.
- Not cancelling on unmount, causing a setState-after-unmount warning.

Best Practices

- Debounce user typing; throttle continuous streams.
- Always expose cancel and call it in cleanup.
- Use { passive: true } for scroll listeners.
- Consider requestAnimationFrame throttling for visual updates.

Follow-up Questions

1. Implement throttle using requestAnimationFrame.
2. What is leading versus trailing edge behaviour?
3. How do you debounce an async function and ignore stale responses?
4. When is IntersectionObserver better than a throttled scroll handler?

Real-world Scenario

An autocomplete fired a request per keystroke. At 40,000 daily users the search cluster took 12x necessary load and rate limiting began rejecting real queries. A 300 ms debounce plus request cancellation cut search traffic by 92 percent with no perceived latency change.

JS Q7

Difficulty: Medium

Category: JavaScript -> Prototypes

Question

Explain the prototype chain. How does class relate to it?

Answer

Every object has an internal `[[Prototype]]` link. Property lookup walks that chain until found or until null. `Object.getPrototypeOf(obj)` reads it; the legacy accessor is `proto`.

For a constructor `F`, `new F()` creates an object whose prototype is `F.prototype`. Methods live on `F.prototype`, shared by all instances - that is the memory win over per-instance functions.

class is syntactic sugar over this: it creates a constructor function, puts methods on prototype, and wires extends by setting the prototype chain. Differences from functions: class bodies are strict mode, not hoisted for use before definition, methods are non-enumerable, and the

constructor cannot be called without new.

```
instance --[[Prototype]]--> Order.prototype --> Object.prototype --> null
```

Example

Prototype chain with class and its ES5 equivalent side by side.

Code Example

```
class Order {
  #internalNote = "private";           // true private field
  static count = 0;
  constructor(id) { this.id = id; Order.count++; }
  describe() { return `Order ${this.id}`; } // on Order.prototype, shared
  get label() { return `#${this.id}`; }
}
class RushOrder extends Order {
  constructor(id, fee) { super(id); this.fee = fee; }
  describe() { return `${super.describe()} (rush +${this.fee})`; }
}

const r = new RushOrder(7, 199);
console.log(r.describe());
console.log(Object.getPrototypeOf(r) === RushOrder.prototype);
console.log(Object.getPrototypeOf(RushOrder.prototype) === Order.prototype);
console.log(r instanceof Order, Order.count);
console.log(Object.keys(r));           // methods are NOT enumerable

// ES5 equivalent of the same wiring
function OrderES5(id) { this.id = id; }
OrderES5.prototype.describe = function () { return "Order " + this.id; };
function RushES5(id, fee) { OrderES5.call(this, id); this.fee = fee; }
RushES5.prototype = Object.create(OrderES5.prototype);
RushES5.prototype.constructor = RushES5;

// hasOwnProperty vs inherited
console.log(r.hasOwnProperty("describe"), "describe" in r);
```

Output

```
Order 7 (rush +199)
true
true
true 1
[ 'id', 'fee' ]
false true
```

Why Interviewers Ask This

Prototypes are what makes JavaScript's object model different from Java's, and class hides them. Interviewers check that you know what the sugar expands to.

Common Mistakes

- Saying JavaScript "has real classes now" - it has prototypal inheritance with class syntax.
- Confusing `F.prototype` (used to build instances) with `instance.proto` (the link itself).

- Adding methods inside the constructor, creating a copy per instance.
- Mutating `Object.prototype` - breaks every object and every `for...in`.
- Forgetting `super()` before using this in a derived constructor.

Best Practices

- Use class syntax; reserve manual prototype work for polyfills.
- Prefer composition over deep inheritance chains.
- `#private` fields over underscore conventions.
- `Object.create(null)` for pure dictionaries with no inherited keys.

Follow-up Questions

1. Difference between `proto` and `prototype`?
2. How does `instanceof` actually work?
3. What is prototypal versus classical inheritance?
4. Why are class methods non-enumerable, and when does that matter?
5. How would you implement `Object.create` yourself?

Real-world Scenario

A utility library added a helper to `Array.prototype` without `Object.defineProperty`. Every `for...in` over arrays in the host app suddenly yielded the method name, corrupting a serialiser that had shipped for years. The fix was a standalone function; the lesson was never to extend built-in prototypes.

SECTION 13: DATA STRUCTURES AND ALGORITHMS

Every solution shows the brute-force approach first, then the optimised one, with complexity.

DSA Q1

Difficulty: Easy

Category: DSA -> Complexity analysis

Question

Explain Big-O, Big-Theta and Big-Omega. Why do we drop constants?

Answer

They bound how cost grows with input size n . Big-O is an upper bound (worst case), Big-Omega a lower bound (best case), Big-Theta a tight bound (both). Interviews say "O" but usually mean Theta of the worst case.

Constants and lower-order terms are dropped because they do not affect growth: $3n^2 + 500n + 9$ is $O(n^2)$ because for large n the quadratic term dominates. Constants still matter in practice - an $O(n)$ pass with terrible cache locality can lose to an $O(n \log n)$ one at realistic sizes - and saying so is a strong signal.

Growth	$n = 10$	$n = 1,000$	$n = 1,000,000$
-----	-----	-----	-----
$O(1)$	1	1	1
$O(\log n)$	3	10	20
$O(n)$	10	1,000	1,000,000
$O(n \log n)$	33	10,000	20,000,000
$O(n^2)$	100	1,000,000	10^{12} (too slow)
$O(2^n)$	1,024	overflow	overflow

Rule of thumb for coding rounds: about 10^8 simple operations per second. $n \leq 20$ suggests exponential or backtracking is acceptable; $n \leq 10^3$ allows $O(n^2)$; $n \leq 10^6$ demands $O(n \log n)$ or better.

Example

Amortised analysis of a dynamic array push: most pushes are $O(1)$, a resize is $O(n)$, but doubling

makes the average $O(1)$.

Code Example

```
#  $O(n^2)$  - nested dependent loops
def has_duplicate_slow(nums):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                return True
    return False

#  $O(n)$  time,  $O(n)$  space - trade memory for time
def has_duplicate_fast(nums):
    seen = set()
    for x in nums:
        if x in seen:           # average  $O(1)$  hash lookup
            return True
        seen.add(x)
    return False

#  $O(n \log n)$  time,  $O(1)$  extra space - when memory is the constraint
def has_duplicate_sorted(nums):
    nums = sorted(nums)        #  $O(n \log n)$ 
    return any(nums[i] == nums[i+1] for i in range(len(nums)-1))

# Space complexity of recursion = call stack depth
def sum_recursive(n):          #  $O(n)$  time,  $O(n)$  SPACE - stack frames
    return 0 if n == 0 else n + sum_recursive(n - 1)

def sum_iterative(n):          #  $O(n)$  time,  $O(1)$  space
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

Output

```
n = 20,000 random ints
has_duplicate_slow : 4.81 s
has_duplicate_fast : 0.002 s
has_duplicate_sorted: 0.004 s

sum_recursive(100000) -> RecursionError: maximum recursion depth exceeded
sum_iterative(100000) -> 5000050000
```

Why Interviewers Ask This

Every algorithm answer must end with a complexity statement. Candidates who cannot analyse their own solution cannot be trusted to choose between two designs.

Common Mistakes

- Ignoring space complexity, especially recursion stack depth.
- Saying a hash lookup is "always $O(1)$ " - worst case is $O(n)$ with adversarial collisions.
- Forgetting that slicing or copying inside a loop adds a hidden factor of n .
- Confusing best case with average case for quicksort.
- Analysing only the loop structure and missing the cost of built-in calls.

Best Practices

- State time and space separately, and name the case (worst, average, amortised).
- Use the operations-per-second heuristic to infer the intended complexity from constraints.
- Mention constant factors and cache behaviour when they matter.

Follow-up Questions

1. Why is dynamic array append amortised $O(1)$?
2. Worst-case complexity of hash map lookup and how it is mitigated?
3. What is the difference between amortised and average case?
4. Given $n \leq 10^5$, which complexities are acceptable?

Real-world Scenario

A deduplication job used a nested loop over 60,000 records - fine on the 500-record test fixture,
36 minutes in production. Replacing it with a set made it 40 ms. The bug was never in the logic; it was the missing complexity analysis at review time.

DSA Q2

Difficulty: Easy

Category: DSA -> Two pointers

Question

Given a sorted array and a target, find two numbers summing to the target. Then solve it for an unsorted array.

Answer

Sorted: two pointers from both ends. If the sum is too small move left right; too large move right left. $O(n)$ time, $O(1)$ space - each step eliminates one candidate permanently.

Unsorted: a hash map of value to index in one pass. $O(n)$ time, $O(n)$ space. Sorting first would cost $O(n \log n)$ and lose the original indices.

Brute force is $O(n^2)$ for both. The pattern generalises: two pointers work whenever the array is sorted and the predicate is monotonic.

Example

[1, 3, 4, 6, 8, 11], target 14 -> indices 3 and 4 (6 + 8).

Code Example

```
# --- brute force: O(n^2) time, O(1) space ---
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

# --- sorted input: two pointers, O(n) time, O(1) space ---
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        if s < target:
            left += 1          # need a bigger sum
        else:
            right -= 1        # need a smaller sum
    return []

# --- unsorted input: hash map, O(n) time, O(n) space ---
def two_sum_hash(nums, target):
    seen = {}                # value -> index
    for i, x in enumerate(nums):
        want = target - x
        if want in seen:
            return [seen[want], i]
        seen[x] = i
    return []

# --- generalisation: three sum, O(n^2) after sorting ---
def three_sum(nums):
    nums.sort()
    out = []
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i-1]:
            continue          # skip duplicate anchors
        lo, hi = i + 1, len(nums) - 1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0: lo += 1
            elif s > 0: hi -= 1
            else:
                out.append([nums[i], nums[lo], nums[hi]])
                while lo < hi and nums[lo] == nums[lo+1]: lo += 1
                while lo < hi and nums[hi] == nums[hi-1]: hi -= 1
                lo += 1; hi -= 1
    return out
```

Output

```
two_sum_sorted([1,3,4,6,8,11], 14) -> [3, 4]
two_sum_hash([8,3,11,1,6,4], 14) -> [0, 4]
three_sum([-1,0,1,2,-1,-4]) -> [[-1,-1,2], [-1,0,1]]

timing on n = 20,000, worst case
brute : 3.94 s
hash  : 0.004 s
```

Why Interviewers Ask This

It is the canonical warm-up, and the follow-up ("what if it is not sorted?", "what about three numbers?") tests whether you can adapt a pattern rather than recall a solution.

Common Mistakes

- Sorting when indices must be returned from the original order.
- Using the same element twice ($\text{left} == \text{right}$).
- Adding the current number to the map before checking for its complement, so $\text{target} = 2 \times x$ matches an element with itself.
- Forgetting duplicate skipping in three-sum, producing repeated triplets.

Best Practices

- Clarify up front: sorted or not, indices or values, duplicates allowed, exactly one solution?
- State the space-time trade-off explicitly - it is the point of the question.
- Two pointers whenever the input is sorted; hashing when it is not.

Follow-up Questions

1. What if the array is sorted but you must return values, not indices?
2. Extend to four-sum - what is the complexity?
3. What if the array is enormous and does not fit in memory?
4. How would you find the pair whose sum is closest to the target?

Real-world Scenario

A reconciliation tool matched payment amounts to invoice amounts with a nested loop over 80,000 records each - 6.4 billion comparisons, hours of runtime. A hash map keyed by amount reduced it to a single pass finishing in under two seconds.

DSA Q3

Difficulty: Medium

Category: DSA -> Sliding window

Question

Find the length of the longest substring without repeating characters.

Answer

Brute force checks every substring for uniqueness: $O(n^3)$, or $O(n^2)$ with an incremental set.

The sliding window solution is $O(n)$: expand right one character at a time; when a duplicate

appears, move left forward past the previous occurrence. Each index enters and leaves the window at most once, so the total work is linear. Store the last index of each character to jump left directly instead of stepping.

Sliding window applies whenever you need the best contiguous subarray satisfying a constraint that is monotone as the window grows.

Example

"abcabcbb" -> "abc", length 3. "pwwkew" -> "wke", length 3.

Code Example

```
# --- brute force: O(n^2) time, O(min(n, charset)) space ---
def longest_unique_brute(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best

# --- sliding window: O(n) time, O(min(n, charset)) space ---
def longest_unique(s):
    last = {} # char -> last index seen
    best = left = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1 # jump past the duplicate, never backwards
        last[ch] = right
        best = max(best, right - left + 1)
    return best

# --- variant: longest substring with at most k distinct characters ---
def longest_k_distinct(s, k):
    from collections import defaultdict
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k: # shrink until the constraint holds
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

# --- fixed-size window: max sum of any k consecutive elements ---
def max_sum_window(nums, k):
    window = sum(nums[:k])
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k] # O(1) slide instead of re-summing
        best = max(best, window)
    return best
```

Output

```
longest_unique("abcabcbb")    -> 3
longest_unique("bbbbbb")     -> 1
longest_unique("pwwkew")     -> 3
longest_unique("")           -> 0
longest_k_distinct("eceba", 2) -> 3      # "ece"
max_sum_window([2,1,5,1,3,2], 3) -> 9    # [5,1,3]

n = 100,000 characters
brute : 12.3 s
window: 0.02 s
```

Why Interviewers Ask This

Sliding window is one of the highest-frequency patterns in coding rounds, and the left pointer must never move backwards - a subtle correctness detail interviewers watch for.

Common Mistakes

- Moving left backwards when the duplicate is outside the current window - hence the `last[ch] >= left` guard.
- Using `left = last[ch] + 1` without that guard, which breaks on inputs like "abba".
- Recomputing the window sum from scratch, losing the $O(1)$ slide.
- Off-by-one in the length right - `left + 1`.
- Not handling the empty string.

Best Practices

- Decide first whether the window is fixed-size or variable-size.
- For variable windows, expand with right and shrink with a while on the constraint.
- Track only what the constraint needs - a count map, not the substring itself.

Follow-up Questions

1. Trace "abba" and explain why the `>= left` guard is required.
2. Return the substring itself rather than its length.
3. Solve "minimum window substring containing all characters of T".
4. How does this change for a Unicode string with combining characters?

Real-world Scenario

A fraud rule needed the longest run of distinct device ids per session over a 400-million-event table. The first implementation was $O(n^2)$ per session and never finished. A sliding window in a streaming job processed the whole day in eleven minutes.

DSA Q4

Difficulty: Medium

Category: DSA -> Trees, BFS and DFS

Question

Compare BFS and DFS on a binary tree. Implement level-order traversal and validate a BST.

Answer

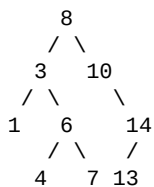
	BFS (level order)	DFS (pre/in/post order)
-----	-----	-----
Data structure	queue	stack or recursion
Space	$O(w)$, w = max width	$O(h)$, h = height
Finds shortest path	yes, in unweighted graphs	no
Natural for	level grouping, nearest node	path problems, subtree aggregation

For a balanced tree, height is $O(\log n)$ and width is $O(n/2)$, so DFS uses less memory. For a degenerate (linked-list-shaped) tree, DFS recursion is $O(n)$ deep and can overflow the stack while

BFS holds only one node per level.

BST validation must compare against an inherited range, not just the parent: a node in the left subtree of the root must be smaller than the root, not merely smaller than its immediate parent. Alternatively, an in-order traversal of a valid BST is strictly increasing.

Example



Code Example

```
from collections import deque

class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

# --- BFS level order: O(n) time, O(w) space ---
def level_order(root):
    if not root:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
```

```

        level.append(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    out.append(level)
    return out

# --- DFS in-order, recursive: O(n) time, O(h) space ---
def inorder(root, acc=None):
    acc = [] if acc is None else acc
    if root:
        inorder(root.left, acc)
        acc.append(root.val)
        inorder(root.right, acc)
    return acc

# --- DFS iterative, avoids stack overflow on deep trees ---
def inorder_iterative(root):
    out, stack, cur = [], [], root
    while cur or stack:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        out.append(cur.val)
        cur = cur.right
    return out

# --- BST validation with an inherited range: the CORRECT approach ---
def is_valid_bst(node, low=float("-inf"), high=float("inf")):
    if not node:
        return True
    if not (low < node.val < high):
        return False
    return (is_valid_bst(node.left, low, node.val) and
            is_valid_bst(node.right, node.val, high))

# --- WRONG approach that passes many tests: only compares with the parent ---
def is_valid_bst_wrong(node):
    if not node: return True
    if node.left and node.left.val >= node.val: return False
    if node.right and node.right.val <= node.val: return False
    return is_valid_bst_wrong(node.left) and is_valid_bst_wrong(node.right)

# --- max depth and diameter, both DFS aggregations ---
def max_depth(root):
    return 0 if not root else 1 + max(max_depth(root.left), max_depth(root.right))

root = Node(8, Node(3, Node(1), Node(6, Node(4), Node(7))), Node(10, None, Node(14, Node(13))))

```

Output

```

level_order(root)      -> [[8], [3, 10], [1, 6, 14], [4, 7, 13]]
inorder(root)          -> [1, 3, 4, 6, 7, 8, 10, 13, 14]      # sorted => valid BST
max_depth(root)        -> 4
is_valid_bst(root)     -> True

# the counter-example that exposes the wrong implementation
bad = Node(8, Node(3, Node(1), Node(20)), Node(10))
is_valid_bst(bad)       -> False      # correct: 20 > 8 but sits in the left subtree
is_valid_bst_wrong(bad) -> True       # WRONG

```

Why Interviewers Ask This

Trees are the most common structure in interviews, and the BST validation counter-example is a classic trap that rewards careful reasoning over recall.

Common Mistakes

- Validating a BST by comparing only with the direct parent.
- In BFS, not capturing `len(q)` before the inner loop, so levels merge.
- Using recursion on a possibly degenerate tree and overflowing the stack.
- Confusing height (edges) with depth (nodes) - clarify the definition.
- Forgetting duplicate handling: is $\leq$ allowed in a BST? Ask.

Best Practices

- BFS with an explicit queue and per-level sizing; DFS iteratively when depth may be large.
- Pass a range down for BST-style validation, or verify in-order is increasing.
- Handle the empty tree first, always.

Follow-up Questions

1. Produce the counter-example tree yourself and explain why it fools the naive check.
2. Give the space complexity of BFS versus DFS for balanced and degenerate trees.
3. How do you serialise and deserialise a binary tree?
4. Find the lowest common ancestor in a BST and in a general binary tree.
5. How does level-order help print a tree right-to-left per level?

Real-world Scenario

A permissions service validated its role hierarchy with the parent-only check. A misconfigured import placed a high-privilege role under a low-privilege branch; the validator passed and the tree-based lookup granted admin rights to a support tier for three weeks. Range-based validation caught it immediately once deployed.

SECTION 5: SYSTEM DESIGN

SD Q1

Difficulty: Medium

Category: System Design -> Scalability fundamentals

Question

Explain vertical versus horizontal scaling, and the difference between latency and throughput.

Answer

Vertical scaling (scale up) means a bigger machine. Simple, no code changes, no distributed problems - but there is a hardware ceiling, cost rises non-linearly, and it remains a single point of failure.

Horizontal scaling (scale out) means more machines behind a load balancer. Near-unbounded and fault tolerant, but it forces you to solve state: sessions, caching, data partitioning, consistency.

Latency is time per operation (measure p50, p95, p99 - never the mean, which hides tail behaviour). Throughput is operations per second. They are related but not the same: batching raises throughput while raising latency.

Little's Law ties them together: $\text{concurrency} = \text{throughput} \times \text{latency}$. To serve 5,000 requests per second at 200 ms each you need 1,000 concurrent slots.

Availability arithmetic: three nines is about 8.8 hours of downtime a year, four nines about 53 minutes, five nines about 5 minutes. Components in series multiply, so five 99.9 percent services in a request path give roughly 99.5 percent.

Example

Capacity estimate for a service at 10 million daily active users.

Code Example

Assumptions

DAU	10,000,000	
requests per user/day	20	
total requests/day	200,000,000	
average QPS	$200e6 / 86,400$	$\approx 2,300$
peak factor	3x	$\approx 7,000$ QPS
payload	2 KB avg	

Server sizing

per-instance capacity	1,000 QPS (measured, not guessed)
instances needed	$7,000 / 1,000 = 7 \rightarrow 10$ with headroom + AZ spread

Storage (1 KB per record, 3 replicas, 2 years)

200e6 records/day x 1 KB	= 200 GB/day
x 365 x 2	= 146 TB
x 3 replicas	= 438 TB
+ 30% index/overhead	≈ 570 TB

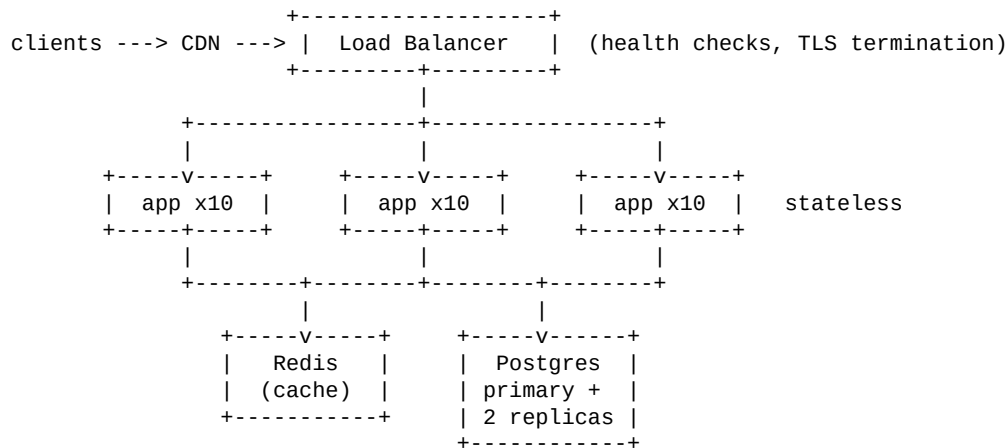
Bandwidth

7,000 QPS x 2 KB	= 14 MB/s = 112 Mbps egress
------------------	-----------------------------

Cache sizing (80/20 rule)

hot 20% of daily records	= $40e6 \times 1$ KB = 40 GB
--------------------------	------------------------------

-> a 64 GB Redis node with room to grow



Output

Result of the estimate

- 10 app instances across 3 availability zones
- ~ 570 TB storage over 2 years -> partition by month, archive to object storage after 90 days
- 64 GB cache serving the hot 20%
- target p99 < 250 ms, availability 99.95%

Why Interviewers Ask This

Every system design interview opens here. They want to see you quantify before you draw boxes, and that you reason about p99 rather than averages.

Common Mistakes

- Drawing an architecture before estimating load.
- Quoting average latency instead of percentiles.
- Assuming horizontal scaling is free - it moves the problem to state and consistency.
- Ignoring the peak-to-average ratio.
- Forgetting replication and index overhead in storage maths.

Best Practices

- State assumptions out loud and write them down; interviewers grade the reasoning.
- Round aggressively - 2,300 QPS not 2,314.
- Keep application servers stateless so scaling out is trivial.
- Design for the peak, autoscale for the trough, and load test to find real per-instance capacity.

Follow-up Questions

1. What breaks first as you scale out - and why is it usually the database?
2. How does Little's Law inform thread pool and connection pool sizing?
3. Why does availability degrade when you add services to a request path?
4. When is vertical scaling the right answer?

Real-world Scenario

A team autoscaled the application tier to 60 instances under launch load. Each instance opened 20 database connections, so 1,200 connections hit a Postgres primary configured for 200; the database spent its time context switching and the whole system got slower as it scaled. A connection pooler capped at 150 total connections restored throughput with fewer instances.

SD Q2

Difficulty: Hard

Category: System Design -> CAP, consistency

Question

Explain the CAP theorem and what it actually forces you to choose. What is eventual consistency?

Answer

CAP says that during a network partition, a distributed system can preserve either consistency (every read sees the latest write) or availability (every request gets a

non-error response), not both. Partition tolerance is not optional in a real network, so the real choice is CP or AP while partitioned.

The common misstatement is "pick two of three". When there is no partition, a system can be both consistent and available; CAP only constrains behaviour during the partition.

PACELC extends it usefully: if Partitioned choose A or C; Else (normal operation) choose Latency or Consistency. This captures the real trade-off most systems make daily.

Eventual consistency means replicas converge if writes stop. Acceptable for like counts, timelines and product search; unacceptable for account balances and seat inventory. Middle grounds

worth naming: read-your-writes, monotonic reads, and causal consistency - usually enough for user-facing correctness.

System	Choice	Note
Postgres single primary	CP	writes stop if the primary is unreachable
DynamoDB, Cassandra	AP, tunable	quorum settings move it toward CP
MongoDB replica set	CP by default	w: majority
ZooKeeper, etcd	CP	consensus, used for coordination
DNS	AP	famously eventually consistent

Example

A shopping cart (AP is fine - merge concurrent edits) versus payment capture (CP required - never double-charge).

Code Example

```
CP behaviour during a partition
+-----+           X network split X           +-----+
| node A | <-----> | node B |
| leader |           | follower|
+-----+           +-----+
writes OK (has quorum)           reads may be refused or stale-flagged
-> minority side returns errors rather than serving stale data

AP behaviour during the same partition
both sides accept writes -> divergence -> conflict resolution on heal
strategies: last-write-wins (data loss), vector clocks, CRDTs (mergeable), app-level merge

// Tunable consistency, Cassandra-style: choose per query
// R + W > N gives strong consistency
// N = 3 replicas
// W=1, R=1 -> fast, eventually consistent      (1 + 1 < 3)
// W=2, R=2 -> strongly consistent              (2 + 2 > 3)
// W=3, R=1 -> fast reads, slow writes

// Read-your-own-writes without full strong consistency:
// route the user to the primary for a short window after their write
const writeAt = Date.now();
sessionStorage.setItem("lastWriteAt", writeAt);
const readPreference =
  Date.now() - Number(sessionStorage.getItem("lastWriteAt")) < 5000
    ? "primary"           // recent writer: read authoritative
```

```

    : "secondaryPreferred";

// Idempotency: the practical defence when you choose AP and must retry
await payments.updateOne(
  { idempotencyKey: key }, // unique index on this field
  { $setOnInsert: { orderId, amount, status: "CAPTURED", at: new Date() } },
  { upsert: true } // duplicate retries are no-ops
);

```

Output

```

Cart service (AP):    partition -> both regions accept adds -> merged on heal (union of items)
                     user impact: an item briefly missing, never an error

Payment service (CP): partition -> minority region rejects captures with 503
                     user impact: "try again", never a double charge

Idempotent capture: 4 retries of the same key -> 1 payment row, 3 no-ops

```

Why Interviewers Ask This

CAP is the vocabulary for every replication and multi-region question. Reciting the theorem is table stakes; the signal is mapping it to concrete product decisions - which features tolerate staleness and which do not.

Common Mistakes

- "Pick two of three" - CAP constrains behaviour only during a partition.
- Claiming a single-node database is CP (with no partition, CAP does not apply).
- Confusing the C in CAP (linearisability) with the C in ACID (constraint validity).
- Treating eventual consistency as a synonym for unreliable.
- Choosing strong consistency globally, then being surprised by cross-region write latency.

Best Practices

- Decide consistency per feature, not per system.
- Make every write idempotent so retries are safe under either choice.
- Use read-your-writes routing for the perceived-correctness cases.
- Prefer CRDTs or explicit merge over last-write-wins, which silently loses data.
- Design and test the partition path; it is the path that will page you.

Follow-up Questions

1. What is PACELC and why is it more useful day to day?
2. How do quorum reads and writes give tunable consistency?
3. What is a split brain and how does fencing prevent it?
4. Where do CRDTs fit, and give a concrete example.
5. How would you implement read-your-own-writes across regions?

Real-world Scenario

A multi-region inventory service used last-write-wins replication for "simplicity". A partition

between regions lasted nine minutes; both accepted decrements for the same SKUs, and on heal the later timestamp overwrote the other region's sales. About 1,800 units were oversold. The rebuild made the stock counter a CRDT-style counter with per-region reservations and a CP path for final allocation.

SECTION 4: COMPUTER NETWORKS

NET Q1

Difficulty: Easy

Category: Networks -> TCP vs UDP

Question

Compare TCP and UDP. When would you deliberately choose UDP?

Answer

	TCP	UDP
Connection	handshake first	connectionless
Delivery	guaranteed, ordered	best effort, may drop or reorder
Congestion control	yes	none (app must implement)
Header	20 bytes	8 bytes
Speed	slower, head-of-line blocking	lower latency
Used by	HTTP/1.1 and 2, SSH, SMTP	DNS, DHCP, VoIP, games, QUIC/HTTP3

TCP's three-way handshake (SYN, SYN-ACK, ACK) costs one round trip before any data, plus another one or two for TLS. UDP sends immediately.

Choose UDP when stale data is worthless: in a voice call, retransmitting a 300 ms-old packet is worse than dropping it. QUIC (the basis of HTTP/3) runs on UDP and rebuilds reliability per stream,

which removes TCP's head-of-line blocking - one lost packet no longer stalls every stream on the connection.

Example

A video call drops frames rather than freezing; a file download must not lose a byte.

Code Example

```
TCP connection establishment and teardown
client      server
| ----- SYN -----> |
```

```

| <----- SYN-ACK -----> |      1 RTT before any application data
| ----- ACK -----> |
| ===== HTTP request =====> |
| <===== HTTP response =====> |
| ----- FIN -----> |
| <----- FIN-ACK -----> |

```

Head-of-line blocking, HTTP/2 over TCP: one lost segment stalls ALL streams

```

stream1 [#####]
stream2 [###X###] <- packet loss here
stream3 [#####] <- delivered but BLOCKED behind stream2 at the transport layer

```

HTTP/3 over QUIC (UDP): streams are independent

```

stream2 loss affects only stream2

```

observe the handshake

```

tcpdump -n -i any 'tcp port 443 and (tcp[tcpflags] & (tcp-syn|tcp-ack) != 0)'

```

TCP: connection refused is a fast, explicit signal

```

curl -v --max-time 3 http://localhost:9999

```

UDP: no such signal - a DNS query just times out

```

dig @192.0.2.1 example.com +timeout=2

```

Output

```

14:02:11.104 IP 10.0.0.5.51234 > 93.184.216.34.443: Flags [S], seq 1829..
14:02:11.128 IP 93.184.216.34.443 > 10.0.0.5.51234: Flags [S.], seq 4021..
14:02:11.128 IP 10.0.0.5.51234 > 93.184.216.34.443: Flags [.], ack 4022..
# 24 ms consumed before the first request byte

```

```

curl: (7) Failed to connect to localhost port 9999: Connection refused
;; connection timed out; no servers could be reached

```

Why Interviewers Ask This

It is the foundational transport question and it leads naturally into HTTP/3, real-time systems and latency budgets.

Common Mistakes

- "UDP is unreliable so never use it" - it is the right choice for real-time media and DNS.
- Not knowing HTTP/3 runs on UDP.
- Claiming TCP guarantees delivery to the application - it guarantees to the kernel buffer; the app can still crash.
- Confusing flow control (receiver-driven) with congestion control (network-driven).

Best Practices

- TCP for correctness-critical transfer; UDP plus application-level recovery for real-time.
- Enable HTTP/2 or HTTP/3 and connection reuse to amortise handshakes.
- Set explicit connect and read timeouts on every client.
- Use TLS 1.3 (one round trip) and session resumption.

Follow-up Questions

1. What problem does QUIC solve that HTTP/2 could not?
2. Explain the TCP sliding window and slow start.
3. What is TIME_WAIT and why can it exhaust ports?
4. How does DNS use both UDP and TCP?

Real-world Scenario

A mobile game used TCP for position updates. On lossy cellular links a single dropped packet stalled the stream and players saw a two-second freeze, then a teleport. Moving positional data to

UDP with client-side interpolation made movement smooth even at 5 percent packet loss.

NET Q2

Difficulty: Medium

Category: Networks -> HTTPS, TLS

Question

What happens when you type a URL and press Enter? Where does TLS fit?

Answer

1. URL parse - scheme, host, port, path.
2. DNS resolution - browser cache, OS cache, /etc/hosts, then recursive resolver -> root -> TLD -> authoritative; result cached per TTL.
3. TCP handshake - SYN, SYN-ACK, ACK (one RTT). HTTP/3 skips this and uses QUIC over UDP.
4. TLS handshake - ClientHello (versions, ciphers, SNI), ServerHello plus certificate, certificate chain validation against the trust store, key exchange (ECDHE for forward secrecy), then symmetric keys. TLS 1.3 needs one RTT, or zero with resumption.
5. HTTP request - method, path, headers, cookies.
6. Server processing - load balancer, app, cache, database.
7. Response - status, headers, body; browser parses HTML, builds the DOM and CSSOM, runs JavaScript, paints.

TLS gives confidentiality (encryption), integrity (MAC/AEAD) and authentication (the certificate proves the server's identity). It does not hide which host you contacted - SNI is plaintext unless Encrypted Client Hello is in use.

Example

`https://shop.example.com/orders/7` from a cold browser cache.

Code Example

```
Timeline for a cold connection
DNS lookup      24 ms  +-----+
TCP handshake   28 ms  +-----+
TLS handshake   31 ms  +-----+
request sent    1 ms  +
server processing 92 ms  +-----+
response download 18 ms  +-----+
total          194 ms

Warm connection (keep-alive): ~110 ms - the first three steps disappear

# inspect the certificate chain and negotiated protocol
openssl s_client -connect example.com:443 -servername example.com -tls1_3 </dev/null \
| openssl x509 -noout -subject -issuer -dates

# measure each phase
curl -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} \
ttfb:%{time_starttransfer} total:%{time_total}\n" -o /dev/null -s https://example.com

# trace resolution
dig +trace example.com
```

Output

```
subject=CN = example.com
issuer=C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
notBefore=Jan 15 00:00:00 2026 GMT
notAfter=Apr 15 23:59:59 2026 GMT

dns:0.024 connect:0.052 tls:0.083 ttfb:0.175 total:0.194
```

Why Interviewers Ask This

It is the broadest possible systems question - it lets the interviewer follow whichever layer they care about, from DNS caching to render blocking.

Common Mistakes

- Skipping DNS or TLS entirely.
- Saying HTTPS "encrypts the URL" - the path is encrypted, but the hostname leaks via SNI and DNS.
- Believing TLS uses asymmetric encryption for the whole session; it is used only to establish symmetric keys.
- Not knowing certificate validation checks chain, hostname, validity dates and revocation.

Best Practices

- TLS 1.3, HSTS, OCSP stapling, and automated certificate renewal.
- Keep connections alive; use HTTP/2 or HTTP/3.
- Put static assets on a CDN close to users; preconnect and DNS-prefetch critical origins.
- Monitor certificate expiry as a first-class alert.

Follow-up Questions

1. What is SNI and why does it matter for shared hosting?
2. How does HSTS prevent SSL stripping?
3. What is forward secrecy and which key exchange provides it?
4. Difference between DNS A, AAAA, CNAME and ALIAS records?
5. What is a TLS termination point and what changes behind it?

Real-world Scenario

A payment gateway's intermediate certificate expired at 02:00 UTC. Browsers with a cached chain kept working, so monitoring stayed green, while fresh mobile clients failed TLS validation. Two hours of failed checkouts followed. The fix was automated renewal plus a synthetic check from a cold client that validates the full chain.

SECTION 14: OBJECT-ORIENTED PROGRAMMING

OOP Q1

Difficulty: Medium

Category: OOP -> SOLID

Question

Explain the SOLID principles with one concrete violation and fix each.

Answer

Principle	Statement	Smell it prevents
Single Responsibility	a class has one reason to change	god classes
Open/Closed	open to extension, closed to modific~	editing a switch for every new case
Liskov Substitution	a subtype must be usable wherever th~	subclass throwing on inherited methods
Interface Segregation	many small interfaces beat one fat o~	implementing methods you must stub out
Dependency Inversion	depend on abstractions, not concreti~	untestable code welded to a driver

The most-violated is Liskov: the classic Square extends Rectangle breaks callers that set width and height independently. Penguin extends Bird with fly() throwing is the same error.

Example

An order processor that formats invoices, charges cards and sends email - three reasons to change.

Code Example

```
// ===== VIOLATION: SRP + OCP + DIP in one class =====
class OrderProcessor {
    void process(Order o, String paymentType) {
        if (paymentType.equals("CARD"))    new StripeClient().charge(o);    // DIP: concrete
        else if (paymentType.equals("UPI")) new RazorpayClient().charge(o); // OCP: edit to extend
        String html = "<h1>Invoice " + o.getId() + "</h1>"; // SRP: formatting
        new Smtmailer().send(o.getEmail(), html); // SRP: delivery
    }
}
```

```
// ===== FIXED =====
interface PaymentGateway { PaymentResult charge(Order order); } // abstraction (DIP)
class StripeGateway implements PaymentGateway { public PaymentResult charge(Order o) {...} }
class RazorpayGateway implements PaymentGateway { public PaymentResult charge(Order o) {...} }
// adding a new gateway requires NO change to existing code (OCP)

interface InvoiceRenderer { String render(Order order); } // one responsibility
interface Notifier { void notify(String to, String body); } // one responsibility (ISP)

class OrderProcessor {
    private final PaymentGateway gateway; // injected, so it is testable (DIP)
    private final InvoiceRenderer renderer;
    private final Notifier notifier;

    OrderProcessor(PaymentGateway g, InvoiceRenderer r, Notifier n) {
        this.gateway = g; this.renderer = r; this.notifier = n;
    }

    void process(Order order) { // one reason to change: orchestration (SRP)
        PaymentResult result = gateway.charge(order);
        if (!result.isSuccess()) throw new PaymentFailedException(result.getError());
        notifier.notify(order.getEmail(), renderer.render(order));
    }
}

// ===== LISKOV violation and the fix =====
class Rectangle { void setWidth(int w){...} void setHeight(int h){...} int area(){...} }
class Square extends Rectangle { // BREAKS callers
    void setWidth(int w) { super.setWidth(w); super.setHeight(w); }
}
// caller assumes independence and now fails:
// r.setWidth(5); r.setHeight(4); assert r.area() == 20; // Square gives 16
// FIX: prefer composition; Shape interface with immutable Square(side) and Rectangle(w,h)
```

Output

```
// with injection, the unit test needs no network
new OrderProcessor(new FakeGateway(), new PlainRenderer(), new CapturingNotifier())
    .process(order);
=> assertions on the captured notification, 0 ms, no Stripe account required

// before: the same test needed live Stripe credentials and an SMTP server
```

Why Interviewers Ask This

SOLID is the shared vocabulary for code review at mid level and above. Reciting the acronym is worth little; identifying a violation in supplied code is the actual test.

Common Mistakes

- Listing the letters without concrete examples.
- Reading SRP as "one method per class".
- Confusing dependency inversion (depend on abstractions) with dependency injection (a technique for supplying them).
- Creating an interface per class reflexively - abstraction with one implementation and no seam is just indirection.
- Using inheritance for code reuse where composition fits.

Best Practices

- Introduce an interface when you have a second implementation or need a test seam - not before.
- Constructor injection; avoid service locators and static singletons.
- Favour composition over inheritance; keep hierarchies shallow.
- Let SOLID be a diagnostic for pain, not a checklist applied up front.

Follow-up Questions

1. Give a Liskov violation from your own experience.
2. How does the strategy pattern implement OCP?
3. When does an interface with a single implementation still pay for itself?
4. How do SOLID and DRY conflict?
5. What is the difference between DIP and DI?

Real-world Scenario

A billing class had 2,400 lines and 11 reasons to change. Every payment provider addition risked the invoice logic, and its tests needed live credentials so they were disabled in CI. Splitting it along the seams above took the test suite from zero effective coverage to 94 percent and cut new provider integration from two weeks to two days.

SECTION 16: OPERATING SYSTEMS

OS Q1

Difficulty: Easy

Category: OS -> Process vs thread

Question

Difference between a process and a thread? What is a context switch?

Answer

A process is an executing program with its own virtual address space, file descriptors and resources. A thread is a unit of execution inside a process; threads share the heap, code and descriptors but each has its own stack, registers and program counter.

	Process	Thread
Memory	isolated	shared heap, own stack
Creation cost	high (page tables, PCB)	low
Crash impact	contained	can take down the process
Communication	IPC: pipes, sockets, shared memory	shared variables (needs synchronisation)
Context switch	expensive (TLB and cache flush)	cheaper (same address space)

A context switch saves the current execution state (registers, program counter, stack pointer) and restores another's. Direct cost is a few microseconds; the indirect cost - cold CPU caches and

TLB misses - is usually larger. Excessive switching is thrashing, visible as high system CPU with low throughput.

Example

A browser uses one process per tab for isolation, and threads inside each tab for rendering, networking and compositing.

Code Example

```
// process: fork() gives a copy-on-write duplicate of the address space
pid_t pid = fork();
if (pid == 0) {
    printf("child pid=%d counter=%d\n", getpid(), counter++); // separate copy
    _exit(0);
} else {
    wait(NULL);
    printf("parent pid=%d counter=%d\n", getpid(), counter); // unchanged by the child
}

// threads: shared memory means a data race without synchronisation
#include <pthread.h>
long shared = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock); // remove this to see the race
        shared++; // read-modify-write is NOT atomic
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

// 4 threads x 100000 increments -> expected 400000

# observe context switches and their cost
vmstat 1 3 # 'cs' column = context switches per second
pidstat -w -p <pid> 1 # voluntary vs involuntary switches per process
```

Output

```
child pid=4821 counter=0
parent pid=4820 counter=0 # isolated address spaces

with mutex : shared = 400000 (correct, 41 ms)
without mutex: shared = 287341 (lost updates, 12 ms - fast and wrong)

procs -----memory----- ---system--
r b swpd free buff cache in cs
3 0 0 812344 91280 402118 1204 18422 <- 18k switches/s: investigate
```

Why Interviewers Ask This

It is the base of every concurrency conversation, and the "why is this fast but wrong" race is a compact demonstration of understanding.

Common Mistakes

- Saying threads have separate memory.
- Believing more threads always means more throughput - beyond core count, switching dominates.
- Not recognising counter++ as three operations (load, add, store).
- Confusing concurrency (interleaved progress) with parallelism (simultaneous execution).

Best Practices

- Size thread pools near core count for CPU-bound work; higher for I/O-bound.
- Prefer immutable data or message passing over shared mutable state.
- Use processes when isolation matters (untrusted plugins, crash containment).
- Watch involuntary context switches as a saturation signal.

Follow-up Questions

1. What is a zombie process and how do you prevent one?
2. Difference between user-level and kernel-level threads?
3. What is copy-on-write in `fork()`?
4. Why is a data race undefined behaviour rather than merely wrong?
5. How do green threads and virtual threads change this picture?

Real-world Scenario

An import service spawned one thread per file and processed 4,000 files concurrently on an 8-core box. `vmstat` showed 60,000 context switches a second with the CPU 70 percent in system time; throughput was lower than with 8 threads. A bounded pool of 16 workers tripled throughput.

SECTION 18: BEHAVIORAL AND HR

HR Q1

Difficulty: Easy

Category: Behavioral -> Self introduction

Question

Tell me about yourself.

Answer

This is not an invitation to narrate your CV. It is a 90-second positioning statement. Use Present - Past - Future:

1. Present (25s) - current role, scope, and one quantified outcome.
2. Past (35s) - the two or three experiences that qualify you for this role, chosen from the job description, not chronologically.
3. Future (20s) - why this company and this role, specifically, now.

Rules: lead with the strongest signal, quantify at least once, and end with a hook the interviewer

wants to pull on. Never recite dates, never mention family or hobbies unless asked, and never say "as you can see on my resume".

Example

Answers for two levels.

Code Example

--- Mid-level backend engineer (2-5 years) ---

"I'm a backend engineer with three years on Node and Postgres, currently at a logistics startup where I own the order pipeline - about 40,000 orders a day. My main piece of work last year was cutting our p99 checkout latency from 2.8 seconds to 400 milliseconds by parallelising service calls and fixing a deep-pagination query; that recovered around 8 percent of drop-off at checkout."

Before that I was at a services company doing Spring Boot integrations for banking clients, which is where I learned to treat idempotency and audit trails as requirements rather than extras.

I'm looking at your team because the job description mentions moving from a monolith to event-driven services - that's exactly the migration I ran last quarter, and I'd like to do it again with a team that has more scale than we do."

--- Fresher / intern ---

"I'm a final-year computer science student at [college], and my focus has been full-stack development. Most of what I know comes from building rather than coursework - my main project is a food-delivery app in the MERN stack that I put in front of real users through my college's food committee; about 200 students used it over a semester, and handling their concurrent orders is where I learned about race conditions and database transactions the hard way.

Alongside that I've solved around 400 DSA problems, mostly arrays, trees and dynamic programming, because I wanted the fundamentals to be solid and not just framework knowledge.

I'm applying here because your internship is backend-focused, and the systems problems you work on are the ones I've only been able to read about so far."

Output

Structure check	
length	80-100 seconds spoken
quantified facts	at least one number
tailoring	at least one reference to this specific role
hook	ends on something the interviewer will ask about

Why Interviewers Ask This

It sets the agenda. Whatever you emphasise is what they will probe, so you are effectively choosing the next ten minutes of the interview. It also tests whether you can structure information under mild pressure.

Common Mistakes

- Chronological life story starting from school.
- Five minutes long, or fifteen seconds long.
- Identical answer regardless of company - interviewers hear the template.
- Listing technologies with no outcome attached.
- Mentioning salary, visa, or notice period unprompted.
- Apologising for gaps or a lack of experience.

Best Practices

- Write it, time it, say it aloud twenty times until it is natural rather than memorised.
- Keep two versions: 90 seconds and 30 seconds.
- Plant one deliberate hook you are ready to discuss in depth.
- Update the last sentence for every company you apply to.

Follow-up Questions

1. Tell me more about that latency work - what did you measure first?
2. Why are you leaving your current role?
3. What would you have done differently on that project?

4. Walk me through the hardest technical decision you made there.

Real-world Scenario

A candidate opened with three minutes on their B.Tech coursework and mentioned a distributed tracing project only in passing at the end. The interviewer had already spent the budget on generic questions, and the one genuinely differentiating experience never got explored. The same candidate, reordered to lead with tracing, converted the next loop.

HR Q2

Difficulty: Medium

Category: Behavioral -> STAR method, failure

Question

Tell me about a time you failed.

Answer

They are testing ownership and learning, not looking for evidence against you. Use STAR: Situation (context, briefly), Task (your responsibility), Action (what you did), Result (outcome plus the lesson, ideally with a number).

Rules for a failure story specifically:

- Pick a real failure with real consequences. Fake ones ("I'm a perfectionist") read as evasion.
- Own your part without blaming teammates - even when others contributed.
- Spend most of the time on the recovery and the systemic fix, not the mistake.
- Close with evidence the lesson stuck: a process you changed that is still in place.
- Never choose a failure that reveals a disqualifying trait: missed deadlines through negligence, hiding problems from stakeholders, or dishonesty.

Example

A production incident told in STAR shape.

Code Example

- S "Two months into my current role I owned a migration adding a NOT NULL column to our orders table - about 30 million rows."
- T "I wrote the migration and was responsible for getting it to production safely."
- A "I tested it on a staging copy that had 50,000 rows, so it ran in under a second and I approved it for a Tuesday afternoon deploy. In production the ALTER took an ACCESS EXCLUSIVE lock and rewrote the whole table. Checkout blocked for 11 minutes. I noticed the alert, cancelled the migration, and confirmed rollback, then wrote the postmortem myself rather than waiting to be asked."
- R "Eleven minutes of failed checkouts, roughly 400 orders. What I changed: I rewrote it as

the three-phase pattern - add nullable, backfill in batches, then add the constraint with NOT VALID and validate separately. I also added a rule to our migration checklist that any DDL on a table over a million rows needs a timing test against a production-sized snapshot, and I set a lock_timeout so a migration fails fast instead of queueing behind traffic. That checklist item has caught two similar migrations since, including one from a senior engineer."

Output

```
Story quality checklist
real consequence stated      yes - 11 min, ~400 orders
personal ownership           yes - "I approved it"
no blame shifted             yes
systemic fix, not just "I'll be careful" yes - checklist + lock_timeout
evidence the fix worked      yes - caught two later migrations
total length                  ~90 seconds
```

Why Interviewers Ask This

Everyone fails; they are measuring self-awareness, whether you fix causes or symptoms, and whether you are safe to trust with production. Candidates who cannot name a failure are usually either inexperienced or not reflective.

Common Mistakes

- "I don't think I've really failed" - reads as arrogance or inexperience.
- Humble-brag failures ("I work too hard").
- Blaming QA, the previous team, or unclear requirements.
- Choosing something trivial with no consequence.
- Ending at the mistake with no lesson or systemic change.
- Rambling through the situation and running out of time before the result.

Best Practices

- Prepare four STAR stories that can be recut for many questions: a failure, a conflict, a leadership moment, and a technical deep dive.
- Keep Situation and Task to about 20 percent, Action and Result to 80 percent.
- Quantify the result; numbers make a story credible.
- Practise out loud and time it - 90 to 120 seconds.

Follow-up Questions

1. What would you do differently if it happened again tomorrow?
2. How did you tell your manager and the affected stakeholders?
3. Has that lesson changed how you review other people's work?
4. Tell me about a time you disagreed with your manager.
5. What is the biggest risk you have taken at work?

Real-world Scenario

Two candidates described the same class of incident. One said "the migration failed and we rolled back, I learned to be more careful". The other described the three-phase pattern, the lock_timeout, and the checklist that later caught a colleague's mistake. Only the second gave the interviewer any evidence that the failure produced durable change - and only the second got the offer.

BUILD STATUS AND ROADMAP

This book is written in batches. This edition contains 62 questions of a planned 1720. Nothing here is placeholder text - every question listed below is complete, with runnable code, expected output and follow-ups. The remaining sections are genuinely not written yet.

Section	Topic	Target	Written	Status
1	SQL	150	30	in progress
2	MERN Stack	120	13	in progress
3	Internship and Fresher	100	0	not started
4	Computer Networks	100	2	in progress
5	System Design	120	2	in progress
6	HTML	60	0	not started
7	CSS	100	0	not started
8	JavaScript	150	7	in progress
9	TypeScript	80	0	not started
10	C# and .NET	100	0	not started
11	Java	120	0	not started
12	Linux	100	0	not started
13	Data Structures and Algorithms	150	4	in progress
14	Object-Oriented Programming	60	1	in progress
15	DBMS	80	0	not started
16	Operating Systems	80	1	in progress
17	Software Engineering Practice	50	0	not started
18	Behavioral and HR	100	2	in progress
	Total	1720	62	3.6 percent

What is complete and usable today

SQL (Q1-30) is the most developed section and can be studied end to end: clause execution order, WHERE vs HAVING, aggregates and NULL semantics, three-valued logic, COALESCE and NULLIF, CASE and conditional pivoting, all five join types, the ON-vs-WHERE outer join trap, self joins, window function fundamentals, ROW_NUMBER vs RANK vs DENSE_RANK, LAG and LEAD, CTEs and materialisation, recursive CTEs with cycle guards, keys, set operators, keyset pagination, views and materialised views, deadlocks and lock ordering, normalisation 1NF to BCNF plus denormalisation, EXPLAIN plan reading, partitioning versus sharding, stored procedures, functions and triggers, GROUPING SETS and ROLLUP, indexes, ACID, isolation levels, and SQL injection. It includes a shared sample database, seven coding challenges with solutions, and two revision sheets.

MERN (Q1-13) covers MongoDB document modelling, the aggregation pipeline, indexes and the ESR

rule, replica sets with write concern and transactions, Express middleware and error handling, JWT authentication with refresh rotation, React state immutability, useEffect and race conditions, memoisation, the Virtual DOM and keys, state management choices, the Node event loop, and streams with backpressure.

Other sections have their opening questions written: JavaScript (hoisting, closures, this, event loop, promise combinators, debounce and throttle, prototypes), DSA (complexity, two pointers, sliding window, trees with BFS/DFS), System Design (capacity estimation, CAP and PACELC), Networks (TCP vs UDP, TLS and the URL walkthrough), OOP (SOLID), OS (process vs thread), Behavioral (self introduction, STAR failure story).

Still to be written

Sections 3, 6, 7, 9, 10, 11, 12, 15 and 17 have no questions yet: Internship and fresher interviews, HTML, CSS, TypeScript, C# and .NET, Java, Linux, DBMS, and software engineering practice. The nine in-progress sections need their remaining questions, per-section coding challenges, mini-project discussions and revision notes.

How to continue the build

Each batch appends complete question blocks to interview-1000-questions.md and regenerates the PDF. Nothing already written is rewritten, so question numbers are stable and can be cross referenced between batches.

```
python3 generate_interview_pdf.py          # rebuild the PDF after any edit
grep -c '^## ' interview-1000-questions.md # question and section-heading count
```